

Laboratorio 1: GPIO en ASM

Sofía Modernell, Guadalupe Sosa, Victoria Etcheverry

Carrera: Ingeniería en Mecatrónica, Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

victoria.etccheverry@estudiantes.utec.edu.uy, sofia.modernell@estudiantes.utec.edu.uy, guadalupe.sosa@estudiantes.utec.edu.uy

26 de septiembre de 2025

Resumen—En el presente laboratorio se abordó el diseño, programación y control de sistemas mecatrónicos utilizando el microcontrolador ATmega328P, integrando hardware y software en dispositivos prácticos. Se implementaron cuatro bloques principales: cinta transportadora con punzonadora, matriz de LEDs 8x8, conversor DAC R-2R con tabla de búsqueda y plotter. Cada proyecto permitió reforzar competencias en control de sistemas, programación en lenguaje ensamblador y verificación experimental. Las prácticas incluyeron la gestión de modos de operación, control de actuadores, comunicación UART y generación de señales analógicas a partir de datos digitales. Se documentaron los resultados y se analizaron las limitaciones técnicas, destacando la importancia de la planificación y verificación rigurosa en sistemas mecatrónicos.

Keywords: ATmega328P, Mechatronic systems, Assembly language programming, Conveyor belt, LED matrix, R-2R DAC, Plotter, UART communication, Hardware-software integration

Abstract—This laboratory focused on the design, programming, and control of mechatronic systems using the ATmega328P microcontroller, integrating hardware and software in practical devices. Four main projects were carried out: a conveyor belt with puncher, an 8x8 LED matrix, an R-2R DAC with a lookup table, and a plotter. Each project reinforced skills in system control, assembly language programming, and experimental validation. Activities included operation mode management, actuator control, UART communication, and analog signal generation from digital data. Results were documented and technical limitations were analyzed, highlighting the importance of planning, verification, and rigorous implementation in mechatronic systems.

I. INTRODUCCIÓN

En el presente laboratorio, realizamos actividades de programación en lenguaje ensamblador aplicadas al microcontrolador ATmega328P, explorando la interacción entre software y hardware a través de sistemas electrónicos y mecánicos. Durante la práctica, desarrollamos distintos proyectos que nos permitieron experimentar directamente cómo la programación en bajo nivel controla periféricos, gestiona señales digitales y temporizaciones, y coordina dispositivos mecánicos en sistemas embebidos.

Las actividades incluyeron la implementación de una cinta transportadora con punzonadora, la programación de una matriz de LEDs de 8x8, la construcción de un conversor digital-analógico (DAC) R-2R y el control de un plotter capaz de dibujar figuras predeterminadas. Cada sistema presentó desafíos particulares en términos de programación, sincronización de acciones y control de entradas y salidas digitales, lo que nos permitió aplicar conceptos de electrónica, comunicación serial y control de movimiento de manera práctica.

Durante el desarrollo, planificamos y diseñamos diagramas de máquina de estados para organizar las secuencias de el

sistemas que lo ameritaba, configuramos los puertos y registros del microcontrolador, definimos variables y constantes, y desarrollamos rutinas que ejecutan acciones temporizadas y responden a señales externas. También utilizamos USART y UART para establecer comunicación serial, permitiendo seleccionar opciones y monitorear el funcionamiento de los sistemas en tiempo real.

Además, realizamos simulaciones previas de los códigos para comprobar su funcionamiento antes de la implementación física, y posteriormente conectamos los componentes siguiendo las configuraciones definidas en el código.

II. OBJETIVOS

Objetivo general

Fortalecer las habilidades en el diseño y control de sistemas mediante la implementación de soluciones basadas en el microcontrolador ATmega328P, desarrollando la capacidad de integrar hardware y software para resolver problemas prácticos. Asimismo, fomentar la comprensión del trabajo con periféricos, la comunicación serial y la generación de señales, promoviendo un enfoque integral que combine teoría, práctica y documentación teórica.

Objetivos específicos:

- Construir y programar el modelo de cinta transportadora con punzonadora, gestionando modos de operación, selección de carga y comunicación serial via USART.
- Implementar una matriz de LEDs controlada por el microcontrolador que despliegue mensajes desplazables y permita interacción con el usuario mediante UART.
- Desarrollar una Look Up Table (LUT) de una señal brindada e implementarla en un conversor DAC R-2R de 8 bits y verificar su comportamiento mediante osciloscopio.
- Programar el funcionamiento de un plotter mediante el microcontrolador, permitiendo la selección y trazado de figuras geométricas predefinidas a través de comunicación serial.
- Gestionar los códigos en lenguaje ensamblador utilizando GitHub, con control de versiones y documentación de las modificaciones realizadas.

III. MATERIALES UTILIZADOS

TABLE I

MATERIALES UTILIZADOS

MATERIALES	CANTIDAD
Arduino Uno Rev3	1
Protoboard	1
Kit de Fischertechnik	1
Fuente de DC	1
Pulsadores	4
LEDs roja	2
LEDs verde	1
LEDs amarilla	3
Driver L298	1
Resistencia 2k	9
Resistencia 1k	7
Osciloscopio	1
Matriz LEDs 8x8	1
Cables	-
Pen Plotter	1
Microchip Studio	-

IV. MARCO TEÓRICO

Arduino UNO Rev3

El Arduino Uno Rev3 es una placa de desarrollo ampliamente utilizada en el ámbito educativo y en el prototipado de sistemas electrónicos, principalmente por su facilidad de uso, bajo costo y la gran comunidad que respalda su ecosistema. Esta placa se basa en el microcontrolador ATmega328P.

El dispositivo funciona con una tensión nominal de 5 V regulados y puede ser alimentado tanto a través del puerto USB como mediante una fuente externa, recomendándose un rango de voltaje entre 7 V y 12 V para asegurar un rendimiento estable. Esta flexibilidad en la alimentación facilita su integración en diferentes entornos y aplicaciones.

En cuanto a su arquitectura de pines, el Arduino Uno dispone de 14 pines digitales de entrada/salida, de los cuales 6 pueden ser utilizados como salidas PWM, además de 6 entradas analógicas identificadas como A0 a A5. La placa incluye también pines de referencia y alimentación, como VIN, AREF, conexiones a tierra, así como salidas de 3.3 V y 5 V para alimentar periféricos externos. Asimismo, incorpora interfaces de comunicación que permiten su interacción con otros dispositivos electrónicos: UART (pines 0 y 1), SPI (pines 10 a 13) e I²C (pines A4 y A5). Esta diversidad de opciones convierte al Arduino Uno en una plataforma versátil para la interconexión de sensores, actuadores y módulos de expansión.

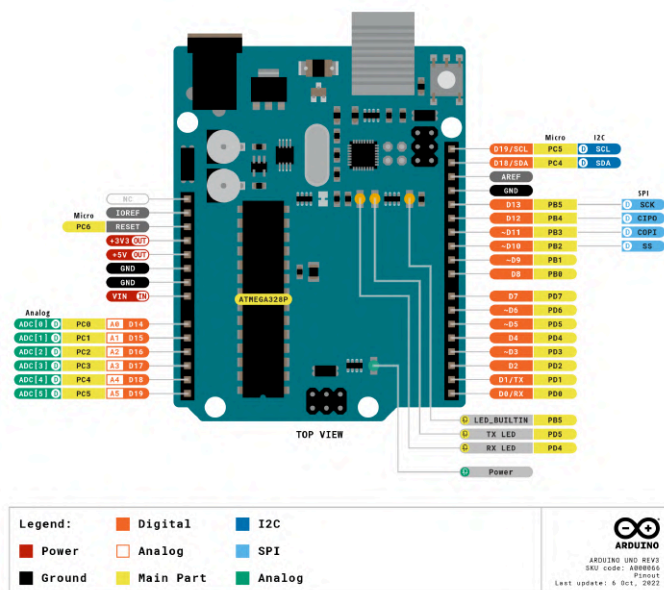


Fig.1 Arduino Pinout

ATmega328p

El ATmega328P es un microcontrolador de 8 bits de la serie megaAVR desarrollado por Atmel, ampliamente utilizado en sistemas embebidos y proyectos de electrónica de bajo consumo y bajo costo. Su arquitectura se basa en un microcontrolador RISC, lo que le permite ejecutar instrucciones en un solo ciclo de reloj, alcanzando hasta 1 MIPS, optimizando la velocidad de procesamiento y el consumo energético.

Entre sus principales características se destacan:

- Memoria: 32 KB de memoria flash (ISP) con capacidad de lectura mientras se escribe, 1 KB de memoria EEPROM y 2 KB de SRAM.
- Entradas/Salidas: Hasta 23 líneas de E/S de propósito general, 32 registros de propósito general y 6 canales PWM.
- Convertidor A/D: 6 canales de 10 bits, que permiten la adquisición de señales analógicas de los sensores.
- Temporizadores: 3 temporizadores flexibles con modo de comparación y un temporizador "watchdog" programable.
- Interfaz de comunicación: Soporta USART, SPI y comunicación serie de 2 hilos.
- Modos de ahorro de energía: Cinco modos configurables por software, que optimizan la autonomía del dispositivo en aplicaciones portátiles.
- Rango de operación: Funciona entre 1,8 y 5,5 V, lo que lo hace compatible con distintos sistemas de alimentación.

El ATmega328P se utiliza en proyectos que requieren control preciso de periféricos, adquisición de datos y transmisión de información, siendo la base de plataformas como Arduino Uno y Nano.

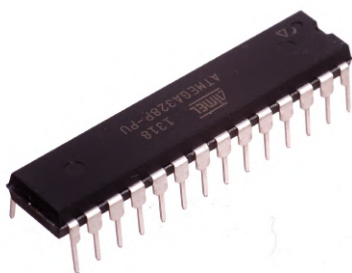


Fig.2 Microcontrolador ATmega328P.

Ensamblador

El lenguaje ensamblador es un lenguaje de programación de bajo nivel que proporciona una representación simbólica de las instrucciones de máquina específicas de una arquitectura de procesador. A diferencia de los lenguajes de alto nivel, el lenguaje ensamblador está estrechamente vinculado al hardware, lo que permite un control preciso sobre los recursos del sistema. Cada arquitectura de procesador tiene su propio conjunto de instrucciones, y el lenguaje ensamblador utiliza mnemónicos para representar estas instrucciones, facilitando su comprensión y escritura por parte del programador.

El programa ensamblador traduce el código fuente escrito en lenguaje ensamblador a código máquina ejecutable por el microprocesador. Este proceso de traducción es generalmente directo, con una correspondencia uno a uno entre las instrucciones mnemotécnicas y las instrucciones de máquina. Sin embargo, algunos ensambladores modernos ofrecen características adicionales, como el soporte para macros, que permiten la reutilización de fragmentos de código y facilitan la programación.

El uso del lenguaje ensamblador es particularmente relevante en situaciones donde se requiere un alto rendimiento, un control detallado del hardware o la optimización de recursos limitados, como en sistemas embebidos o en el desarrollo de controladores de dispositivos. Aunque los lenguajes de alto nivel han ganado popularidad debido a su facilidad de uso y portabilidad, el lenguaje ensamblador sigue siendo una herramienta valiosa en áreas donde el control preciso y la eficiencia son esenciales.

UART y USART

Los protocolos USART (Universal Synchronous Asynchronous Receiver Transmitter) y UART (Universal Asynchronous Receiver Transmitter) son fundamentales en sistemas embebidos para la transmisión de datos entre dispositivos. Ambos permiten la comunicación serial, pero presentan diferencias clave en su funcionamiento y aplicaciones.

El UART es un protocolo de comunicación serial asíncrona que utiliza dos líneas: una para la transmisión (TX) y otra para la recepción (RX). Su principal característica es que no requiere una señal de reloj externa, ya que la sincronización entre el transmisor y el receptor se establece mediante la configuración previa de parámetros como la velocidad de transmisión (baud rate), bits de datos, paridad y bits de parada. Esta simplicidad lo hace ideal para comunicaciones de corto alcance y baja velocidad, como la conexión entre microcontroladores y módulos periféricos.

Por otro lado, el USART extiende las capacidades del UART al permitir tanto comunicación asíncrona como síncrona.

Además de las líneas TX y RX, incorpora dos señales adicionales: XCK (Clock) y XDIR (Direction). La señal XCK proporciona un reloj para la sincronización de datos en modo síncrono, mientras que XDIR indica la dirección de la transmisión en comunicaciones bidireccionales. Estas características permiten al USART operar a mayores velocidades y con mayor precisión en entornos donde la sincronización es crítica, como en comunicaciones industriales o entre dispositivos que requieren alta tasa de transferencia de datos.

En términos de compatibilidad, es común que los microcontroladores implementen interfaces USART que puedan configurarse para operar en modo UART, adaptándose así a diferentes necesidades de comunicación sin requerir hardware adicional. Esta flexibilidad facilita el diseño de sistemas que pueden cambiar entre modos síncronos y asíncronos según los requisitos específicos de la aplicación.

Diagrama de máquina de estados

El diagrama de máquina de estados en UML es un diagrama de comportamiento que describe cómo un objeto cambia entre distintos estados a lo largo de su ciclo de vida, en función de eventos que ocurren. Un estado representa una situación en la que el objeto cumple ciertas condiciones o ejecuta actividades específicas, mientras que una transición muestra el paso de un estado a otro, provocado por un evento y opcionalmente condicionado por una guarda o acompañado de una acción.

Dentro del diagrama se incluyen símbolos estandarizados como el estado inicial (círculo sólido) y el estado final (círculos concéntricos). También es posible modelar estados compuestos, que contienen subestados, así como regiones ortogonales, que permiten representar comportamientos concurrentes. En algunos casos se emplean estados de historia, que retoman el subestado previamente activo en lugar de reiniciar.

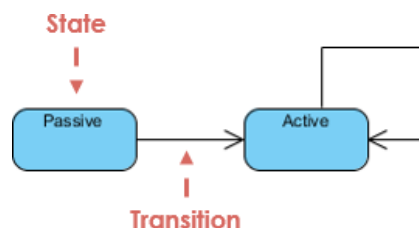


Fig.3 Estado y transición en diagramas de máquinas de estado Punzonadora con cinta transportadora

En el kit Smart Beginner se incluye un sistema que simula una cinta transportadora con punzonadora, destinado a la enseñanza de automatización y control. Este sistema permite experimentar con el transporte de objetos a lo largo de una banda motorizada, mientras se ejecuta un mecanismo de perforación de manera sincronizada. La banda transportadora está diseñada para desplazarse de forma continua mediante un pequeño motor eléctrico, permitiendo que los objetos se muevan a lo largo de un recorrido predefinido dentro del área del kit.

El mecanismo de punzonadora del kit se acciona en coordinación con el movimiento de la cinta, realizando perforaciones en los objetos de manera controlada. Este proceso permite comprender cómo se integran movimiento y acción sobre los materiales en sistemas automatizados, sin requerir equipos industriales de gran escala. La combinación de la banda y la punzonadora en el kit permite a los estudiantes

observar y controlar aspectos como la temporización, la sincronización y la precisión de las acciones mecánicas.

El uso de este sistema en el kit educativo facilita la comprensión de conceptos de automatización, como la secuencia de operaciones, la coordinación de componentes mecánicos y la programación de movimientos. A través de la manipulación de la cinta y la punzonadora, los estudiantes pueden experimentar de manera práctica con principios de ingeniería, control y procesos industriales, en un entorno seguro y de escala reducida.

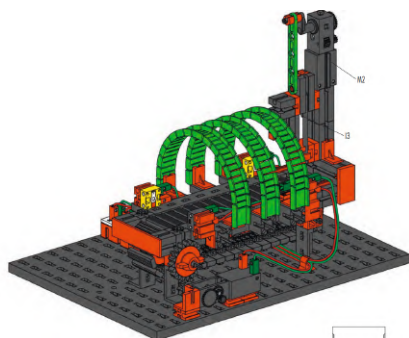


Fig.4 Cinta transportadora con punzonadora
Puente H- Driver L298

Un puente H es una configuración circuital con diodos que permite controlar el giro de un motor, funcionando como controlador de la polaridad aplicada al motor. Existen drivers que ya integran esta electrónica, facilitando el control de motores sin necesidad de armar el circuito completo. Un ejemplo clásico es el L293; en esta sección se analiza el driver L298.

El driver L298 permite controlar dos motores de corriente continua o motores paso a paso con una corriente máxima de 2 amperios. Actualmente existen módulos comerciales que incluyen el driver soldado y conectores para las entradas y salidas, facilitando su uso. En el módulo típico del driver L298 se encuentran los siguientes componentes:

Regulador de voltaje LM7805, que permite alimentar la lógica del módulo.

Conectores de salida Output A y Output B para conectar los motores.

Terminales de control (control inputs), que incluyen pines de habilitación para cada motor.

Un jumper que permite decidir si se usa o no el regulador de voltaje LM7805.

Es importante considerar el voltaje con el que se alimentará el módulo. Aunque el driver soporta tensiones de hasta 36 V, si la alimentación es similar a la del microcontrolador (entre 6 y 12 V), no es necesario activar el regulador, ya que el L298 tolera esos niveles. En cambio, si se alimenta con tensiones superiores a 12 V, es obligatorio activar el jumper para usar el regulador; de lo contrario, el módulo puede dañarse rápidamente.

Para controlar un motor de corriente continua con el driver L298, normalmente se utilizan dos pines de control que determinan el sentido de giro del motor, enviando señales digitales de uno o cero según corresponda.

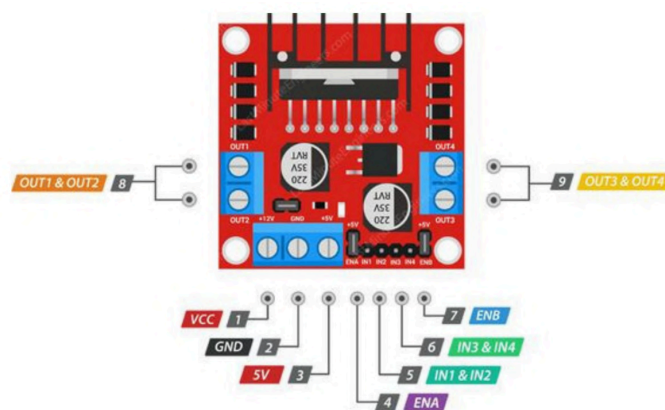


Fig.5 Driver L298 Pinout

DAC R-2R

Un DAC R-2R es un convertidor digital-a-analógico que usa únicamente dos valores de resistor (R y 2R) configurados en una red tipo escalera ("ladder") para convertir un número binario en una señal analógica proporcional.

En su operación, cada bit digital (en lógica alta o baja) se aplica en distintos puntos de la escalera de resistores, generando divisiones de voltaje que luego se combinan y son convertidas a salida analógica mediante un amplificador operacional.

Se puede derivar una expresión general para el voltaje de salida. Si hay n bits, el valor de la salida es la suma ponderada de los bits en posiciones distintas, y el "paso" mínimo (LSB) es $V_{LSB} = V_{IN}/2^n$.

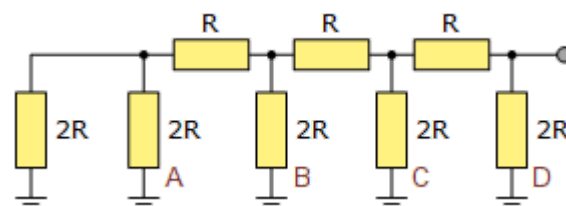


Fig.6 R-2R DAC

Matriz de LEDs 8x8

La matriz LED 8x8 es un dispositivo compuesto por 64 diodos emisores de luz dispuestos en ocho filas y ocho columnas, que permiten representar símbolos, caracteres o animaciones simples mediante el encendido controlado de cada punto luminoso. El principio de funcionamiento se basa en la multiplexación, que consiste en activar de forma secuencial cada fila o columna a gran velocidad, aprovechando la persistencia visual del ojo humano para percibir una imagen continua.

El control directo de la matriz requiere hasta 16 pines de salida, lo que resulta poco eficiente al trabajar con microcontroladores de bajo número de pines como el Arduino Uno.

En el ámbito de la programación, el control de la matriz puede realizarse de forma directa o mediante librerías específicas para Arduino, que simplifican la escritura de caracteres, el encendido de puntos individuales y la generación de animaciones. Gracias a estas herramientas, es posible implementar efectos de desplazamiento, rotación o transiciones

fluidas, lo que convierte a la matriz LED 8×8 en una opción versátil y didáctica para proyectos de visualización electrónica.

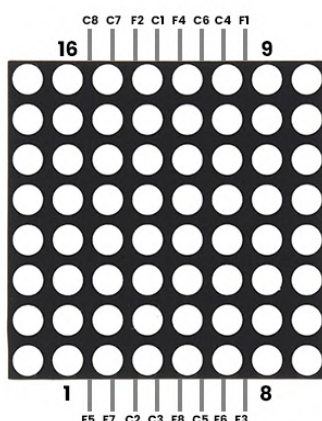


Fig.7 Matriz de LEDs 8x8 Pinout

Pen Plotter

Un pen plotter es un dispositivo automatizado diseñado para dibujar imágenes o diagramas sobre una superficie plana utilizando un bolígrafo, marcador u otro instrumento de escritura. A diferencia de las impresoras convencionales, que aplican tinta mediante procesos de impresión, las pen plotters mueven el instrumento de manera controlada sobre los ejes X e Y, permitiendo reproducir con precisión figuras vectoriales.

En su funcionamiento típico, la pen plotter utiliza motores paso a paso para controlar el movimiento horizontal y vertical del bolígrafo, mientras que un mecanismo adicional permite levantar y bajar el instrumento de escritura según sea necesario. La máquina puede ser controlada mediante comandos de control de movimiento, como los que se encuentran en el G-code, lo que permite programar trayectorias complejas y repetir dibujos de manera consistente.

Este tipo de dispositivos se emplea tanto en proyectos educativos como en aplicaciones de prototipado rápido y automatización, ofreciendo una plataforma práctica para comprender conceptos de control de movimiento, sincronización de ejes y coordinación mecánica, sin necesidad de equipos industriales complejos.

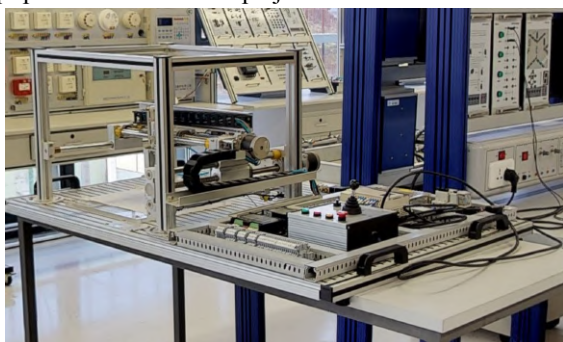


Fig.8 Pen Plotter

TABLE II
CONEXIONES PEN PLOTTER

Pin Digital	Conexión
D2	Bajar solenoide
D3	Subir solenoide
D4	Movimiento hacia abajo
D5	Movimiento hacia arriba
D6	Movimiento hacia la derecha
D7	Movimiento hacia la izquierda

V. METODOLOGÍA

En el presente laboratorio se llevaron a cabo actividades basadas en la programación en lenguaje ensamblador aplicadas al microcontrolador ATmega328P. La práctica estuvo compuesta por cuatro actividades principales: la ejecución de una cinta transportadora con punzonadora, la implementación de una matriz LED, el desarrollo de un conversor DAC y la utilización de un plotter. A continuación, se detalla el procedimiento correspondiente a cada una de estas etapas.

Parte A:

En esta actividad se planteó la implementación de un código para una cinta transportadora con punzonadora. El sistema debía realizar el siguiente procedimiento: elección de carga, avance de la cinta para posicionar la carga debajo de la punzonadora, detención de la cinta para posteriormente punzar la carga y, finalmente, retorno de la cinta y la carga a su posición inicial. La elección de carga debía contemplar tres opciones: carga ligera, carga media y carga pesada. Cada una de estas presentaba un tiempo especificado para realizar las funciones: avance 3 segundos y pausa de 2 segundos (carga ligera), avance 4 segundos y pausa 2 segundos (carga mediana), avance 5 segundos y pausa 3 segundos (carga pesada). Este proceso debía funcionar mediante dos tipos de comunicación: USART y lógica cableada.

Iniciando con la resolución de esta actividad, se desarrolló un diagrama de máquina de estados, el cual permitió una clara visualización del problema para su posterior solución. Dicho diagrama cuenta con seis estados: en espera, elección de carga, posicionado, punzonado, descarga y fin del ciclo. Este sistema se vinculó con entradas físicas (pulsadores), tiempos de espera y comunicación serial para controlar las salidas.

Respecto al código, se inició con la configuración de los puertos del microcontrolador, preparando los registros que se utilizaron como variables, habilitando temporizadores e inicializando el USART a 9600 baudios. Posteriormente, se programó el envío de un mensaje de inicio para confirmar que el sistema está activo, quedando en espera en el bucle principal. Dentro de este bucle, se leen las entradas del sistema (pulsadores y sensores) constantemente, comparando cada lectura con la anterior para detectar cambios de estados en el sistema. Si el estado cambia, el programa transmite por USART un mensaje indicando la nueva condición, sincronizando la lógica con una salida informativa.

La lógica principal se basa en el recorrido de los estados mencionados: en el primer estado, el sistema permanece inactivo hasta que se reciba un comando (mediante USART) o una señal física desde un pulsador, si esto sucede, se cambia al siguiente estado, seleccionando una de las cargas disponibles,

encendiendo leds que indican la elección. Este estado, se vincula con el siguiente, activando la salida con los intervalos de tiempo correspondientes, esperando una señal externa para continuar (sensores de posición). Más adelante, se ejecuta una secuencia de pulsos que activa la punzonadora, seguida de un avance mecánico hasta llegar al estado final, en el cual se enciende un LED de cierre segundos antes de reiniciar el ciclo completo.

Los intervalos de tiempo se implementaron mediante un temporizador que genera retardos de un segundo, multiplicados por un contador para alargar las interrupciones según sea necesario. A su vez, la comunicación USART permite controlar y monitorear el flujo: al recibir caracteres específicos, se avanza en la máquina de estados, se selecciona la carga, o incluso se reinicia el sistema en cualquier momento.

Previo a la implementación física, se simuló el funcionamiento del código en PICsimLab considerando todas las entradas del sistema como pulsadores y todas las salidas como LEDs.

Posteriormente, se ensambló la cinta transportadora con punzonadora utilizando los kits Fischertechnik, siguiendo paso a paso el manual de instrucciones. Una vez finalizado el armado, se procedió a conectar la cinta con el microcontrolador y la fuente de alimentación. Para estas conexiones, se consideraron las declaraciones del código desarrollado previamente.

Con respecto a la conexión entre la cinta y el microcontrolador, se conectó el sensor de inicio a PC2 y el sensor de finalización a PD6. Por otra parte, los motores (cinta y punzonadora) se conectaron a un puente H y posteriormente al arduino: la cinta a PB5 y PC3 y la punzonadora a PC0. Esta conexión extra al puente H fue necesaria debido a que el arduino trabaja con 5 V y la cinta con aproximadamente 9 V. Además, se requirió utilizar todos los pines del arduino, lo que impidió utilizar la adaptación del kit para su propia batería, ya que esto limitaba los pines de conexión.

Posteriormente, se estableció el vínculo entre la lógica cableada y el arduino, para esto se realizaron las siguientes asignaciones: botón de inicio a PD2, botón de carga ligera a PD3, botón de carga mediana a PD4, botón de carga pesada a PD5, led de espera a PB0, led de funcionamiento a PB1, led de carga ligera a PB2, led de carga mediana a PB3, led de carga pesada a PB4 y led de finalización a PC1.

Finalmente, se corroboró el correcto funcionamiento de todos los casos posibles. La cinta transportadora con punzonadora respondió de manera adecuada tanto a los comandos enviados por USART como a la lógica cableada, cumpliendo con las secuencias de avance, posicionamiento, punzonado y descarga definidas en la máquina de estados.

Parte B.

En esta actividad, se planteó el desarrollo de un código capaz de comunicar al usuario, mediante UART, un conjunto de opciones predefinidas para posteriormente representarlas en una matriz de LEDs de 8x8. Las opciones implementadas fueron: un mensaje desplazante y dos figuras (una cara feliz y un corazón). Los requisitos funcionales establecidos incluyeron: una clara visualización del mensaje mediante un desplazamiento continuo y suave, una velocidad de scroll ajustable, comunicación exclusivamente mediante UART y una clara documentación del código para facilitar su reutilización.

El programa está organizado en tres bloques principales que interactúan para controlar una matriz de LEDs y la comunicación UART. Primero, se inicializa la UART para habilitar la transmisión y recepción de datos seriales, permitiendo mostrar mensajes en la terminal y recibir la selección del usuario. Al iniciar, el programa muestra un mensaje de bienvenida y un menú con las opciones disponibles: “1: Mensaje desplazante”, “2: Carita Feliz” y “3: Corazón”. Una vez seleccionada una opción, el programa ejecuta la rutina correspondiente.

En el caso de la opción 1, se activa el mensaje desplazante (scroll), el cual se implementa almacenando cada columna de los caracteres del mensaje en un buffer de memoria. El desplazamiento se logra mediante un índice llamado offset que recorre las columnas del buffer, mostrando progresivamente ocho columnas a la vez en la matriz de LEDs. Cada columna se envía a los pines correspondientes de las filas y columnas del display mediante multiplexado: se activa una fila a la vez, cargando los bits de la columna actual en los puertos de salida y aplicando un retardo muy breve antes de pasar a la siguiente fila. La velocidad del desplazamiento se controla mediante un contador de frames, al llegar al final del mensaje, el scroll se reinicia de manera continua, creando un efecto de movimiento fluido.

Por otra parte, al seleccionar las opciones 2 o 3, el programa permite mostrar figuras fijas: una carita feliz o un corazón, activando secuencialmente las columnas de la matriz para cada fila según la forma de la figura.

Además, mientras se ejecuta cualquier opción, el programa revisa constantemente si hubo un cambio de selección mediante la UART, permitiendo que se pueda cambiar dinámicamente entre scroll y las figuras sin necesidad de reiniciar el sistema.

Posteriormente a la realización del código, se procedió a realizar la conexión física de la matriz LEDs y el microcontrolador. Esta etapa consistió principalmente en asociar cada pin de la matriz con el correspondiente pin del Arduino, asegurando que esta correspondencia siguiera lo definido en el código. Para proteger los LEDs y limitar la corriente se colocaron resistencias en los pines correspondientes a las columnas de la matriz. La conexión se realizó siguiendo un esquema claro y ordenado, garantizando que cada fila y cada columna pudiera ser controlada de manera independiente mediante las rutinas de multiplexado implementadas en el código. La configuración exacta de los pines utilizados y su correspondencia con la matriz se detalla en el apartado de resultados como “TABLE V”.

Finalmente, se verificó el correcto funcionamiento del sistema y se realizó la documentación requerida.

Parte C:

En este apartado se requirió la realización de un código capaz de implementar una LUT que muestre mediante un osciloscopio una señal analógica, implementando además un conversor DAC R-2R de 8 bits.

En primer lugar, se inicializó el sistema configurando la pila y estableciendo el puerto D como salida digital, de modo que cada valor escrito en este puerto fuera convertido en un voltaje proporcional por el DAC. Posteriormente, se implementó una LUT en memoria de programa que contenía valores correspondientes a la forma de onda a generar, los cuales fueron recorridos secuencialmente mediante el registro puntero

Z. Para garantizar la temporización adecuada, se configuró el Timer0 en modo CTC utilizando el registro OCR0A como comparador, lo que permitió establecer una frecuencia de muestreo estable y ajustar la frecuencia de la señal. En cada coincidencia del temporizador, se cargó el siguiente valor de la LUT y se envió al puerto de salida, reiniciando el puntero al llegar al final de la tabla para lograr una señal periódica y continua.

Posteriormente, se ensambló la parte física, implementando red resistiva R-2R utilizando resistencias de 2k ohm y 1k ohm, donde cada bit del puerto de salida del microcontrolador se conectó a una rama de la red de resistencias. En esta disposición, cada resistencia de 1k ohm se conecta al pin correspondiente del microcontrolador y a una resistencia de 2k ohm que va a tierra, mientras que otra resistencia de 2k ohm continúa hacia el siguiente escalón. Repitiendo esto para los ocho bits, se obtuvo en el nodo final la salida analógica proporcional al valor digital aplicado en el puerto, asegurando que el bit más significativo aportará la mayor contribución en voltaje y el menos significativo la menor. Finalmente, el último nodo se conectó a la entrada del osciloscopio, permitiendo visualizar y analizar la señal generada por el sistema.

Parte D y E:

En esta última actividad se desarrolló un sistema de control para un plotter con solenoide, utilizando lenguaje ensamblador y el microcontrolador ATmega328P. El objetivo fue implementar un programa capaz de recibir comandos por UART y dibujar figuras predeterminadas (un triángulo, una cruz o ambas de manera secuencial) sobre la superficie de trabajo.

El código comienza con la inicialización de la pila y la configuración de los periféricos necesarios. Se habilita la comunicación UART a 9600 baudios para enviar mensajes informativos y recibir la opción seleccionada por el usuario. Además, se configuran como salidas los pines digitales destinados a controlar tanto los motores del sistema de movimiento como el solenoide encargado de subir y bajar el marcador. Se utilizó además un LED de prueba en PB5 como indicador del estado de ejecución.

El sistema transmite al inicio un mensaje de bienvenida y un menú de selección con tres opciones: “1: Triángulo”, “2: Cruz” y “T: Ambas figuras”. El usuario selecciona la figura deseada ingresando el carácter correspondiente en la terminal, lo que actualiza la variable de control interna. Al recibir una opción válida, el sistema enciende el LED, envía un mensaje de confirmación por UART y llama a una función para posicionar el cabezal. Al finalizar, se asegura que el solenoide quede en la posición elevada y el sistema retorna al menú, listo para recibir nuevas instrucciones.

La lógica principal del trazado de las figuras se basa en una serie de subrutinas que controlan el solenoide para subir y bajar el marcador, junto con rutinas de movimiento que activan los motores del cabezal durante tiempos específicos. Para trazar el triángulo, el programa primero baja el marcador y luego ejecuta tres movimientos coordinados: hacia abajo, hacia la izquierda, y finalmente una diagonal hacia arriba y a la derecha. La rutina para la cruz baja el marcador y traza dos líneas perpendiculares, una vertical (moviéndose arriba y abajo) y otra horizontal (moviéndose izquierda y derecha). La rutina que dibuja ambas figuras simplemente ejecuta las dos anteriores en secuencia, asegurando que el cabezal se posiciona

en el centro entre cada una. Los retardos se implementaron mediante una subrutina de temporización en milisegundos, ajustando la duración de cada desplazamiento.

Las conexiones se encontraban previamente realizadas, solo fue necesario analizar las asociaciones de los pines para asegurar declaraciones correctas en el código y conectar el arduino a una terminal para enviar el código

VI. RESULTADOS

En este apartado se presenta y analiza cada resultado o avance obtenido mediante la práctica, lo cuál incluye documentación visual de las actividades y fragmentos importantes de sus códigos.

Parte A:

Máquina de Estados:

En el Apéndice I se presenta el diagrama de la máquina de estados, mientras que en el Apéndice II se muestra el código implementado para su realización. Dicho diagrama describe el funcionamiento secuencial del sistema, el cuál se explicará brevemente. El Estado 0 representa la condición inicial o de espera, donde todos los actuadores están apagados y únicamente el LED de espera se encuentra encendido, pasando al Estado 1 al recibir la orden de inicio desde un pulsador o por USART. En el Estado 1, se activa el LED de funcionamiento y, según la señal de entrada (sensor de inicio y pulsadores de carga), se selecciona uno de los tres subestados que definen el tipo de carga. Posteriormente, en el Estado 2, la cinta avanza con el LED correspondiente a la carga seleccionada encendido, manteniendo este estado durante un tiempo determinado antes de pasar a una etapa de espera y luego a la punzonadora. El Estado 3 ejecuta la acción de punzonado, manteniendo el LED de carga activo según el subestado en curso, y tras un tiempo específico retorna a la espera o avanza al siguiente estado. En el Estado 4, la cinta retrocede con el LED de carga correspondiente encendido, hasta completar el tiempo asignado para cada subestado. Finalmente, el Estado 5 indica la finalización del ciclo con el LED de fin encendido, regresando después de 5 segundos al estado inicial. Adicionalmente, en cualquier punto del proceso es posible forzar el retorno al Estado 0 mediante la recepción de la orden USART = 'R', asegurando así una condición de reinicio inmediato del sistema.

Documentación:

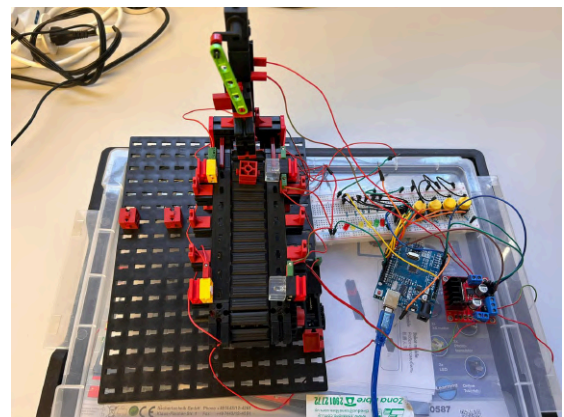


Fig. 9 Configuración apartado A - Cinta transportadora con punzonadora

En la presente imagen se observa la conexión realizada entre el microcontrolador y la cinta transportadora, incluyendo la interacción mediante pulsadores.

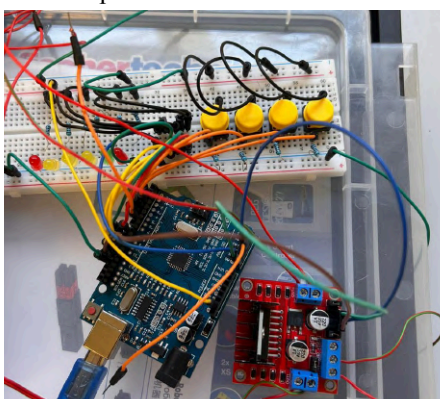


Fig. 10 Configuración apartado A - Lógica cableada

En este caso, se aprecia puntualmente la parte de la comunicación mediante lógica cableada. Los pulsadores funcionan como entradas del sistema, seleccionando el comienzo y la carga a transportar. Por otra parte, las LEDs funcionan como salidas, comunicando al usuario en qué estado se encuentra el sistema.

TABLE III
MAPEO DE CONEXIONES PARTE A - ENTRADAS

ENTRADAS	PUERTO	PIN
Btn_inicio	D	PD2
Btn_ligera	D	PD3
Btn_media	D	PD4
Btn_pesada	D	PD5
Sensor_fin	D	PD6
Sensor_inicio	C	PC2

En esta tabla se especifica cómo se realizó la conexión del microcontrolador respecto a las entradas del sistema, puntualmente pulsadores y sensores.

TABLE IV
MAPEO DE CONEXIONES PARTE A - SALIDAS

ENTRADAS	PUERTO	PIN
Led_espera	B	PB0
Led_funciona	B	PB1
led_ligera	B	PB2
Led_media	B	PB3
Led_pesada	B	PB4
Motor Cinta B	B	PB5
Motor Cinta S	C	PC3
Motor Punzon	C	PC0
Led_fin	C	PC1

En esta segunda tabla se observa las conexiones del microcontrolador con las salidas del sistema (Motores y LEDs).

Parte B:

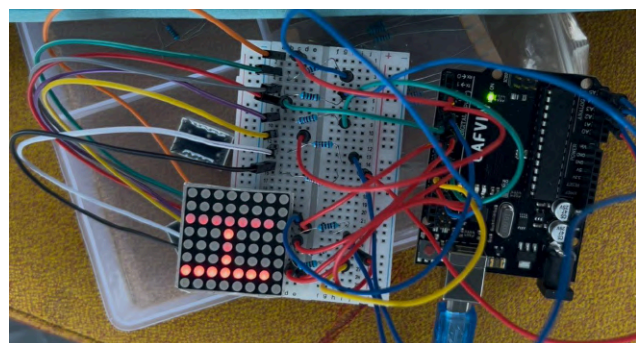


Fig. 11 Configuración apartado B - Matriz Led

En esta figura se observa la conexión entre la matriz led y el arduino, como se mencionó previamente, se conectan resistencias a las columnas para limitar el paso de corriente. Los pines se conectan al arduino como se especifica en la tabla a continuación.

TABLE V
MAPEO DE CONEXIONES PARTE B

PINES MATRIZ	PINES ARDUINO
F1 - 9	PD2
F2 - 14	PD3
F3 - 8	PD4
F4 -12	PD5
F5 -1	PD6
F6 - 7	PD7
F7 - 2	PB0
F8 - 5	PB1
C1 - 13	PB2
C2 - 3	PB3
C3 - 4	PB4
C4 - 10	PB5
C5 - 6	PC5
C6 - 11	PC4
C7 - 15	PC3
C8 - 16	PC2

Tabla que refleja la interacción entre la matriz y el arduino.

TABLE VI
CORAZÓN EN MATRIZ LED

	1	1			1	1	
1	1	1	1	1	1	1	1
1	1	1	1	1	1	1	1
	1	1	1	1	1	1	
		1	1	1	1		
			1	1			

En esta tabla vemos una demostración de cómo se recolectaron los datos para pasarlos a la matriz led, este método consta de crear una tabla de 8x8 y colocar un "1" en donde se prenderán las LEDs para la figura que se pretende mostrar, posterior a esto, se escribe en binarios los datos de las filas, tal así como se muestra a continuación:

Fila 1 = 0b00000000

Fila 2 = 0b01100110
 Fila 3 = 0b11111111
 Fila 4 = 0b11111111
 Fila 5 = 0b01111110
 Fila 6 = 0b00111100
 Fila 7 = 0b00011000
 Fila 8 = 0b00000000

Este procedimiento se realizó para todas las letras del mensaje deslizante. Una vez hecho esto, se puede visualizar en código de la siguiente manera:

.db 0b00000000, 0b01100110, 0b11111111, 0b11111111,
 0b01111110, 0b00111100, 0b00011000, 0b00000000

Parte C:

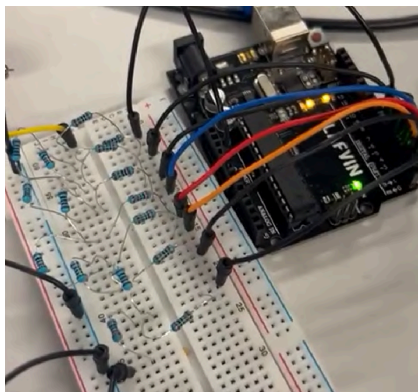


Fig. 12 Configuración - Conversor DAC

En esta figura se observa la red resistiva aplicada para el conversor, recordando que se realizó con resistencias de 2k ohm y 1k ohm.

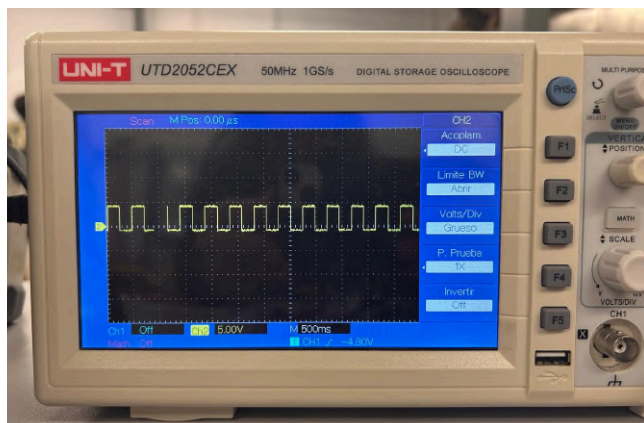


Fig. 13 Señal Analógica - Conversor DAC

En la presente figura se observa la señal analógica lograda gracias al conversor DAC.

Parte E y D:

```
Terminal 0
Bienvenido al sistema de dibujo
Seleccione una opcion:
1: Triangulo 2: Cruz T: Ambas figuras
Dibujando figura...
Ambas figuras seleccionadas
Bienvenido al sistema de dibujo
Seleccione una opcion:
1: Triangulo 2: Cruz T: Ambas figuras
Dibujando figura...
Triangulo seleccionado
Bienvenido al sistema de dibujo
Seleccione una opcion:
1: Triangulo 2: Cruz T: Ambas figuras
```

Fig. 14 Mensaje de bienvenida UART - Plotter

En esta figura se visualiza el mensaje de bienvenida mediante UART, como se muestra, el mensaje se ve desde la terminal virtual.

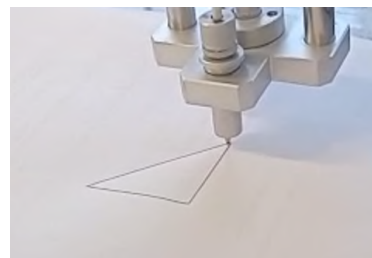


Fig. 15 Figura Triángulo - Plotter

En la presente imagen se observa una de las figuras dibujadas por el Plotter: un triángulo.

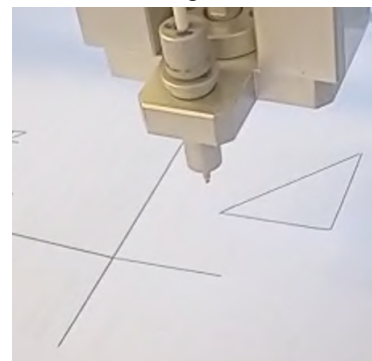


Fig. 16 Figura Cruz - Plotter

Mientras que, en esta imagen, se muestra la segunda figura (Cruz) junto a la anteriormente mencionada.

En este caso el plotter se encontraba en la opción 3, por esta razón, se aprecian ambas figuras.

Generalidades:

En todas las consignas se utilizaron conceptos básicos que son especialmente necesarios para el correcto funcionamiento del código. A continuación se explican algunos de estos conceptos y se adjuntan ejemplos de los mismos:

Configuración de Puertos y registros:

Los códigos implementados utilizan registros temporales, definidos con la directiva .def. Estos registros se emplean para almacenar datos intermedios, índices de bucles, máscaras o flags durante la ejecución, facilitando la manipulación de información de manera eficiente. Además, en esta sección, se asignan y configuran los pines de los microcontroladores para interactuar con los periféricos correspondientes

Se puede apreciar en este fragmento del código B - matriz led:

```
.def fila    = r20
```

```
.def columnas = r21
.def indice   = r22
.def flags    = r23
.def deb_der  = r24
.def deb_izq  = r25
```

Uso de constantes:

Los códigos implementados utilizan constantes necesarias, definidas con .equ, que facilitan la configuración de parámetros como el ancho de mensajes, velocidades de scroll, baudios de UART o direcciones de memoria. Ejemplo extraído del código D - Plotter:

```
.equ BAJA_SOLENOIDE = 2
.equ SUBIR_SOLENOIDE = 3
.equ MOV_ABAJO      = 4
.equ MOV_ARRIBA     = 5
.equ MOV_DERECHA    = 6
.equ MOV_IZQUIERDA  = 7
```

Uso de bucles y control de flujo:

Se implementan bucles loop y controles condicionales (cpi, brne, breq) para iterar sobre datos o estados, permitiendo recorrer LUTs, barrido de leds o movimiento secuencial de motores.

Interrupciones y temporizadores:

Los cuatro programas pueden depender de la temporización para tareas periódicas. En el caso del DAC, se utiliza un timer para actualizar la salida de manera constante; en la matriz LED, la temporización permite el refresco del multiplexado; y en el plotter y la cinta, se controla la activación del solenoide y los movimientos de los motores. Aunque la implementación específica varía en cada caso, todos comparten la lógica de usar registros de temporización y flags para ejecutar acciones de manera precisa y ordenada. A continuación se muestra las interrupciones (por polling) y temporizadores aplicadas en el código A - cinta transportadora:

; Espera de 1 segundo usando Timer0

```
espera_1s:
    ldi r17,0
els:
    sbis TIFR0,TOV0
    rjmp els
    ldi r16,(1<<TOV0)
    out TIFR0,r16
    inc r17
    cpi r17,61
    brne els
    ret
```

; Espera múltiple

```
espera_multi:
    rcall espera_1s
    dec CONTADOR
    brne espera_multi
    ret
```

Manipulación de datos binarios:

Todos los códigos realizan operaciones a nivel de bit para interactuar directamente con el hardware. Se emplean instrucciones como sbi, cbi, ldi, out, and y or para manejar pines individuales o grupos de bits según las necesidades de cada periférico. Esta manipulación binaria es esencial para

controlar salidas digitales, actualizar registros de la LUT, encender o apagar LEDs y mover motores de forma controlada. Un ejemplo aplicable en este caso, es el siguiente fragmento del código C - Conversor DAC:

```
main_loop:
wait_flag:
    sbis TIFR0,OCF0A
    ; limpiar bandera
    ldi tmp,(1<<OCF0A)
    out TIFR0,tmp
    ; siguiente muestra desde la LUT
    lpm sample,Z+ ; leer byte desde memoria de
programa
    out PORTD,sample ; escribir byte completo en
PORTD (R-2R)
```

En este fragmento se manipulan datos binarios a nivel de registro para controlar el hardware: se utiliza lpm sample, Z+ para leer un byte desde la LUT almacenada en la memoria de programa y luego out PORTD, sample para enviar ese valor al puerto D, de manera que cada bit del registro controla un pin individual del microcontrolador, permitiendo manejar un DAC R-2R.

Estructura modular:

Cada programa sigue una estructura modular clara que facilita la lectura y depuración. Esta estructura incluye la configuración inicial del programa (.cseg, .org), la inicialización de los periféricos, el bucle principal donde se ejecuta la lógica central y, en caso de ser necesario, subrutinas o rutinas de interrupción. Este enfoque organizado permite separar las distintas funciones y simplifica tanto la implementación como el mantenimiento del código.

Stack pointer:

Todos los programas hacen uso del Stack Pointer para gestionar el almacenamiento temporal de direcciones de retorno y datos durante la ejecución de subrutinas o interrupciones. Esto permite que el microcontrolador mantenga un control seguro sobre el flujo del programa, asegurando que las llamadas a subrutinas, los retornos y las interrupciones no corrompan los registros temporales ni los datos del sistema. Fragmento extraído del código B - Matriz led, a modo de ejemplo:

```
ldi r16,high(RAMEND)
out SPH,r16
ldi r16,low(RAMEND)
out SPL,r16
```

UART:

Los códigos que requieren comunicación serial utilizan la UART para transmitir y recibir información con otros dispositivos o con la computadora. Esto incluye la configuración de velocidad de transmisión (baud rate), activación de transmisor y receptor, y la lectura/escritura de datos mediante registros de la UART. La comunicación serial permite enviar comandos, datos de control o información de estado, y es común a los programas de DAC, matriz LED y plotter cuando se requiere monitoreo o control externo.

A continuación se muestra como se implementó en el código B - matriz led, teniendo en cuenta que se usó esta estructura para todos los demás ítems.

; Inicialización UART

```

ldi temp, high(UBRR_VALUE)
sts UBRR0H, temp
ldi temp, low(UBRR_VALUE)
sts UBRR0L, temp
ldi temp, (1<<RXEN0)|(1<<TXEN0)
sts UCSR0B, temp
ldi temp, (1<<UCSZ01)|(1<<UCSZ00)
sts UCSR0C, temp

```

Donde: UBRR0H / UBRR0L configura la velocidad de transmisión (baud rate), UCSR0B habilita el receptor (RXEN0) y el transmisor (TXEN0) y UCSR0C configura la comunicación a 8 bits (UCSZ01=1 y UCSZ00=1).

```

; Recepción y chequeo de información
UART_CheckReceive:
lds temp,UCSR0A
sbrs temp,RXC0
ret
lds temp,UDR0
cpi temp,'1'
breq set1
cpi temp,'2'
breq set2
cpi temp,'3'
breq set3
ret
set1:
ldi opcion_actual,1
ret
set2:
ldi opcion_actual,2
ret
set3:
ldi opcion_actual,3
ret

```

En esta parte se verifica si hay un dato recibido (mediante RXC0): si no hay retorna inmediatamente, si hay un carácter recibido ("1", "2" o "3") actualiza la variable (opcion_actual) que controla la matriz.

```

; Transmisión de un carácter
UART_Transmit:
lds temp,UCSR0A
sbrs temp,UDRE0
rjmp UART_Transmit
sts UDR0,r18
ret

```

Donde se espera hasta que el buffer de transmisión esté vacío (UDRE0) y se envía el byte en r18 al registro UDR0 para transmitirlo por la UART.

```

; Envío de un string
UART_SendString:
lpm r18,Z+
cpi r18,0
breq fin_str
rcall UART_Transmit
rjmp UART_SendString
fin_str:

```

```
ret
```

En este caso, se lee un string desde la memoria del programa (FLASH) usando el puntero Z y se transmite cada carácter hasta encontrar un 0 que indique el final del string.

```
; Transmisión de un carácter
```

MostrarMenu:

```

ldi ZH,high(WelcomeMsg<<1)
ldi ZL,low(WelcomeMsg<<1)
rcall UART_SendString
ldi ZH,high(MenuMsg<<1)
ldi ZL,low(MenuMsg<<1)
rcall UART_SendString
ldi ZH,high(MenuOpciones<<1)
ldi ZL,low(MenuOpciones<<1)
rcall UART_SendString
ret

```

En este fragmento, se cargan los punteros Z con los mensajes y se llama a UART_sendString para enviar cada mensaje por la UART, mostrando el menú inicial al usuario.

VII. CONCLUSIONES

Durante la realización de este laboratorio se abordó el diseño, programación y control de distintos sistemas mecatrónicos utilizando el microcontrolador ATmega328P, con el objetivo de integrar de manera efectiva el hardware y software en la operación de dispositivos prácticos. Se trabajó en la implementación de cuatro bloques principales: la cinta transportadora con punzonadora, la matriz de LEDs, el conversor DACR-2R con tabla de búsqueda y el plotter, cada uno de los cuales permitió reforzar competencias en control de sistemas, programación en lenguaje ensamblador y verificación experimental de resultados.

En el caso de la cinta transportadora con punzonadora se logró cumplir con los objetivos de gestión de modos de operación, detección de objetos mediante sensores y coordinación con la cinta transportadora. Para esta implementación, se decidió usar un puente H (Driver L298) en lugar de la F5 board prevista para el control de los motores, dado que esta última no ofrece los suficientes pines de conexión para manejar simultáneamente los LEDs, pulsadores y sensores de la cinta. Se identificó una limitación en el comportamiento final del mecanismo: al activarse el sensor, la punzonadora no mantiene la posición baja por el tiempo esperado, sino que realiza un par de ciclos de bajar y subir repetidamente. A pesar de esta desviación respecto al comportamiento esperado, el resto de las funciones operan correctamente, permitiendo validar la programación desarrollada y reforzando la comprensión de la integración entre hardware y software en un sistema mecatrónico.

La implementación de la matriz de LEDs permitió desplegar mensajes desplazables de manera efectiva, así como interactuar con el usuario a través del monitor serial. El sistema respondió correctamente a los comandos enviados, mostrando los patrones programados de forma clara y sin errores de visualización. Esta práctica facilitó la comprensión del manejo de periféricos de salida, la programación secuencial y la comunicación mediante UART. La operación estable de la matriz de LEDs evidencia que los objetivos planteados para este bloque se cumplieron satisfactoriamente, consolidando la

familiarización con este tipo de dispositivos y su control mediante microcontrolador.

Durante la implementación del conversor DAC R-2R con tabla de búsqueda, se logró generar la señal analógica a partir de datos digitales propuestos de manera efectiva, obteniendo una onda cuadrada con un duty cycle de 50%. La señal fue verificada mediante el osciloscopio y mostró un comportamiento consistente con lo esperado. Esta práctica permite comprender y aplicar los principios de conversión digital-analógica, así como el uso de tablas de búsqueda en lenguaje ensamblador para controlar con precisión la salida del DAC.

Aunque la resolución del DAC de 8 bits y el ruido presente en el sistema experimental impusieron ciertas limitaciones, la señal generada permitió validar la estrategia de generación de señales y reforzó el aprendizaje sobre la integración de componentes digitales y analógicos.

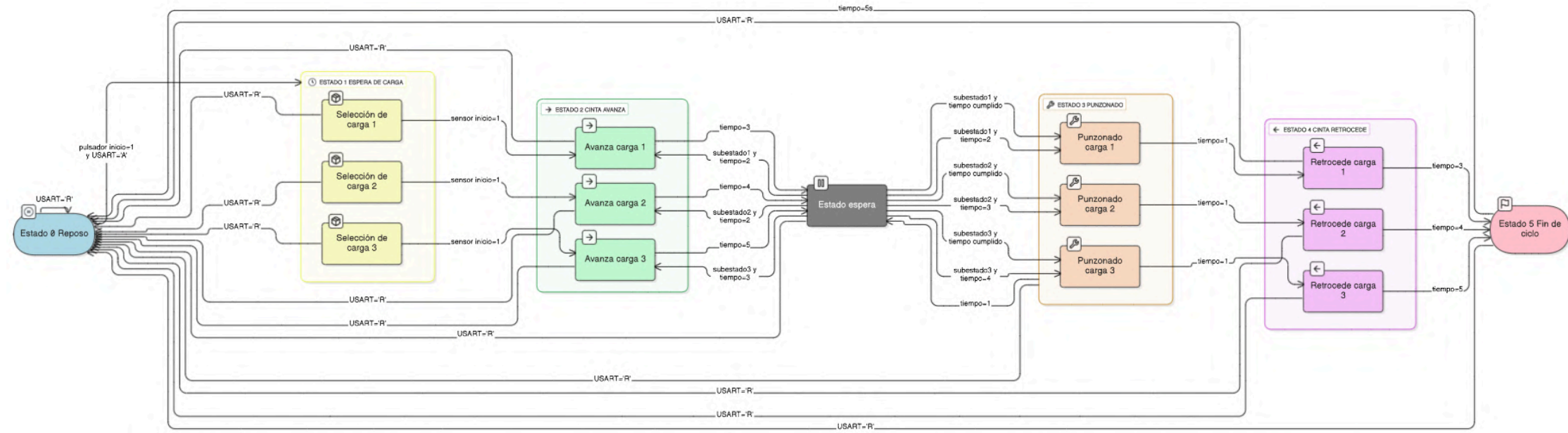
En cuanto a la implementación del plotter, se utilizó correctamente el monitor serial para la interacción y se comprendieron en detalle las formas de movimiento y la combinación de las mismas para generar figuras. La práctica permitió familiarizarse con el plotter como herramienta fundamental para el trazado automático, consolidando conocimientos sobre el control secuencial de actuadores y la coordinación de movimientos.

De las figuras propuestas, se lograron dibujar exitosamente dos de las tres planteadas: el triángulo y la cruz, sin presentar ningún inconveniente durante su ejecución. Sin embargo, la generación del círculo resultó demasiado compleja; a pesar de los varios intentos realizados, no se logró encontrar la lógica adecuada para su implementación, limitación atribuible parcialmente a nuestra experiencia como programadoras. No obstante, no afectó el aprendizaje adquirido. En general, los demás objetivos propuestos para el bloque del plotter se cumplieron satisfactoriamente, permitiendo validar los conceptos de control de movimientos.

En resumen, el laboratorio permitió consolidar el aprendizaje sobre diseño y control de sistemas mecatrónicos mediante microcontroladores, fortaleciendo la capacidad de integración de componentes físicos y su programación, generar señales analógicas a partir de datos digitales, controlar actuadores y periféricos, y validar experimentalmente los resultados obtenidos. A través de la experiencia adquirida, se comprendieron las limitaciones técnicas y las consideraciones prácticas que influyen en el desempeño de los sistemas, así como la importancia de la planificación, verificación y documentación rigurosa en la implementación de soluciones.

VIII. APÉNDICE

Apéndice I. Diagrama de máquina de estados Parte A: Cinta transportadora con Punzonadora:



Apéndice II.Código para diagrama de máquina de estados
Parte A: Cinta transportadora con Punzonadora:

```
title Diagrama de flujo: Máquina de estados
para control de cinta y punzonadora
direction right
```

```
Estado 0 Reposo [shape: oval, color:
lightblue, icon: pause-circle]
Estado 1 Espera de carga [color: yellow,
icon: clock] {
  Selección de carga 1 [icon: package,
color: yellow]
  Selección de carga 2 [icon: package,
color: yellow]
  Selección de carga 3 [icon: package,
color: yellow]
}
Estado 2 Cinta avanza [color: green, icon:
arrow-right] {
  Avanza carga 1 [icon: arrow-right, color:
green]
  Avanza carga 2 [icon: arrow-right, color:
green]
  Avanza carga 3 [icon: arrow-right, color:
green]
}
Estado espera [color: gray, icon: pause]
Estado 3 Punzonado [color: orange, icon:
tool] {
  Punzonado carga 1 [icon: tool, color:
orange]
  Punzonado carga 2 [icon: tool, color:
orange]
  Punzonado carga 3 [icon: tool, color:
orange]
}
Estado 4 Cinta retrocede [color: purple,
icon: arrow-left] {
  Retrocede carga 1 [icon: arrow-left,
color: purple]
  Retrocede carga 2 [icon: arrow-left,
color: purple]
  Retrocede carga 3 [icon: arrow-left,
color: purple]
}
Estado 5 Fin de ciclo [shape: oval, color:
pink, icon: flag]
```

```
// Relaciones y transiciones
// Inicio
Estado 0 Reposo > Estado 1 Espera de carga:
```

pulsador inicio=1 y USART='A'

```
// Selección de subestado en Estado 1
// Transición a Estado 2 según sensor inicio
Selección de carga 1 > Avanza carga 1:
sensor inicio=1
Selección de carga 2 > Avanza carga 2:
sensor inicio=1
Selección de carga 3 > Avanza carga 3:
sensor inicio=1
```

```
// Estado 2: Avance de cinta y transición a
espera o siguiente estado
Avanza carga 1 > Estado espera: tiempo=3
Avanza carga 2 > Estado espera: tiempo=4
Avanza carga 3 > Estado espera: tiempo=5
```

```
// Estado espera: regreso a Estado 2 o
avance a Estado 3 según temporización
// Estado espera > Estado 3 Punzonado (según
temporización)
Estado espera > Punzonado carga 1:
subestado1 y tiempo cumplido
Estado espera > Punzonado carga 2:
subestado2 y tiempo cumplido
Estado espera > Punzonado carga 3:
subestado3 y tiempo cumplido
```

```
// Estado 3: Punzonado y transición a espera
o siguiente estado
// Estado espera > Punzonado carga X (loop
si corresponde)
Estado espera > Punzonado carga 1:
subestado1 y tiempo=2
Estado espera > Punzonado carga 2:
subestado2 y tiempo=3
Estado espera > Punzonado carga 3:
subestado3 y tiempo=4
```

```
// Estado 3 > Estado 4 Cinta retrocede
Punzonado carga 1 > Retrocede carga 1:
tiempo=1
Punzonado carga 2 > Retrocede carga 2:
tiempo=1
Punzonado carga 3 > Retrocede carga 3:
tiempo=1
```

```
// Estado 4: Retroceso y transición a Estado
5
Retrocede carga 1 > Estado 5 Fin de ciclo:
tiempo=3
```

```

Retrocede carga 2 > Estado 5 Fin de ciclo:
tiempo=4
Retrocede carga 3 > Estado 5 Fin de ciclo:
tiempo=5

// Estado 5: Fin de ciclo y reinicio
// Transición global de reset desde
cualquier estado
Estado 0 Reposo > Estado 0 Reposo: USART='R'
Avanza carga 1 < Estado espera: subestado1 y
tiempo=2
Avanza carga 2 < Estado espera: subestado2 y
tiempo=2
Avanza carga 3 < Estado espera: subestado3 y
tiempo=3
Estado 0 Reposo < Estado 5 Fin de ciclo:
tiempo=5s
Estado 0 Reposo < Estado 1 Espera de carga:
USART='R'
Estado 0 Reposo < Estado 1 Espera de carga:
USART='R'
Estado 0 Reposo < Selección de carga 2:
USART='R'
Estado 0 Reposo < Selección de carga 3:
USART='R'
Estado 0 Reposo < Avanza carga 1: USART='R'
Estado 0 Reposo < Avanza carga 2: USART='R'
Estado 0 Reposo < Avanza carga 3: USART='R'
Estado 0 Reposo < Estado espera: USART='R'
Estado 0 Reposo < Retrocede carga 1:
USART='R'
Estado 0 Reposo < Retrocede carga 2:
USART='R'
Estado 0 Reposo < Retrocede carga 3:
USART='R'
Estado 0 Reposo < Estado 5 Fin de ciclo:
USART='R'
Estado espera < Estado 3 Punzonado: tiempo=1
Estado 0 Reposo < Estado 3 Punzonado:
USART='R'

```

IX. EVIDENCIAS

S. Modernell, V. Etchevery, y G. Sosa, [https://github.com/Victoriaetch/Microcontroladores/tree/main/Laboratorios/Laboratorio%201]. [Accedido: Septiembre, 2025]

REFERENCIAS

[1] Arduino, “Arduino Uno Rev3 – Reliable ATmega328P Board with Digital & Analog I/O,” *Arduino Store*. [Online]. Available: <https://store-usa.arduino.cc/products/arduino-uno-rev3>. [Accessed: Sep. 26, 2025].

[2] Wikipedia, “Atmega328,” *Wikipedia*, 20-Mar-2024. [Online]. Available: <https://es.wikipedia.org/w/index.php?title=Atmega328&oldid=158934266>. [Accessed: Sep. 20, 2025].

[3] Wikipedia, “Lenguaje ensamblador,” *Wikipedia, la enciclopedia libre*. [Enlace](#). [Accedido: 26-sep-2025].

[4] Cadence PCB Solutions, “USART vs. UART: What's the Difference?”, *Cadence PCB Design & Analysis*, 8 de febrero de 2023. [Enlace](#). [Accedido: 26-sep-2025].

[5] Visual Paradigm, “What is State Machine Diagram?”, *Visual Paradigm*. [Online]. Available: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-state-machine-diagram/>. [Accessed: Sep. 26, 2025].

[6] Fischertechnik, “Smart Beginner Set,” Fischertechnik. Available: <https://www.fischertechnik.biz/fischertechnik-bt-smart-beginner-set>. [Accessed: 26-sep-2025].

[7] Fischertechnik, “163330_bt_smart_beginner,” Fischertechnik. Available: https://ev1.utec.edu.uy/moodle/pluginfile.php/1281615/mod_resource/content/0/163330_bt_smart_beginner.pdf. [Accessed: 26-sep-2025].

[8] Electronics Tutorials, “R-2R DAC (R-2R Digital-to-Analogue Converter),” *Electronics Tutorials*, [Online]. Available: <https://www.electronics-tutorials.ws/combinatoin/r-2r-dac.html>. [Accessed: 26-Sep-2025].

[9] Talos Electronics. (s.f.). Puente H Doble L298N. Recuperado el 21 de junio de 2025, de <https://www.taloselectronics.com/blogs/tutoriales/puente-h-doble-l298n>

[10] Uelectronics, “Cómo conectar y programar la Matriz LED 8×8 1088AS,” *Uelectronics Blog*, 20-Mar-2024 (aproximado). [Online]. Available: <https://blog.uelectronics.com/tarjetas-desarrollo/arduino/como-programar-y-a-hacer-la-conexion-matriz-led8x8/>. [Accessed: Sep. 20, 2025].

Laboratorio 2: GPIO en C

Sofía Modernell, Guadalupe Sosa, Victoria Etcheverry

Carrera: Ingeniería en Mecatrónica, Universidad Tecnológica (UTEC)

Fray Bentos, Uruguay

victoria.etccheverry@estudiantes.utec.edu.uy, sofia.modernell@estudiantes.utec.edu.uy, guadalupe.sosa@estudiantes.utec.edu.uy

20 de octubre de 2025

Resumen—Este informe presenta el diseño, simulación e implementación de cuatro sistemas embebidos basados en el microcontrolador ATmega328P, programados en lenguaje C a bajo nivel. La metodología combinó la simulación con el montaje físico, y la gestión del código se realizó mediante control de versiones en GitHub. Los proyectos desarrollados incluyen: el control de un pen plotter vía UART para dibujar figuras, un detector de color que utiliza una fotocelda y un LED RGB como filtro activo, un piano electrónico de ocho notas gestionado por una máquina de estados, y una cerradura digital con teclado matricial, interfaz en LCD por I2C y almacenamiento de contraseña no volátil en la memoria EEPROM.

Keywords: ATmega328P, Lenguaje C, UART, I2C, EEPROM, Pen Plotter, Teclado Matricial, LCD, Servomotor, Fotocelda, NeoPixel.

Abstract—This report presents the design, simulation, and implementation of four embedded systems based on the ATmega328P microcontroller, programmed in low-level C language. The methodology combined simulation with physical assembly, and code management was performed using version control on GitHub. The developed projects include: the control of a pen plotter via UART to draw figures; a color detector using a photocell and an RGB LED as an active filter, an eight-note electronic piano managed by a state machine, and a digital lock with a matrix keypad, an LCD interface via I2C, and non-volatile password storage in EEPROM.

I. INTRODUCCIÓN

En el presente informe se aborda el diseño, simulación e integración de soluciones basadas en el ATmega328P. El trabajo se centra en la aplicación práctica de técnicas de control, sensado y comunicación serial (UART e I2C) para desarrollar sistemas capaces de interactuar con su entorno mediante una variedad de periféricos, como sensores, actuadores y dispositivos de salida visual o auditiva.

Para alcanzar este objetivo, el trabajo se articula en cuatro proyectos principales que demuestran la integración de estos conceptos. Estos proyectos son: el control de un pen plotter para el dibujo de figuras seleccionables por menú; un sistema de detección y emulación de colores; un piano electrónico con modos de reproducción manual y automática; y una cerradura digital con validación de contraseña almacenada en la memoria EEPROM.

La metodología empleada incluyó la programación en C, la simulación en entornos como Proteus y PicSimLab, y la gestión de código mediante control de versiones en GitHub. Este documento detalla el marco teórico de los componentes utilizados y la metodología de implementación para cada uno de los cuatro sistemas desarrollados.

II. OBJETIVOS

Objetivo general

Diseñar, simular e integrar soluciones basadas en el microcontrolador ATmega328P, aplicando técnicas de control, sensado y comunicación serial para el desarrollo de sistemas capaces de interactuar con su entorno mediante sensores, actuadores y dispositivos de salida visual o auditiva.

Objetivos específicos:

- Programar el control de un plotter, utilizando comunicación serial para permitir al usuario la selección de dibujos a realizar mediante un menú interactivo.
- Implementar un sistema de detección de colores mediante fotocelda y sean emulados mediante un servomotor, tira de LEDs y monitor serial.
- Desarrollar un piano electrónico de ocho notas, con reproducción manual mediante pulsadores y selección de canciones predefinidas mediante comandos UART.
- Diseñar una cerradura digital con teclado matricial y LCD, capaz de validar contraseñas y activar una alarma tras 3 intentos fallidos.
- Gestionar los códigos en lenguaje C bajo nivel utilizando GitHub, con control de versiones y documentación de las modificaciones realizadas.

III. MATERIALES UTILIZADOS

TABLE I

MATERIALES UTILIZADOS

MATERIALES	CANTIDAD
Arduino Uno Rev3	1
Protoboard	1
Fuente de DC	1
Pulsadores	8
LED roja	1
LED verde	1
Resistencia 10kΩ	1
Fotocelda	1
Tira/Matriz de LEDs	1
Servomotor	1
LED RGB ks	1
Resistencia 1kΩ	8
Buzzer	1
LCD 16x2	1
Módulo PCF8574	1
Keypad/Teclado Matricial	1
Cables	-
Pen Plotter	1

Microchip Studio	-
Arduino IDE	-
Proteus	-
PicSimLab	-

IV. MARCO TEÓRICO

Arduino UNO Rev3

El Arduino Uno Rev3 es una placa de desarrollo ampliamente utilizada en el ámbito educativo y en el prototipado de sistemas electrónicos, principalmente por su facilidad de uso, bajo costo y la gran comunidad que respalda su ecosistema. Esta placa se basa en el microcontrolador ATmega328P.

El dispositivo funciona con una tensión nominal de 5 V regulados y puede ser alimentado tanto a través del puerto USB como mediante una fuente externa, recomendándose un rango de voltaje entre 7 V y 12 V para asegurar un rendimiento estable. Esta flexibilidad en la alimentación facilita su integración en diferentes entornos y aplicaciones.

En cuanto a su arquitectura de pines, el Arduino Uno dispone de 14 pines digitales de entrada/salida, de los cuales 6 pueden ser utilizados como salidas PWM, además de 6 entradas analógicas identificadas como A0 a A5. La placa incluye también pines de referencia y alimentación, como VIN, AREF, conexiones a tierra, así como salidas de 3.3 V y 5 V para alimentar periféricos externos. Asimismo, incorpora interfaces de comunicación que permiten su interacción con otros dispositivos electrónicos: UART (pines 0 y 1), SPI (pines 10 a 13) e I²C (pines A4 y A5). Esta diversidad de opciones convierte al Arduino Uno en una plataforma versátil para la interconexión de sensores, actuadores y módulos de expansión.

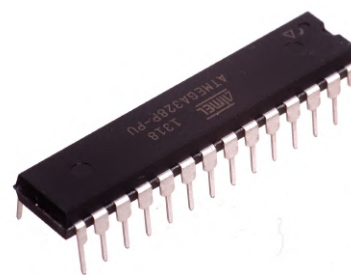


Fig.2 Microcontrolador ATmega328P.

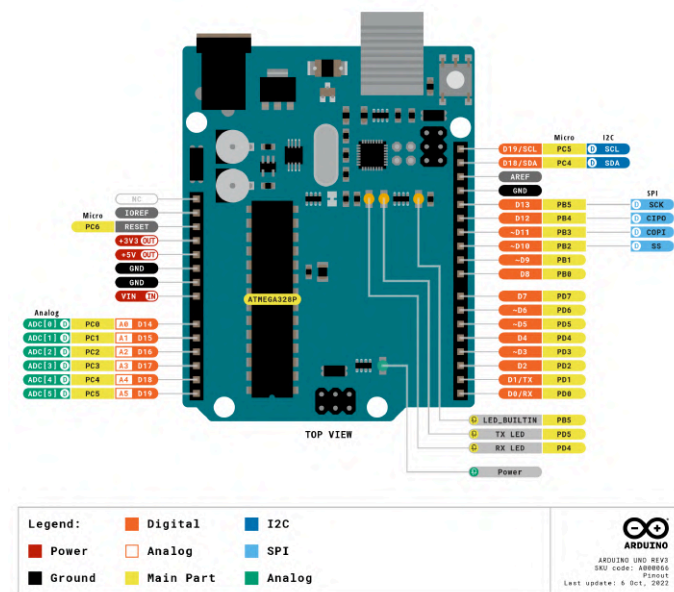


Fig.1 Arduino Pinout

ATmega328p

El ATmega328P es un microcontrolador de 8 bits de la serie megaAVR desarrollado por Atmel, ampliamente utilizado en sistemas embebidos y proyectos de electrónica de bajo consumo y bajo costo. Su arquitectura se basa en un microcontrolador RISC, lo que le permite ejecutar instrucciones en un solo ciclo de reloj, alcanzando hasta 1

MIPS, optimizando la velocidad de procesamiento y el consumo energético.

Entre sus principales características se destacan:

- Memoria: 32 KB de memoria flash (ISP) con capacidad de lectura mientras se escribe, 1 KB de memoria EEPROM y 2 KB de SRAM.
- Entradas/Salidas: Hasta 23 líneas de E/S de propósito general, 32 registros de propósito general y 6 canales PWM.
- Convertidor A/D: 6 canales de 10 bits, que permiten la adquisición de señales analógicas de los sensores.
- Temporizadores: 3 temporizadores flexibles con modo de comparación y un temporizador “watchdog” programable.
- Interfaz de comunicación: Soporta USART, SPI y comunicación serie de 2 hilos.
- Modos de ahorro de energía: Cinco modos configurables por software, que optimizan la autonomía del dispositivo en aplicaciones portátiles.
- Rango de operación: Funciona entre 1,8 y 5,5 V, lo que lo hace compatible con distintos sistemas de alimentación.

El ATmega328P se utiliza en proyectos que requieren control preciso de periféricos, adquisición de datos y transmisión de información, siendo la base de plataformas como Arduino Uno y Nano.

Lenguaje C

C es un lenguaje de programación de propósito general desarrollado por Dennis Ritchie en los Laboratorios Bell entre 1969 y 1972, orientado originalmente a la implementación de sistemas operativos como Unix. Es valorado por la eficiencia del código que genera y se considera un lenguaje de nivel medio, pues combina estructuras de alto nivel con un control de bajo nivel, como el acceso directo a la memoria mediante punteros. El lenguaje posee un núcleo simple que se complementa con bibliotecas y utiliza un preprocesador para tareas como la inclusión de archivos. Carece de recolección de basura nativa, por lo que el programador es responsable de gestionar la memoria explícitamente.

UART

El protocolo UART (Universal Asynchronous Receiver Transmitter) es fundamental en sistemas embebidos para la transmisión de datos entre dispositivos.

El UART es un protocolo de comunicación serial asíncrona que utiliza dos líneas: una para la transmisión (TX) y otra para la recepción (RX). Su principal característica es que no requiere una señal de reloj externa, ya que la sincronización entre el transmisor y el receptor se establece mediante la configuración previa de parámetros como la velocidad de transmisión (baud rate), bits de datos, paridad y bits de parada.

Esta simplicidad lo hace ideal para comunicaciones de corto alcance y baja velocidad, como la conexión entre microcontroladores y módulos periféricos.

Pen Plotter

Un pen plotter es un dispositivo automatizado diseñado para dibujar imágenes o diagramas sobre una superficie plana utilizando un bolígrafo, marcador u otro instrumento de escritura. A diferencia de las impresoras convencionales, que aplican tinta mediante procesos de impresión, las pen plotters mueven el instrumento de manera controlada sobre los ejes X e Y, permitiendo reproducir con precisión figuras vectoriales.

En su funcionamiento típico, la pen plotter utiliza motores paso a paso para controlar el movimiento horizontal y vertical del bolígrafo, mientras que un mecanismo adicional permite levantar y bajar el instrumento de escritura según sea necesario. La máquina puede ser controlada mediante comandos de control de movimiento, como los que se encuentran en el G-code, lo que permite programar trayectorias complejas y repetir dibujos de manera consistente.

Este tipo de dispositivos se emplea tanto en proyectos educativos como en aplicaciones de prototipado rápido y automatización, ofreciendo una plataforma práctica para comprender conceptos de control de movimiento, sincronización de ejes y coordinación mecánica, sin necesidad de equipos industriales complejos.

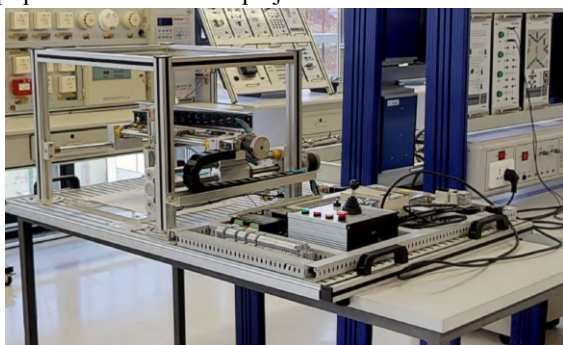


Fig.3 Pen Plotter

TABLE II

CONEXIONES PEN PLOTTER

Pin Digital	Conexión
D2	Bajar solenoide
D3	Subir solenoide
D4	Movimiento hacia abajo
D5	Movimiento hacia arriba
D6	Movimiento hacia la derecha
D7	Movimiento hacia la izquierda

Fotocelda

Una fotorresistencia, también conocida como resistor dependiente de la luz o LDR, es un componente electrónico cuya resistencia eléctrica se modifica según la intensidad de la luz incidente. Normalmente, su resistencia disminuye a medida que aumenta la luz. El valor resistivo puede ser muy alto en la oscuridad, alcanzando varios megaohmios, y descender significativamente, hasta unos 100 ohmios o menos, bajo una iluminación brillante.

Este componente basa su funcionamiento en el efecto fotoeléctrico. Está fabricado con un material semiconductor de alta resistencia, como el sulfuro de cadmio. Cuando la luz

incide sobre el material, los fotones proporcionan energía a los electrones para saltar a la banda de conducción, permitiendo el flujo de electricidad y disminuyendo así la resistencia total. Estos dispositivos tienen un tiempo de respuesta relativamente lento, del orden de una décima de segundo, lo que limita su uso en aplicaciones de alta velocidad, pero resulta útil para filtrar variaciones rápidas o para aplicaciones generales como el control del alumbrado público o en medidores de luz.



Fig.4 Fococelda

Matriz/Tira NeoPixel (WS2812)

Los LED RGB direccionables, conocidos popularmente como NeoPixel o por su designación técnica WS2812, son una familia de componentes LED que se presentan en tiras, matrices u otros formatos. Estos dispositivos permiten un control individualizado de cada LED, facilitando la creación de efectos de iluminación complejos. Cada píxel suele consistir en un encapsulado 5050 que integra tres LED (rojo, verde y azul) junto con un chip controlador, como el WS2811 o el WS2812. Esto los diferencia de las tiras analógicas, donde todos los LED están conectados en paralelo y muestran el mismo color [1].

El control de estas tiras digitales se realiza mediante una única línea de datos. Los píxeles se pueden encadenar conectando el pin de salida de datos de uno al pin de entrada del siguiente, requiriendo solo tres conexiones principales: Vcc (generalmente 5V), GND (tierra) y Datos. Debido a su consumo de energía, que puede alcanzar hasta 60 mA por píxel a máximo brillo, no deben alimentarse directamente desde un microcontrolador como Arduino. Se requiere una fuente de alimentación externa de 5V dimensionada para la corriente total de la tira.

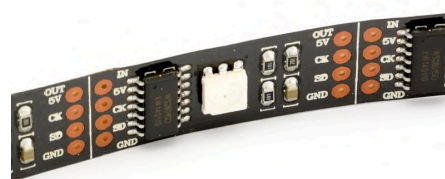


Fig.5 Tiras LED RGB direccionables

Servomotor

Un servomotor, también llamado servo, es un dispositivo similar a un motor de corriente continua pero que incorpora un sistema de regulación. Esta característica le permite ser controlado con precisión tanto en velocidad como en posición, y mantenerse estable en un punto específico dentro de su rango de operación. Internamente, está conformado por un motor eléctrico, un conjunto de engranajes, un potenciómetro para la retroalimentación de la posición y un circuito de control.

El método de control estándar para los servomotores es la modulación por ancho de pulsos (PWM), generalmente

operando a una frecuencia de 50 Hz. La electrónica del servo interpreta el ancho del pulso recibido para determinar la posición deseada; por ejemplo, un pulso de 1.5 milisegundos suele corresponder a la posición neutral, mientras que pulsos más cortos o más largos mueven el eje en direcciones opuestas. Tradicionalmente están limitados a rangos como 180 grados.



Fig.6 Servomotor

LED RGB ks

Un LED RGB es un componente que combina tres diodos (rojo, verde y azul) en un solo encapsulado. Permite generar una amplia gama de colores mediante el control del brillo individual de cada uno de estos tres LED internos. Por ejemplo, al encender el rojo y el azul a máxima intensidad mientras se apaga el verde, se obtiene el color púrpura. El módulo LED RGB KS, facilita la implementación al incluir las resistencias de 220 ohmios necesarias directamente en el circuito.

Este módulo en particular es de tipo cátodo común, lo que significa que los tres LED de color comparten una conexión a tierra. Dispone de cuatro pines etiquetados como R, G, B y – (negativo). Para su correcto funcionamiento, los pines R, G y B deben conectarse a pines del microcontrolador que soporten la modulación por ancho de pulsos (PWM), como los pines 3, 5, 6, 9, 10 y 11 en un Arduino Uno, mientras que el pin negativo debe conectarse a tierra (GND).

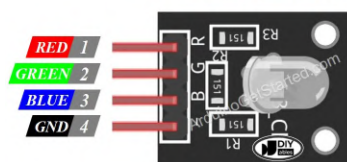


Fig.7 LED RGB ks Pinout

Diagrama de máquina de estados

El diagrama de máquina de estados en UML es un diagrama de comportamiento que describe cómo un objeto cambia entre distintos estados a lo largo de su ciclo de vida, en función de eventos que ocurren. Un estado representa una situación en la que el objeto cumple ciertas condiciones o ejecuta actividades específicas, mientras que una transición muestra el paso de un estado a otro, provocado por un evento y opcionalmente condicionado por una guarda o acompañado de una acción.

Dentro del diagrama se incluyen símbolos estandarizados como el estado inicial (círculo sólido) y el estado final (círculos concéntricos). También es posible modelar estados compuestos, que contienen subestados, así como regiones ortogonales, que permiten representar comportamientos concurrentes. En algunos casos se emplean estados de historia, que retoman el subestado previamente activo en lugar de reiniciar.

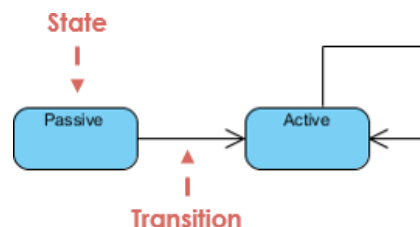


Fig.8 Estado y transición en diagramas de máquinas de estado Buzzer pasivo

Un zumbador es un transductor electroacústico que genera un sonido o zumbido de un mismo tono, ya sea continuo o intermitente, y se utiliza comúnmente como mecanismo de señalización o aviso. Su construcción puede basarse en un sistema electromecánico o en un disco piezoeléctrico junto a una lámina metálica. Una distinción importante es entre zumbadores activos y pasivos. El zumbador pasivo carece de un oscilador interno, por lo que necesita una señal de frecuencia externa para poder producir un sonido. Aunque esto lo hace más complejo de controlar, requiriendo una señal de entrada que varíe en frecuencia, como una PWM generada por un microcontrolador, ofrece mayor flexibilidad y es adecuado para generar diferentes tonos musicales o señales sonoras complejas.



Fig.9 Buzzer

TABLE III

FRECUENCIAS ACÚSTICAS TRADUCIDAS EN HERTZ

TONOS	OCTAVA		
	3	4	5
C	131	262	523
D	147	294	587
E	165	330	659
F	175	349	698
G	196	392	784
A	220	440	880
B	247	494	988

LCD 16x2

Las pantallas de cristal líquido (LCD) de caracteres, como los modelos 16x2 o 20x4, son componentes comunes para mostrar datos como texto o símbolos en proyectos. Su funcionamiento se basa en el uso de cristales líquidos que, al recibir una corriente eléctrica, cambian su alineación para modular una luz de fondo, permitiendo o bloqueando selectivamente el paso de la luz a través de filtros polarizados para formar los caracteres en la pantalla. La mayoría de estas pantallas están basadas en el controlador Hitachi HD44780.

Estas pantallas suelen tener 16 pines. Estos incluyen los pines de alimentación VCC y GND, los pines A y K para alimentar la luz de fondo, y un pin V0 que se conecta a un potenciómetro para ajustar el contraste de la pantalla. Para la comunicación, se utilizan los pines RS (Register Select), para

diferenciar entre comandos y datos de caracteres, y E (Enable), para indicarle a la pantalla cuándo debe leer los datos. Aunque la pantalla dispone de ocho pines de datos (D0-D7) para operar en modo de 8 bits, comúnmente se utiliza el modo de 4 bits. Este modo ahorra conexiones al microcontrolador, ya que solo requiere el uso de cuatro líneas de datos (D4-D7) para enviar la información en dos partes.

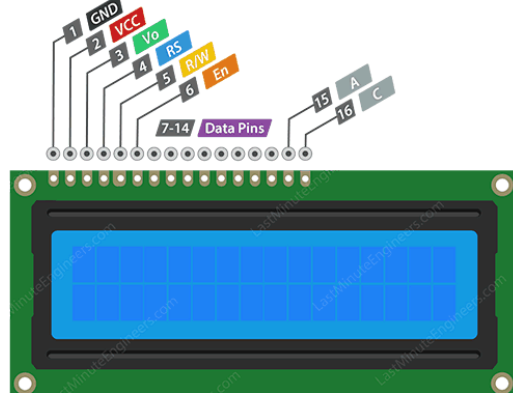


Fig.10 LCD 16X2 Pinout

Keypad/Teclado Matricial

Un teclado matricial es un dispositivo de entrada que permite a los usuarios interactuar con un proyecto, por ejemplo, para introducir contraseñas o navegar por menús. Está compuesto por un conjunto de pulsadores organizados en una matriz de filas y columnas, siendo las configuraciones más habituales las de 3x4 y 4x4. Su estructura interna suele constar de tres membranas superpuestas: dos capas conductoras separadas por una intermedia no conductora. Cuando se presiona una tecla, las dos membranas conductoras entran en contacto, permitiendo el flujo de corriente.

Para detectar qué tecla específica ha sido pulsada, el microcontrolador debe identificar la posición (X, Y) de la tecla dentro de la matriz. Esto se logra determinando la fila y la columna que se han conectado en el momento de la pulsación.

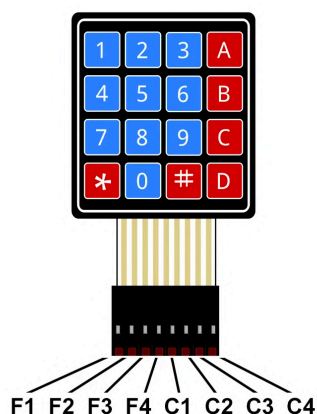


Fig.11 Keypad/Teclado Matricial Pinout

EEPROM

EEPROM son las siglas de Electrically Erasable Programmable Read-Only Memory, que se traduce como memoria de solo lectura programable y borrable eléctricamente. Es un tipo de memoria ROM no volátil, lo que significa que retiene los datos incluso sin alimentación eléctrica. A diferencia de las EPROM, que deben borrarse exponiéndolas a rayos ultravioleta, las EEPROM pueden

programarse, borrarse y reprogramarse utilizando únicamente impulsos eléctricos.

Las celdas de memoria de una EEPROM están constituidas por transistores MOS con una compuerta flotante. Aunque estos dispositivos pueden leerse un número ilimitado de veces, el número de ciclos de borrado y escritura es limitado, situándose habitualmente entre cien mil y un millón de ciclos. Para la comunicación, las EEPROM suelen emplear protocolos estándar como I²C o SPI, o bien se integran directamente en chips como microcontroladores para lograr una mayor rapidez de acceso. La memoria flash es considerada una forma avanzada de EEPROM.

I²C

I²C, sigla de Inter-Integrated Circuit, es un bus de datos serie desarrollado por Philips en 1982, diseñado para la comunicación interna entre diferentes circuitos integrados dentro de un mismo equipo, como un microcontrolador y sus periféricos. Funciona bajo un protocolo maestro-esclavo, donde el dispositivo maestro es el que inicia la transferencia de datos. Este bus también es conocido como TWI (Two-Wire Interface), una designación introducida por Atmel que es técnicamente idéntica.

El bus I²C solo requiere dos líneas de señal: una para los datos (SDA) y otra para el reloj (SCL). Ambas líneas son de drenador abierto y requieren resistencias de pull-up. Esta configuración permite que múltiples dispositivos se conecten al mismo bus. La comunicación se inicia con una condición de arranque y los datos se transfieren en paquetes de 8 bits (octetos), donde cada paquete es seguido por un bit de confirmación (ACK) del receptor. El direccionamiento estándar utiliza 7 bits, lo que permite hasta 112 nodos en el bus, aunque se introdujo un modo de 10 bits para ampliar esta capacidad. La principal ventaja de I²C es su capacidad para controlar una red de dispositivos usando solo dos pines de E/S, lo que reduce la complejidad y el costo, siendo comúnmente usado para sensores, memorias EEPROM y relojes en tiempo real.

V. METODOLOGÍA

En el presente laboratorio se llevaron a cabo actividades basadas en la programación en lenguaje C aplicadas al microcontrolador ATmega328P. La práctica estuvo compuesta por cuatro actividades principales: la utilización de un plotter para llevar a cabo ciertas figuras, un detector de colores, un piano electrónico y una cerradura digital. A continuación, se detalla el procedimiento correspondiente a cada una de estas etapas.

Parte A:

En este problema se plantea la utilización de un plotter, mediante el desarrollo de un código en C, para realizar figuras geométricas y pictóricas: triángulo, cruz, círculo, gato y rana. La metodología se dividió en dos fases clave: la adaptación de figuras ya existentes y el desarrollo e implementación de las nuevas figuras.

Para las primeras tres figuras se aprovechó la disponibilidad del código en lenguaje ensamblador desarrollado en laboratorios previos. El proceso en esta fase consistió en la traducción directa y verificación de dicho código al lenguaje C, asegurando la lógica de movimientos y la secuencia de pasos del plotter.

Con respecto a las figuras nuevas (rana y gato) se llevó a cabo un proceso iterativo y detallado para lograr su implementación. Inicialmente, se seleccionaron imágenes cuadrículadas de estas figuras, las cuales fueron replicadas a mano alzada en una cuadrícula de referencia, permitiendo un mejor conocimiento de los centímetros y movimientos empleados. Una vez hecho esto, se tradujo los centímetros a paso y se realizó un conteo de todos los movimientos realizados (arriba, abajo, derecha, izquierda, diagonal arriba y diagonal abajo), para la implementación de estos datos en la plataforma de simulación Tutule de Python. Esta herramienta facilitó la visualización directa de las figuras generadas con la secuencia de pasos contada, lo que permitió la rápida identificación y corrección de errores de trayectoria. Finalmente, una vez validados los parámetros en la simulación, el código de Python se transcribió al lenguaje C, integrando las secuencias de pasos específicas mediante la reutilización y adaptación de las funciones de control del plotter ya definidas.

En relación al código, el mismo se compone de varias secciones que permiten el movimiento coordinado del sistema. En primer lugar, se incluyen las librerías necesarias: `avr/io.h` para acceder a los registros del microcontrolador, `util/delay.h` para generar retardos, `math.h` para realizar operaciones trigonométricas y cálculos de desplazamiento, y `stdio.h` para utilizar la comunicación serial a través de funciones como `printf()`.

Luego se definen los pines, PD2 y PD3 se destinan al solenoide, encargado de subir y bajar el lápiz, mientras que los pines PD4 a PD7 controlan los movimientos en los ejes X e Y del plotter. Estas definiciones permiten un código más legible y facilitan la identificación de cada función física dentro del programa. También se configura la comunicación serial (UART), definiendo la frecuencia del microcontrolador ($F_{CPU} = 16 \text{ MHz}$) y la velocidad de transmisión (BAUD = 9600 bps). A partir de estos valores se calcula el registro UBRR_VAL, que asegura una comunicación estable entre el microcontrolador y el dispositivo externo. En la función de inicialización se habilita la transmisión y recepción de datos, y se establece el formato de comunicación con 8 bits de datos y un bit de parada. Además, se configura el puerto D para que los pines encargados del control de motores y solenoide funcionen como salidas, inicializándolos en nivel bajo para evitar movimientos no deseados al iniciar el sistema.

Por último, el código incluye también las funciones de movimiento manual a elección del usuario. Cada una de ellas representa un desplazamiento específico: arriba, abajo, derecha e izquierda, y también las funciones de subir y bajar el lápiz mediante el solenoide. Cada movimiento se ejecuta activando los pines correspondientes del puerto D durante un intervalo de tiempo controlado con `_delay_ms()`, lo que determina la cantidad de pasos realizados por el motor. Estas funciones pueden combinarse para generar trayectorias más complejas, como líneas diagonales o curvas aproximadas, así como la elección del lugar de dibujo dentro de la hoja.

Finalmente, se verifica el correcto funcionamiento del plotter y se realizan los ajustes necesarios a los movimientos. Una vez obtenido el resultado esperado, se documentó el proceso completo, garantizando la trazabilidad del desarrollo y la claridad en la implementación del sistema.

Parte B:

Este apartado corresponde a la implementación de un sistema de selección y clasificación de colores basado en un microcontrolador ATmega328P, una fotocelda (LDR), una tira de LEDs WS2812B, un servomotor y una LED RGB KS. El sistema funciona de tal manera que, al direccionar la fotocelda hacia uno de los colores del círculo cromático, el servomotor debe señalar ese mismo color, mientras que la tira WS2812B se ilumina con el tono correspondiente y se envía mediante UART la información identificada (nombre del color y valor ADC).

El proceso de detección se basa en la lectura secuencial de los tres componentes cromáticos (rojo, verde y azul), utilizando la LED RGB KS como fuente de iluminación selectiva. Esta LED actúa como un filtro óptico controlado, permitiendo obtener las intensidades reflejadas por el color bajo prueba para cada componente RGB. Dichas lecturas son procesadas mediante una función de promediado y normalización, lo que permite determinar con mayor precisión la posición del color dentro del espectro visible.

El sistema incluye un método de calibración de colores, necesario porque la luz percibida por la fotocelda varía según el entorno y las condiciones de iluminación. Durante la calibración, se toman múltiples lecturas de cada color con los filtros RGB, y se calcula el promedio de las mediciones para cada componente (R, G, B), definiendo así el vector de referencia del color. Además, se determina la variación de las lecturas (desviación típica) para establecer un margen de tolerancia alrededor del vector de referencia; este margen permite que pequeñas diferencias en la iluminación o la posición del sensor no afecten la identificación. De este modo, mientras más se recorra el espectro del color durante la calibración, mayor es la precisión en la clasificación.

Para identificar un color, se calcula la distancia euclidiana entre la medición actual y los vectores de referencia en el espacio RGB. El color correspondiente a la menor distancia dentro del margen de tolerancia es seleccionado como el detectado; si ninguna medición entra dentro del margen, el color se considera “indefinido”.

Entre las funciones implementadas en el código se destacan `ADC_Promedio()`, encargada del filtrado y promedio de las señales analógicas; `detectar_color_vectorial()`, que realiza el cálculo vectorial y la selección del color más próximo en el espacio RGB; `calibrateCurrentColor()`, responsable de obtener los parámetros estadísticos durante el proceso de calibración; y por último, `setNeoPixelColor()` y `Servo_Move_Delay()`, que gestionan la retroalimentación visual mediante la tira WS2812B y el movimiento del servomotor, respectivamente.

El código implementado en el montaje físico fue desarrollado en Arduino IDE, incluyendo todas las funcionalidades de calibración y control del servo y los LEDs. Por su parte, la simulación se ejecutó en AVR puro (C) y se utilizó para validar la lógica de detección y clasificación de colores; sin embargo, la simulación no incluye el método de calibración, por lo que los valores de referencia fueron tomados de mediciones predefinidas. Finalmente, el sistema fue probado en ambos entornos, verificando la correcta correspondencia entre color detectado, iluminación y movimiento del servomotor, así como la comunicación estable mediante UART.

Parte C:

Este problema se basa en la implementación de un piano electrónico desarrollado en lenguaje C para un ATmega328p. El sistema cuenta con ocho pulsadores que simulan las teclas

del instrumento, de manera que cada uno de ellos representa una nota musical específica: Do, Re, Mi, Fa, Sol, La, Si y Do agudo. Estos sonidos se reproducen en un buzzer al seleccionar la frecuencia correspondiente a cada nota. Además, el piano cuenta con tres opciones, las cuáles se gestionan mediante comunicación UART: reproducir canción 1, reproducir canción 2 y modo piano, este último permite al usuario tocar las notas manualmente.

El programa se estructura en torno a una máquina de estados finitos, que define los modos de operación como ESTADO_P (modo piano), ESTADO_1 (canción 1) y ESTADO_2 (canción 2), se puede encontrar el diagrama de máquinas de estados en el Apéndice I y el código fuente utilizado para su generación en el Apéndice II. Cada estado ejecuta tareas específicas a través de distintas funciones, permitiendo un control claro y modular del flujo del programa. El cambio entre modos se realiza mediante comandos enviados por UART: el carácter '1' activa la reproducción de la primera canción, '2' la segunda, y 'S' detiene la ejecución para regresar al modo piano libre.

El primer paso consistió en la elección y traducción de las partituras, las cuáles se encuentran en el Apéndice III y IV, de ambas canciones a tocar, convirtiendo las notas musicales en sus frecuencias acústicas equivalentes (expresadas en Hertz). Se utilizaron valores estándar para las notas comprendidas entre las octavas G3 y C5, los cuales fueron definidos como constantes en el código, estableciendo la frecuencia precisa de cada tono. Dicha frecuencia representa la cantidad de ciclos de onda sonora que el sistema debe generar por segundo. Para el modo de piano libre, se implementó un arreglo que asocia cada pulsador físico con la frecuencia (en Hz) correspondiente a su nota musical.

Para la ejecución automática de melodías, las figuras musicales (negra, corchea y blanca) se tradujeron a duraciones temporales en milisegundos (ms). La negra se tomó como unidad de tiempo base, mientras que las demás figuras derivan de ella: la corchea equivale a la mitad y la blanca al doble. En el caso de figuras puntilladas, la duración se calculó multiplicando el valor base por $3/2$, respetando las proporciones musicales originales.

El código principal se organiza en distintas funciones de control y lectura, lo que permite mantener un funcionamiento ordenado. La función `main()` inicializa los puertos, configura la comunicación UART y gestiona el bucle principal del programa. Por su parte, la función `tarea_principal_p()` gestiona la lectura de los pulsadores y la reproducción de notas en modo libre, mientras que `chequear_interrupcion_uart()` y `chequear_interrupcion_rapida()` supervisan de manera constante la entrada de comandos para modificar el comportamiento del sistema en tiempo real. De este modo, el usuario puede alternar entre reproducir canciones o tocar manualmente sin necesidad de reiniciar el microcontrolador.

Finalmente, el teclado fue implementado tanto en simulación mediante Proteus como en montaje físico, validando su funcionamiento en ambos entornos. Esta verificación permitió comprobar la correcta correspondencia entre las notas, las frecuencias y las duraciones musicales, además de confirmar la estabilidad de la comunicación UART y la exactitud temporal de los tonos.

Parte D :

Este proyecto plantea la implementación de una cerradura electrónica que permite el ingreso y la modificación de una

contraseña de acceso, almacenada de forma permanente en la memoria interna del microcontrolador. El sistema utiliza un teclado matricial y una pantalla LCD que trabajan en conjunto para crear un hardware interactivo y funcional, capaz de enviar mensajes visuales y responder de forma inmediata ante cada acción.

La solución puede dividirse en cuatro módulos principales: comunicación (I2C), interfaz de usuario (teclado y LCD), almacenamiento (EEPROM) e integración y funcionamiento general.

La comunicación entre el microcontrolador y la pantalla LCD se lleva a cabo mediante el protocolo I2C, para relacionar estos dos conceptos se utiliza un módulo PCF8574 con una dirección 0x27. Este componente actúa como expansor de puertos y posibilita el control del LCD a través de solo dos líneas de comunicación (SDA y SCL), conectadas a los pines PC4 y PC5 del microcontrolador. Para simplificar la implementación se emplearon 3 librerías que permitieron tanto la inicialización del sistema como la interacción con la pantalla LCD: `twi.c`, `twi.h` y `twi_lcd.h`. Las primeras dos contienen funciones de bajo nivel necesarias para gestionar la comunicación, mientras que la tercera incluye las funciones específicas para inicializar la pantalla, ubicar el cursor y mostrar mensajes.

Con respecto a la interfaz de usuario, se utilizó una matriz de 4 filas por 4 columnas, permitiendo un alcance de 16 teclas diferentes. Las filas se conectan al puerto C (de A0 a A3) y las columnas al puerto D (de D2 a D6), configuradas como entradas con resistencias de pull-up activadas. El sistema de lectura del teclado consiste en activar una fila a la vez y leer el estado de las columnas para determinar qué tecla se ha presionado. Para esto, se utiliza un arreglo que asocia la posición física de cada tecla con el carácter que representa, lo que permite identificar de manera precisa la entrada del usuario.

La memoria EEPROM interna del microcontrolador cumple la función de almacenamiento permanente de la contraseña, lo que permite apagar o reiniciar el dispositivo sin pérdida alguna. Esta parte consta de 2 funciones principales: una destinada a guardar la clave actual y otra a leer la clave previamente almacenada. Destacando que al inicio del programa, cuando aún no se ha declarado una nueva contraseña, la memoria contiene valores vacíos (0xFF) por lo que el sistema inicializa la clave por defecto como "1234".

Durante el funcionamiento, la pantalla LCD muestra inicialmente un mensaje de bienvenida y luego presenta un menú con dos opciones principales: ingresar la clave o cambiar la clave. Si se selecciona la primera opción, el usuario debe introducir su clave para verificar el acceso, si es correcta se enciende un led verde, en cambio, si es incorrecta, se enciende un led rojo. Además, el sistema incorpora una medida de seguridad basada en la activación de una alarma sonora mediante un buzzer tras 3 intentos fallidos.

En el caso de seleccionar la opción de cambio de clave, el sistema solicita primero la contraseña actual y, una vez verificada, permite ingresar una nueva. Esta nueva clave debe tener entre cuatro y seis dígitos, en el caso de que la cantidad sea menor se visualiza un mensaje que dice "Min 4 dígitos" y rechazando una longitud mayor. Posteriormente la nueva contraseña se almacena nuevamente en la EEPROM, reemplazando la anterior. Durante el ingreso de la contraseña,

la pantalla LCD muestra asteriscos (*) en lugar de los dígitos reales, lo que mejora la privacidad del usuario. Además, la tecla "A" se utiliza para borrar el último dígito ingresado en caso de error, y la tecla "#" cumple la función de confirmar el ingreso.

Adicionalmente, se utilizó un segundo código basado en el desarrollado en Microchip Studio, pero adaptado para el entorno Arduino IDE, con el fin de facilitar la implementación física del proyecto. Esto se debe a que el entorno Arduino dispone de múltiples librerías ya configuradas, lo que reduce el margen de error durante la carga del programa en el microcontrolador y agiliza el proceso de prueba en el prototipo real.

Finalmente, el sistema fue implementado y probado mediante una simulación en PICSimLab, donde se verificó el correcto funcionamiento del teclado, la comunicación I2C con el módulo PCF8574, el control de los LEDs y del buzzer, así como la lectura y escritura en la memoria EEPROM. Posteriormente, se llevó a cabo el ensamblaje físico del circuito, comprobando en la práctica el comportamiento del sistema y la coherencia entre la simulación y el dispositivo real.

VI. RESULTADOS

En este apartado se ejemplifican y desarrollan los resultados obtenidos en cada parte de la práctica.

Parte A:

El plotter se mueve usando los pines del microcontrolador, cada dirección tiene un pin asignado:

```
#define MOV_ARRIBA PD0
#define MOV_ABAJO PD1
#define MOV_IZQUIERDA PD2
#define MOV_DERECHA PD3
```

Para realizar un paso se activa el pin correspondiente por un breve tiempo:

```
PORTD |= (1 << MOV_ARRIBA); // Activa
movimiento arriba
_delay_ms(50); // Duración
del paso
PORTD &= ~(1 << MOV_ARRIBA); // Apaga el pin
```

A continuación, se presentan las figuras junto con sus características principales

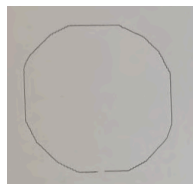


Fig.12 Círculo implementado con el Plotter

En esta figura se observa como se ve implementada la figura geométrica "círculo" empleada en el código.

El dibujo del círculo se realiza calculando las posiciones de cada punto mediante funciones trigonométricas (sin() y cos()) de la librería math.h. Esto permite dividir el círculo en pequeños segmentos y mover el lápiz a lo largo de la trayectoria circular.

```
PORTD |= (1 << BAJA_SOLENOIDE);
_delay_ms(500);
for(int i=1;i<=puntos;i++){
    float angulo=2*M_PI*i/puntos;
    float x=radio*cos(angulo);
    float y=radio*sin(angulo);
    int dx=round(x-x_ant);
    int dy=round(y-y_ant);
    // Mover en X
    if(dx>0) for(int j=0;j<dx;j++){
PORTD|=(1<<MOV_DERECHA); ... }
    else if(dx<0) for(int j=0;j<-dx;j++){
PORTD|=(1<<MOV_IZQUIERDA); ... }
    // Mover en Y
    if(dy>0) for(int j=0;j<dy;j++){
PORTD|=(1<<MOV_ABAJO); ... }
    else if(dy<0) for(int j=0;j<-dy;j++){
PORTD|=(1<<MOV_ARRIBA); ... }
    x_ant=x; y_ant=y;
}
```

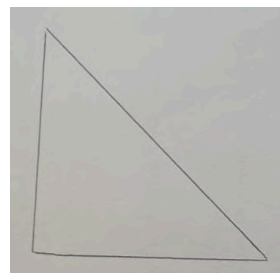


Fig.13 Triángulo implementado con el Plotter

En esta figura se observa como se ve implementada la figura geométrica "Triángulo" empleada en el código.

Para lograr esta figura se implementa un movimiento vertical y horizontal para formar los catetos y posteriormente un movimiento vertical para la hipotenusa. Al finalizar, sube el solenoide. Además, hay un ajuste de posición para no sobrescribir las otras figuras. Un ejemplo de la aplicación del código es:

```
// Hipotenusa
for(int i = 0; i < 100; i++){
    PORTD|= (1<<MOV_ARRIBA)|(1<<MOV_IZQUIERDA);
    _delay_ms(500);
    PORTD&=~((1<<MOV_ARRIBA)|(1<<MOV_IZQUIERDA));
    _delay_ms(100);
}
```

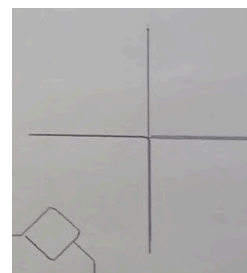


Fig.14 Cruz implementada con el Plotter

En esta figura se observa como se ve implementada la figura “cruz” empleada en el código.

En el caso de esta figura se realizan dos líneas perpendiculares, se dibuja primero la línea vertical y luego la horizontal, regresando al centro

```
// Línea horizontal
for(int i = 0; i < 50; i++){ PORTD |=
(1<<MOV_IZQUIERDA); ... }
for(int i = 0; i < 100; i++){ PORTD |=
(1<<MOV_DERECHA); ... }
for(int i = 0; i < 50; i++){ PORTD |=
(1<<MOV_IZQUIERDA); ... }
```

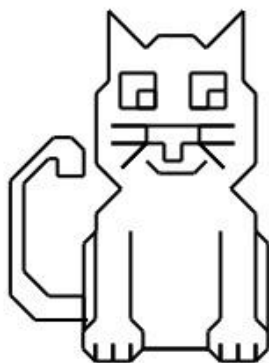


Fig.15 Gato Implementado en Turtle (Python)

En esta figura se aprecia el primer prototipo de la figura “gato” a llevarse a cabo. El código desarrollado para realizar este dibujo se puede encontrar en la plataforma Github, específicamente en la carpeta “Problema A” al nombre de “Código:PythonTurtle_Gato”.



Fig.16 Gato implementado con el Plotter

En esta figura se observa como se ve implementada la figura “gato” empleada en el código. Como se puede observar, se decidió omitir la cola agregada en la figura anterior.

Se dibuja mediante una serie de movimientos específicos para cada parte del cuerpo: cuerpo, patas, boca, nariz y ojos, alternando el solenoide arriba y abajo según sea necesario. Se usa una constante pasos_por_cm para convertir dimensiones físicas a pasos del plotter.

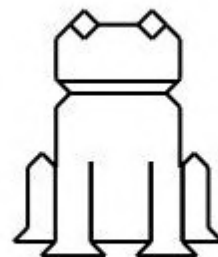


Fig.17 Rana implementado en Turtle (Python)

En este caso se puede observar el prototipo desarrollado en lenguaje python de la rana, esto permitió una visualización del resultado final. Este código se puede encontrar en la plataforma Github en la carpeta “Problema A” como “Código_PythonTurtle_Rana”.

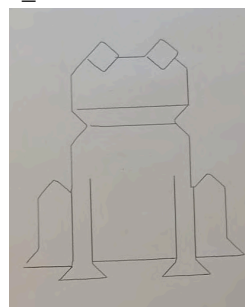


Fig.18 Rana implementada con el Plotter

En esta figura se observa como se ve implementada la figura “rana” empleada en el código, la cuál fue desarrollada en base a la figura anterior.

Esta figura se dibuja siguiendo un patrón de movimientos planificados para formar la silueta de la rana, con el solenoide bajado para marcar el papel durante los trazos principales. Se utiliza la función pasos_por_cm previamente mencionada.

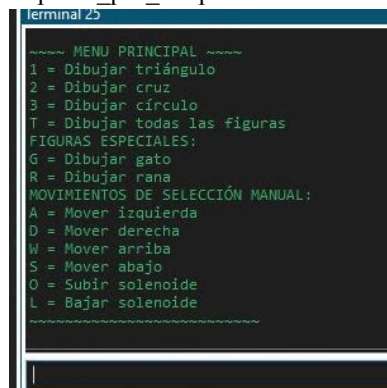


Fig.19 Mensaje UART Plotter

La presente figura ejemplifica el mensaje de inicialización del Plotter. Cómo se visualiza en la imagen, se cuenta con un menú principal que contiene 4 opciones de dibujo (triángulo, cruz, círculo o todas las figuras), además de las opciones de figuras especiales (rana y gato) y los movimientos independientes del plotter.

Parte B:

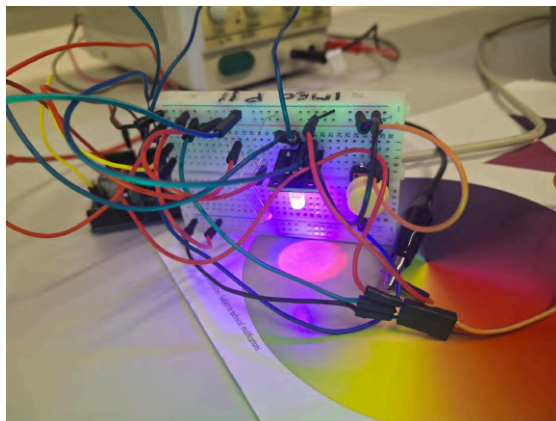


Fig.20 Ensamblaje protoboard seleccionador de colores

En la presente figura se visualiza las conexiones realizadas en la protoboard para el problema del seleccionador de colores, estas conexiones se pueden verificar en la TABLE IV.

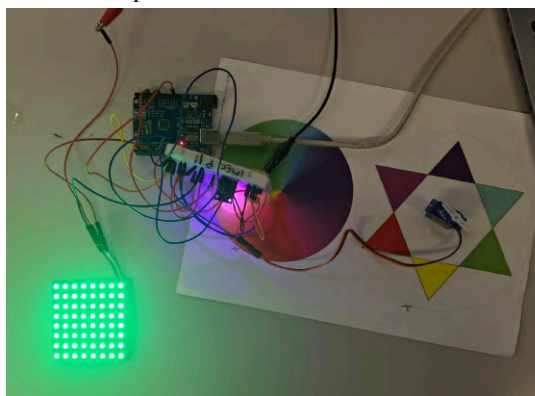


Fig.21 Ensamblaje completo seleccionador de colores

Esta figura refleja el ensamblaje completo del sistema.

TABLE IV

CONEXIONES SELECCIONADOR DE COLORES

Elemento	Pin Microcontrolador
Servomotor	
Vin	PB3
Matriz/Tira NeoPixel (WS2812)	
Vin	PD6
LED RGB ks	
ROJO	PB0
VERDE	PB1
AZUL	PB2
Fotoresistencia	
Sensor LDR	PC0
Pulsador	
Calibración	PD2

Tabla de conexiones para sistema seleccionador de colores y microcontrolador.

Para el funcionamiento del código se implementaron funciones y estructuras claves, a continuación se ejemplifican alguna de ellas.

Comenzando con la definición de la estructura ColorReference, que almacena los valores de referencia RGB, el ángulo correspondiente del servo y el radio máximo de tolerancia. Esto permite comparar los valores previamente calibrados:

```
typedef struct {
    const char* name;
    uint16_t r_ref, g_ref, b_ref;
    uint8_t angle;
    float max_radius;
} ColorReference;
```

Para la lectura de los colores, se encendieron los tres LEDs RGB de forma secuencial, tomando el promedio de la señal analógica de la LDR con la función ADC_Promedio():

```
uint16_t ADC_Promedio(uint8_t
canal_analogico, uint8_t muestras) {
    uint32_t suma = 0;
    for (uint8_t i = 0; i < muestras; i++) {
        suma += analogRead(canal_analogico);
        delay(5);
    }
    return (uint16_t)(suma / muestras);
}
```

La detección del color se realizó calculando la distancia euclidiana entre el vector medido y cada color de referencia, determinando el color más cercano dentro del radio de tolerancia:

```
float distance = sqrt(dr * dr + dg * dg + db
* db);
if (distance < minDist) {
    minDist = distance;
    closestColor = &colorPalette[i];
}
```

El sistema mide los valores RGB, identifica el color y posiciona el servomotor en el ángulo correspondiente, además de mostrar el color detectado en la tira, que se interpreta como NeoPixel:

```
detectedColor =
detectar_color_vectorial(adc_r, adc_g,
adc_b);
setNeoPixelColor(detectedColor->name);
Servo_Move_Delay(detectedColor->angle);
```

El modo de calibración se activa mediante un botón con antirebote que permite actualizar los valores de referencia y el rango de aceptación de cada color de forma estadística (promedio y desviación estándar):


```
max_radius = sqrt(pow(sigma_r * 3.0, 2) +
pow(sigma_g * 3.0, 2) + pow(sigma_b * 3.0,
2));
colorPalette[i].max_radius = max_radius;
```

Finalmente, todo esto se integra y se pueden observar los resultados finales mediante UART:

```
-----
--- MODO CALIBRACION INICIADO ---
Coloque muestra: ROJO
Calibrando: ROJO...
Nuevo vector de ROJO: R=794 G=726 B=708 | Radio Sigma (3.00x): 102.65
LISTO. Coloque muestra: VERDE
Calibrando: VERDE...
Nuevo vector de VERDE: R=735 G=774 B=695 | Radio Sigma (3.00x): 24.82
LISTO. Coloque muestra: AMARILLO
Calibrando: AMARILLO...
Nuevo vector de AMARILLO: R=816 G=813 B=750 | Radio Sigma (3.00x): 13.37
LISTO. Coloque muestra: CIAN
Calibrando: CIAN...
Nuevo vector de CIAN: R=732 G=778 B=765 | Radio Sigma (3.00x): 24.37
LISTO. Coloque muestra: VIOLETA
Calibrando: VIOLETA...
Nuevo vector de VIOLETA: R=642 G=624 B=670 | Radio Sigma (3.00x): 29.42
LISTO. Coloque muestra: MAGENTA
Calibrando: MAGENTA...
Nuevo vector de MAGENTA: R=778 G=664 B=694 | Radio Sigma (3.00x): 38.30
--- MODO CALIBRACION FINALIZADO ---
-----

--- CLASIFICACION EXITOSA ---
1. Valor de lectura (Fotocelda/LDR): R=799, G=704, B=726
2. Color detectado: ROJO
3. Valor de referencia (Vector): R_ref=794, G_ref=726, B_ref=708
4. Diferencia (Distancia Euclidiana): 28.86 (Max Radio: 102.65)
-----

--- CLASIFICACION EXITOSA ---
1. Valor de lectura (Fotocelda/LDR): R=801, G=677, B=667
2. Color detectado: MAGENTA
3. Valor de referencia (Vector): R_ref=778, G_ref=664, B_ref=694
4. Diferencia (Distancia Euclidiana): 37.78 (Max Radio: 38.30)
-----

--- CLASIFICACION EXITOSA ---
1. Valor de lectura (Fotocelda/LDR): R=722, G=773, B=704
2. Color detectado: VERDE
3. Valor de referencia (Vector): R_ref=735, G_ref=774, B_ref=695
4. Diferencia (Distancia Euclidiana): 15.84 (Max Radio: 24.82)
-----
```

Fig.22 Visualización de mensajes UART Seleccionador de Colores

Parte C:

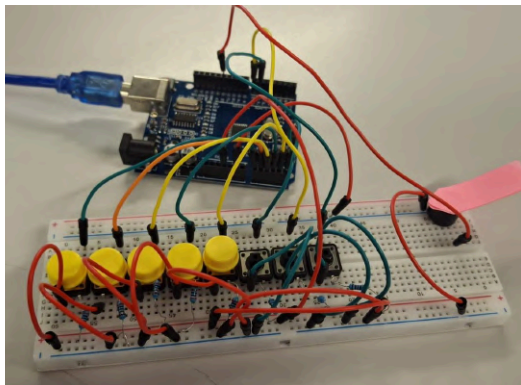


Fig.23 Ensamblaje Piano electrónico

En esta figura se observa el ensamblaje físico del piano electrónico, la conexión exacta se puede verificar en la tabla a continuación:

TABLE V
CONEXIONES PIANO ELECTRÓNICO

Elemento	Pin Microcontrolador
Buzzer	
Buzzer	PB4
Pulsadores	
Do (C4)	PC0
Re (D4)	PC1
Mi (E4)	PC2
Fa (F4)	PC3
Sol (G4)	PC4
La (A4)	PC5
Si (B4)	PD7
Do agudo (C5)	PD6

Tabla de conexiones sistema de piano electrónico y microcontrolador.

Para un correcto funcionamiento del código se comenzó con la definición de las notas y frecuencias correspondientes en Hertz:

```
#define C4 262
#define D4 294
#define E4 330
#define F4 349
#define G4 392
#define A4 440
#define B4 494
#define C5 523
```

Para usar estos tonos se utiliza la función `reproducir_tono`, la cuál se encarga de coordinar la frecuencia y la duración de cada nota: primero calcula el número total de ciclos completos de onda que deben generarse para que el tono suene durante el tiempo musical especificado; luego, un bucle `for` ejecuta esa cantidad de ciclos, alternando entre los estados HIGH y LOW del pin del buzzer, garantizando así la frecuencia y duración correctas. Dentro de este mismo bucle se incluye una verificación periódica de la interfaz UART, que permite detectar si el usuario ha enviado el comando 'S' (Stop), interrumpiendo la melodía inmediatamente y retornando al modo piano libre.

```
uint8_t reproducir_tono(uint16_t freq,
uint16_t dur_ms);
```

Un ejemplo de uso sería:

```
reproducir_tono(C4, CORCHEA);
```

Para que esto funcione, el buzzer debe generar el sonido correspondiente. Esto se logra controlando el buzzer mediante

pulsos con la frecuencia deseada. Cada ciclo de la onda dura 1/freq segundos. En el código se implementa así:

```
void generar_pulso(uint16_t freq) {
    if (freq == 0) return;
    uint32_t periodo_us = 1000000UL /
    (uint32_t)freq;
    uint16_t retardo_pulso_us =
    (uint16_t)(periodo_us / 2);
    PORTB |= (1<<BUZZER);
    retardo_microsegundos(retardo_pulso_us);
    PORTB &= ~(1<<BUZZER);
    retardo_microsegundos(retardo_pulso_us);
}
```

Para utilizar esta función en el código se la llama y se indica la frecuencia a implementar.

Con respecto al modo de piano libre, la lectura de botones se lleva a cabo verificando que botón está presionado y reproduciendo la nota correspondiente:

```
for (i = 0; i < NUM_BOTONES; i++) {
    uint8_t estado = !(PINC & (1 <<
    botones[i]));
    if (estado) {
        generar_pulso(notas_piano[i]);
        break;
    }
}
```

Para la reproducción de canciones se codifican las mismas como secuencias de notas y duraciones. Pudiendo detener la reproducción mediante UART:

```
void cancion1(void){
    if(reproducir_tono(G3, CORCHEA*3/2))
return;
    if(reproducir_tono(C4, CORCHEA)) return;
    if(reproducir_tono(D4, CORCHEA)) return;
    if(reproducir_tono(E4, NEGRA)) return;
}
```

La interacción con el usuario se realiza mediante UART:

```
--- Menú ---
Seleccione una opción:
1 : Tocar cancion 1: 'El reino del revés'
2 : Tocar cancion 2: 'Himno a la alegría'
-----
Modo actual: Piano libre
-> CAMBIO: Cancion 1'

--- Tocando 'El reino del reves - Maria Elena Walsh'. Presione 'S' para detener ---
--- Detenido. Volviendo a Piano (P) ---
-> CAMBIO: Cancion 2'

--- Tocando 'Himno a la alegria- Ludwig van Beethoven'. Presione 'S' para detener ---
--- Detenido. Volviendo a Piano libre---
```

Fig.24 Visualización menú UART piano electrónico Parte D:

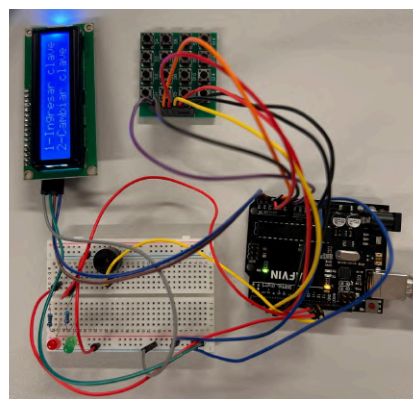


Fig.25 Ensamblaje Cerradura electrónica

En esta imagen se observa el ensamblaje de la cerradura electrónica, las conexiones utilizadas se pueden encontrar en la tabla a continuación.

TABLE VI
CONEXIONES CERRADURA ELECTRÓNICA

Elemento	Pin Microcontrolador
LCD - PCF8574	
SDA	PC4
SCL	PC5
Señales visuales y auditivas	
Rojo	PB4
Verde	PB5
Buzzer	PB6
Teclado	
F1	PC0
F2	PC1
F3	PC2
F4	PC3
C1	PD2
C2	PD3
C3	PD4
C4	PD6

Tabla que refleja las conexiones entre el sistema de cerradura electrónica y el microcontrolador.

Con respecto al teclado matricial 4x4, se define el mapeo de teclas y los pines asociados:

```
char keys[ROWS][COLS] = {
    {'1', '2', '3', 'A'},
```

```

{'4','5','6','B'},
{'7','8','9','C'},
{'*','0','#','D'}
};
uint8_t rowPins[ROWS] = {A0,A1,A2,A3};
uint8_t colPins[COLS] = {2,3,4,6};

```

Para la lectura del teclado, se configura cada fila como salida y cada columna como entrada con pull-up interno. Además, se utiliza una función llamada `keypad_getkey` que escanea las filas y columnas y retorna el carácter correspondiente a la tecla presionada, asegurando un manejo básico del rebote (debounce).

Para la gestión de la clave se utiliza la memoria EEPROM. La lectura de la clave se realiza verificando si existe un valor previamente almacenado; si no se encuentra, se asigna un valor por defecto "1234" y se guarda automáticamente en la EEPROM:

```

void leer_clave_EEPROM(char *clave){
    bool vacia = true;
    for(uint8_t i=0;i<MAX_CLAVE;i++){
        byte b = EEPROM.read(i);
        if(b==0xFF){ clave[i]='\0'; break; }
        clave[i] = b; vacia = false;
    }
    if(vacia){
        strcpy(clave, "1234");
        guardar_clave_EEPROM(clave);
    }
}

void guardar_clave_EEPROM(char *clave){
    for(uint8_t i=0;i<MAX_CLAVE;i++){
        if(i < strlen(clave)) EEPROM.update(i,
        clave[i]);
        else EEPROM.update(i, 0xFF);
    }
}

```

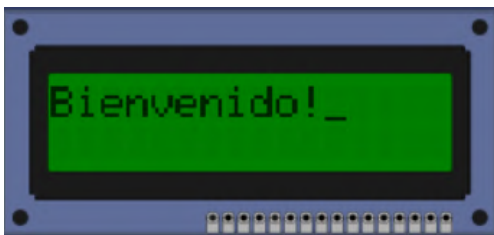


Fig.26 Mensaje de Bienvenida cerradura electrónica

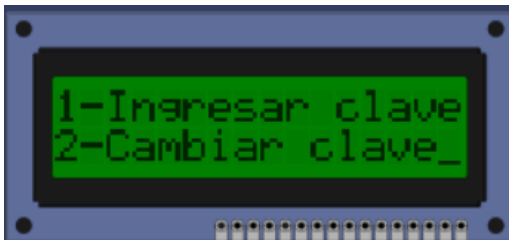


Fig.27 Menú de opciones Cerradura electrónica

Tanto en esta figura como en la anterior, se puede visualizar los mensajes que presenta la pantalla LCD, al inicio da un mensaje de bienvenida y posteriormente despliega las opciones posibles.

Esta comunicación del usuario con el sistema se logra mediante I2C. Para esto se utiliza la librería `LiquidCrystal_I2C`, la cuál maneja la pantalla LCD y muestra menús, instrucciones, asteriscos por cada dígito ingresado, y alertas:

```

LiquidCrystal_I2C lcd(0x27,16,2);
lcd.init();
lcd.backlight();
lcd.setCursor(0,0);
lcd.print("Ingreso clave:");

```

Para el código AVR puro fue necesario crear nuestras propias librerías, las mismas se pueden encontrar en la plataforma Github.

VII. CONCLUSIONES

A lo largo del desarrollo del laboratorio se integraron conocimientos de diseño, programación y control aplicados a distintos sistemas mecatrónicos basados en el microcontrolador ATmega328P. El trabajo tuvo como propósito principal consolidar la relación entre hardware y software en la operación de dispositivos automatizados, abordando de forma práctica la implementación de cuatro bloques que combinan sensado, procesamiento y actuación:

Parte A

La implementación del sistema de control del plotter se realizó de forma exitosa, cumpliendo con la propuesta planteada y demostrando la eficacia de la programación en lenguaje C junto con la comunicación serial para el manejo preciso de los movimientos del dispositivo.

Para su desarrollo se requirió un trabajo metódico y ordenado, donde cada etapa se abordó de manera estructurada. Este enfoque permitió minimizar errores durante la implementación y optimizar el tiempo de depuración, garantizando un funcionamiento fluido y confiable del sistema.

Un aspecto fundamental para alcanzar estos resultados fue el trabajo manual de planificación y conteo de movimientos, realizado previo a la programación. Este proceso permitió anticipar la secuencia de desplazamientos del plotter, ajustar los tiempos de conmutación y definir con precisión las trayectorias necesarias para cada figura. Dicha metodología resultó ser de gran valor práctico y se considera recomendable para futuras aplicaciones similares, ya que facilita la transición entre el diseño conceptual y la programación efectiva del sistema.

Asimismo, la incorporación de movimientos manuales controlados desde el menú enriqueció significativamente la experiencia del usuario. Esta funcionalidad permite no solo evitar el solapamiento de figuras, sino también habilitar la creación de trazos personalizados. De esta forma, el usuario puede generar sus propios dibujos de manera interactiva, controlando incluso la bajada y subida del solenoide para definir cuando el plotter debe trazar o no.

En conjunto, este problema demostró la importancia de la planificación, la precisión en el control de los actuadores y la

correcta integración entre hardware y software. La experiencia obtenida refuerza la necesidad de un enfoque meticuloso en el desarrollo de sistemas y sienta una base sólida para futuras implementaciones más complejas de automatización y control.

Parte B

El desarrollo del sistema de selección de colores se llevó a cabo exitosamente, cumpliendo con las funcionalidades planteadas en la consigna. El sistema logró detectar colores a partir de la intensidad lumínica captada por la fotocelda, representar los valores identificados en la tira de LEDs y posicionar el servomotor en el ángulo correspondiente al color, integrando adecuadamente los módulos de sensado, procesamiento y actuación.

A pesar del correcto funcionamiento general, se observó una cierta imprecisión en la detección de colores, principalmente asociada a las variaciones de iluminación ambiental al momento de medir. Debido a esto, se destaca la necesidad de calibrar el sistema de cada uso, ya que los valores de referencia dependen directamente de las condiciones lumínicas del entorno. Esta calibración resulta esencial para asegurar la correcta correspondencia entre el color detectado y el color mostrado, garantizando así la confiabilidad del sistema.

Durante la etapa de implementación física, el montaje en protoboard presentó limitaciones estructurales y de robustez, generando cierta inestabilidad en la medición debido a la cantidad de cables y conexiones intermedias. Esto dificultó la manipulación del circuito y afectó la movilidad al realizar mediciones.

En cuanto al desarrollo del software, la simulación inicial se realizó utilizando AVR puro, lo que permitió comprobar el funcionamiento lógico del código. Sin embargo, al llevar a cabo la implementación física, fue necesario recurrir al pseudocódigo de Arduino debido a incompatibilidades con las librerías AVR, las cuales no pudieron aplicarse directamente en el entorno real. Este hecho pone en evidencia la necesidad de mejorar y optimizar las librerías del entorno AVR, para facilitar la portabilidad entre simulación y práctica sin perder funcionalidades.

Como posible mejora futura, se propone la incorporación de un sensor de color (como el TCS3200) en lugar de una LDR. Esto permitiría una mayor precisión y estabilidad en la lectura cromática, eliminando la dependencia de la iluminación ambiental, así como la calibración repetitiva del sistema, aumentando la confiabilidad del sistema.

Parte C

La realización del piano eléctrico se ejecutó de manera exitosa, cumpliendo plenamente con los objetivos planteados. El sistema logró reproducir correctamente las ocho notas musicales asignadas mediante los pulsadores, generando tonos claros y estables a través del buzzer. Además, la implementación de la comunicación serial permitió seleccionar y reproducir canciones predefinidas, demostrando la versatilidad del sistema y la correcta integración entre las funciones manuales y automáticas.

Durante la práctica, se comprobó la precisión del control de frecuencia por PWM, así como la adecuada respuesta del buzzer ante las variaciones de señal. El sistema mostró un rendimiento confiable, sin errores en la ejecución de notas ni interrupciones en la comunicación, lo que evidencia un

correcto manejo de temporizadores y sincronización de procesos dentro del microcontrolador.

Más allá de su simplicidad aparente, el problema permitió consolidar conceptos fundamentales de programación a bajo nivel, tales como el control de periféricos digitales, el uso de interrupciones y la transmisión serial de datos.

Parte D

La implementación de la cerradura electrónica se completó con éxito, cumpliendo con todas las funcionalidades previstas. El sistema permite el ingreso, validación y modificación de contraseña mediante un teclado matricial, mostrando mensajes informativos en la pantalla LCD y utilizando indicadores visuales y acústicos para señalar los diferentes estados del proceso. Además, se implementó el almacenamiento persistente de la contraseña en la memoria EEPROM, asegurando la conservación de los datos tras apagados o reinicios del sistema.

Durante la etapa de simulación, el problema fue programado utilizando AVR puro, lo que permitió validar el funcionamiento lógico y la interacción entre los distintos módulos del sistema. Sin embargo, al pasar a la implementación física, fue necesario, de nuevo, realizar una adaptación a pseudocódigo de Arduino, debido a limitaciones en las librerías utilizadas del entorno AVR, que impidieron la correcta ejecución del programa en el hardware real. Esta situación evidenció la importancia de mejorar la compatibilidad y robustez de las librerías AVR, a fin de facilitar la transición entre simulación y práctica sin necesidad de modificar la integridad del código.

En resumen, el laboratorio permitió aplicar de forma integrada los conocimientos sobre el microcontrolador, combinando simulación, programación y control de sistemas. A través de los cuatro problemas desarrollados se comprobó la importancia de un trabajo planificado y metódico, la correcta calibración de los sistemas y la adaptación entre entornos de simulación e implementación física. En conjunto, la experiencia reforzó la comprensión del vínculo entre hardware y software, consolidando habilidades esenciales para el diseño y desarrollo de soluciones automatizadas.

VIII. APÉNDICE

Apéndice I. Diagrama de máquina de estados Parte C: Piano.

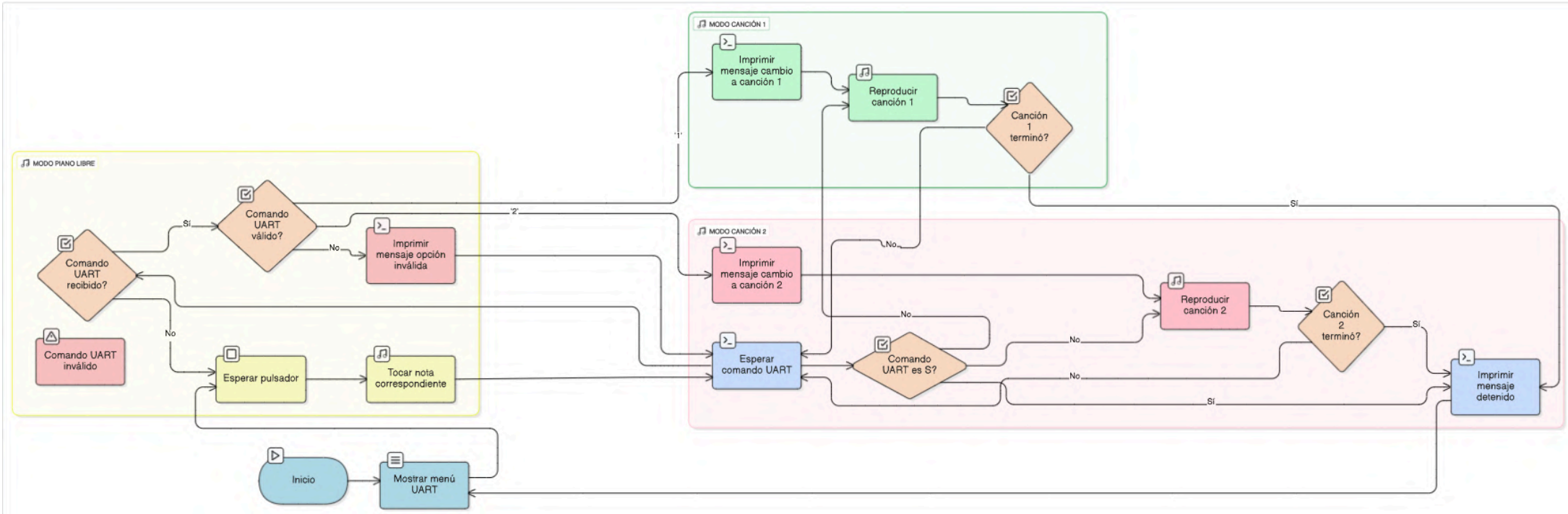


Fig.28 Diagrama de máquina de estados Parte C: Piano.

Apéndice II.Código para diagrama de máquina de estados Parte C: Piano.

```
title Flujo de control: Piano y reproducción
de canciones por UART
direction right
```

```
// Grupos y nodos
```

```
Inicio [shape: oval, color: lightblue, icon:
play]
```

```
Mostrar menú UART [color: lightblue, icon:
menu]
```

```
Modo Piano Libre [color: yellow, icon:
music] {
  Esperar pulsador [icon: square, color:
yellow]
  Tocar nota correspondiente [icon: music,
color: yellow]
  Esperar comando UART [icon: terminal,
color: blue]
  Comando UART recibido? [shape: diamond,
color: orange, icon: check-square]
  Comando UART válido? [shape: diamond,
color: orange, icon: check-square]
  Comando UART inválido [icon:
alert-triangle, color: red]
  Imprimir mensaje opción inválida [icon:
terminal, color: red]
}
```

```
Modo Canción 1 [color: green, icon: music] {
  Imprimir mensaje cambio a canción 1 [icon:
terminal, color: green]
  Reproducir canción 1 [icon: music, color:
green]
  Canción 1 terminó? [shape: diamond, color:
orange, icon: check-square]
  Esperar comando UART [icon: terminal,
color: blue]
  Comando UART es S? [shape: diamond, color:
orange, icon: check-square]
  Imprimir mensaje detenido [icon: terminal,
color: blue]
}
```

```
Modo Canción 2 [color: pink, icon: music] {
  Imprimir mensaje cambio a canción 2 [icon:
terminal, color: pink]
  Reproducir canción 2 [icon: music, color:
pink]
  Canción 2 terminó? [shape: diamond, color:
```

```
orange, icon: check-square]
```

```
  Esperar comando UART [icon: terminal,
color: blue]
```

```
  Comando UART es S? [shape: diamond, color:
orange, icon: check-square]
```

```
  Imprimir mensaje detenido [icon: terminal,
color: blue]
}
```

```
// Relaciones principales
```

```
Inicio > Mostrar menú UART
```

```
Mostrar menú UART > Esperar pulsador
```

```
// Bucle principal de Modo Piano Libre
```

```
Esperar pulsador > Tocar nota
```

```
correspondiente
```

```
Tocar nota correspondiente > Esperar comando
UART
```

```
Comando UART recibido? > Comando UART
válido?: Sí
```

```
Comando UART recibido? > Esperar pulsador:
No
```

```
// Validación de comando UART en Modo Piano
Libre
```

```
Comando UART válido? > Imprimir mensaje
opción inválida: No
```

```
Imprimir mensaje opción inválida > Esperar
comando UART
```

```
Comando UART válido? > Imprimir mensaje
cambio a canción 1: '1'
```

```
Comando UART válido? > Imprimir mensaje
cambio a canción 2: '2'
```

```
// Transición a Modo Canción 1
```

```
Imprimir mensaje cambio a canción 1 >
```

```
Reproducir canción 1
```

```
Reproducir canción 1 > Canción 1 terminó?
```

```
Esperar comando UART > Comando UART es S?
```

```
Comando UART es S? > Imprimir mensaje
detenido: Sí
```

```
Comando UART es S? > Reproducir canción 1:
No
```

```
// Transición a Modo Canción 2
```

```
Imprimir mensaje cambio a canción 2 >
```

```
Reproducir canción 2
```

```
Reproducir canción 2 > Canción 2 terminó?
```

```
Canción 2 terminó? > Imprimir mensaje
detenido: Sí
```

```
Esperar comando UART > Comando UART es S?
```

Comando UART es S? > Imprimir mensaje
 detenido: Sí
 Comando UART es S? > Reproducir canción 2:
 No
 Comando UART recibido? < Esperar comando
 UART
 Esperar comando UART < Canción 1 terminó?:
 No
 Imprimir mensaje detenido < Canción 1
 terminó?: Sí
 Mostrar menú UART < Imprimir mensaje
 detenido
 Esperar comando UART < Canción 2 terminó?:
 No

Apéndice III: Partitura canción 1: “El reino del revés” de Maria
 Elena Walsh.

El reino del revés

maria elene walsh

me di je ron que en rei no del re vés na dael pá ja rón vuc lael pez

que los ga tos no ha cen miau y di cen yes porques tu dián mu choin gles

Va mos a ver co moes el rei no del re vés va mos a ver comoes

el rei no del re vés

Fig.29 Partitura “El reino del revés” de Maria Elena Walsh.

Apéndice IV: Partitura canción 2: “Himno a la alegría” de
 Ludwig van Beethoven

HIMNO A LA ALEGRÍA
L. V. Beethoven

Ritmo:
 Tono:
 Tiempo:

♩ = 97
 Balada

mi mi fa sol sol fa mi re do do re mi mi re re

mi mi fa sol sol fa mi re do do re mi re do do

re mi do re mi fa mi do re mi fa mi re do re sol

mi mi fa sol sol fa mi re do do re mi re do do

Fig.30 Partitura “Himno a la alegría” de Ludwig van
 Beethoven.

IX. EVIDENCIAS

S. Modernell, V. Etchevery, y G. Sosa,
[<https://github.com/Victoriaetch/Microcontroladores/tree/main/Laboratorios/Laboratorio%202%20>]. [Accedido: Octubre, 2025]

REFERENCIAS

[1] Arduino, "Arduino Uno Rev3 – Reliable ATmega328P Board with Digital & Analog I/O," *Arduino Store*. [Online]. Available: <https://store-usa.arduino.cc/products/arduino-uno-rev3>. [Accessed: Sep. 26, 2025].

[2] Wikipedia, "Atmega328," *Wikipedia*, 20-Mar-2024. [Online]. Available: <https://es.wikipedia.org/w/index.php?title=Atmega328&oldid=158934266>. [Accessed: Sep. 20, 2025].

[3] "C (lenguaje de programación)," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: [https://es.wikipedia.org/wiki/C_\(lenguaje_de_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/C_(lenguaje_de_programaci%C3%B3n)).

[4] Cadence PCB Solutions, "USART vs. UART: What's the Difference?," *Cadence PCB Design & Analysis*, 8 de febrero de 2023. [Enlace](#). [Accedido: 26-sep-2025].

[5] R. J. H., "AVR UART Tutorial," RJH Coding. Available: <http://www.rjhcoding.com/avr-uart.php>. [Accessed: 10-oct-2025].

[6] Python and Turtle, "Python Turtle - Online." Available: <https://pythonandturtle.com/turtle/>. [Accessed: 16-oct-2025].

[7] "Fotorresistor," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Fotorresistor>.

[8] "Getting Started With NeoPixel WS2812 RGB LED," *Instructables*. [En línea]. Disponible: <https://www.instructables.com/Getting-Started-With-NeoPixel-WS2812-RGB-LED/>.

[9] "Servomotor," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Servomotor>.

[10] "Mixing colors in Arduino using the KS and KY- RGB LED module," *Arduinointro.com*. [En línea]. Disponible: <https://arduinointro.com/articles/projects/mixing-colors-in-arduino-using-the-ks-and-ky-rgb-led-module>.

[11] Visual Paradigm, "What is State Machine Diagram?," *Visual Paradigm*. [Online]. Available: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-state-machine-diagram/>. [Accessed: Sep. 26, 2025].

[12] "Zumbador," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Zumbador>.

[13] "Descubre las diferencias entre zumbador activo y pasivo y aprende a crear tu propio theremin," *Complubot*. [En línea]. Disponible: <https://complubot.com/descubre-las-diferencias-entre-zumbador-activo-y-pasivo-y-aprende-a-crear-tu-propio-theremin/>.

[14] "Arduino 1602 Character LCD Tutorial," *Last Minute Engineers*. [En línea]. Disponible: <https://lastminuteengineers.com/arduino-1602-character-lcd-tutorial/>.

[15] "Teclados matriciales," *El octavo bit*. [En línea]. Disponible: <https://eloctavobit.com/modulos-sensores/teclados-matriciales>.

[16] "EEPROM," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/EEPROM>.

[17] "I²C," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/I%C2%B2C>.

Laboratorio 3: GPIO en C y Python

Sofía Modernell, Victoria Etcheverry, Guadalupe Sosa,
Carrera: Ingeniería en Mecatrónica, Universidad Tecnológica (UTEC)
Fray Bentos, Uruguay

safia.modernell@estudiantes.utec.edu.uy, victoria.etccheverry@estudiantes.utec.edu.uy, guadalupe.sosa@estudiantes.utec.edu.uy
7 de noviembre de 2025

Resumen—Este informe detalla el diseño, simulación e integración de cinco sistemas basados en el ATmega328P. Utilizando programación en C y una validación dual (simulación en Proteus/PicSimLab e implementación física), se desarrollaron: (1) un plotter (motores paso a paso); (2) un control de temperatura (LM35, UART); (3) un control de posición de motor DC (PWM, potenciómetros); (4) una matriz RGB (joystick); y (5) una cerradura RFID (RC522, EEPROM, I2C LCD). Se usaron scripts de Python para la visualización en tiempo real, validando los algoritmos de control y la integración de protocolos (UART, I2C, SPI).

Keywords: ATmega328P, Python, C, PWM, Plotter, TCLAB, LM35, Potenciómetro, WS2812, RFID-RC522, LCD, RFID-RC522, EEPROM, I2C, SPI, UART.

Abstract—This report details the design, simulation, and integration of five automation systems based on the ATmega328P. Using C programming and dual validation (simulation in Proteus/PicSimLab and physical implementation), the following were developed: (1) a plotter (stepper motors); (2) a temperature control system (LM35, UART); (3) a DC motor position control (PWM, potentiometers); (4) an RGB matrix (joystick); and (5) an RFID lock (RC522, EEPROM, I2C LCD). Python scripts were used for real-time visualization, validating the control algorithms and the integration of protocols (UART, I2C, SPI).

I. INTRODUCCIÓN

El objetivo fundamental de este laboratorio fue aplicar los principios de la ingeniería de control y la programación de sistemas embebidos en un entorno práctico. El trabajo se centró en el microcontrolador ATmega328P, pero con el desafío de migrar de las abstracciones de alto nivel (como las de la plataforma Arduino) a la programación directa en lenguaje C para la gestión de registros y periféricos.

Para consolidar estos conceptos, el laboratorio se estructuró en torno a la resolución de cinco problemas de automatización. Estos ejercicios cubren un espectro que va desde el control de movimiento de precisión (cinemática de 2 ejes) y el control de procesos en lazo cerrado (sistemas térmicos), hasta el diseño de interfaces hombre-máquina (HMI) y la implementación de sistemas de identificación y almacenamiento persistente de datos.

Este informe documenta el proceso de diseño, la lógica de control, la simulación (en Proteus y PicSimLab) y la implementación física de cada uno de estos sistemas. El énfasis recae en la integración de múltiples protocolos (UART, I2C, SPI) y la validación de los algoritmos en hardware real, demostrando la capacidad del ATmega328P para gestionar tareas de control complejas.

II. OBJETIVOS

Objetivo general

Diseñar, simular e integrar soluciones basadas en el microcontrolador ATmega328P, aplicando técnicas de control, sensado, comunicación y procesamiento digital para la resolución de problemas prácticos de automatización y control electrónico.

Objetivos específicos:

- Programar el control de un Plotter, capaz de ejecutar movimientos coordinados en los ejes X e Y mediante motores paso a paso.
- Desarrollar un sistema de control de temperatura inteligente, que mida y regule la temperatura mediante sensores LM35 y actuadores, con monitoreo y ajuste dinámico a través de comunicación UART.
- Construir un sistema de control por PWM que ajuste la posición de un motor DC para igualar el valor de un potenciómetro de referencia, visualizando la respuesta y el sentido de giro en simulación y práctica real.
- Programar una matriz RGB controlada por joystick analógico, permitiendo el desplazamiento del LED activo y el cambio aleatorio de color al presionar el botón.
- Diseñar e implementar una cerradura electrónica con tecnología RFID, capaz de registrar, verificar y actualizar credenciales de acceso, mostrando mensajes tanto en una LCD I²C como por UART, almacenando información en la memoria EEPROM.

III. MATERIALES UTILIZADOS

TABLE I

MATERIALES UTILIZADOS

MATERIALES	CANTIDAD
Arduino Uno Rev3	1
Protoboard	1
Fuente de DC	1
Cables	-
Pen Plotter	1
TCLAB	1
Ventilador DC 24V 3110KL-05W-B69	1
L298N	1
Potenciómetro	2
Motor DC	1
Joystick	1
Matriz <i>WS2812</i>	1
Resistencia 220	1
Pulsadores	2
Led verde	1
Led roja	1
Resistencia 1k	2
RFID-RC522	1
LCD 16x2	1
IO PCF8564T	1
Microchip Studio	-
PyCharm	-
Proteus	-
PicSimLab	-

IV. MARCO TEÓRICO

Arduino UNO Rev3

El Arduino Uno Rev3 es una placa de desarrollo ampliamente utilizada en el ámbito educativo y en el prototipado de sistemas electrónicos, principalmente por su facilidad de uso, bajo costo y la gran comunidad que respalda su ecosistema. Esta placa se basa en el microcontrolador ATmega328P. [1]

El dispositivo funciona con una tensión nominal de 5 V regulados y puede ser alimentado tanto a través del puerto USB como mediante una fuente externa, recomendándose un rango de voltaje entre 7 V y 12 V para asegurar un rendimiento estable. Esta flexibilidad en la alimentación facilita su integración en diferentes entornos y aplicaciones. [1]

En cuanto a su arquitectura de pines, el Arduino Uno dispone de 14 pines digitales de entrada/salida, de los cuales 6 pueden ser utilizados como salidas PWM, además de 6 entradas analógicas identificadas como A0 a A5. La placa incluye también pines de referencia y alimentación, como VIN, AREF, conexiones a tierra, así como salidas de 3.3 V y 5 V para alimentar periféricos externos. Asimismo, incorpora interfaces de comunicación que permiten su interacción con otros dispositivos electrónicos: UART (pines 0 y 1), SPI (pines 10 a 13) e I²C (pines A4 y A5). Esta diversidad de opciones convierte al Arduino Uno en una plataforma versátil para la interconexión de sensores, actuadores y módulos de expansión. [1]

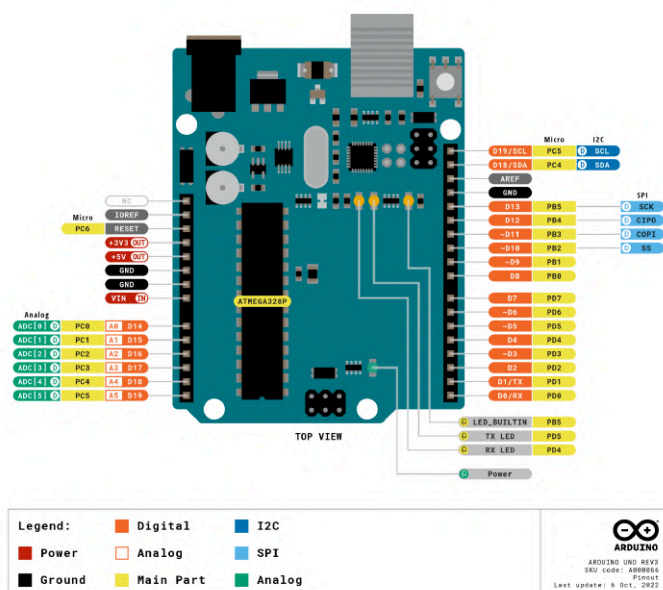


Fig.1 Arduino Pinout

ATmega328p

El ATmega328P es un microcontrolador de 8 bits de la serie megaAVR desarrollado por Atmel, ampliamente utilizado en sistemas embebidos y proyectos de electrónica de bajo consumo y bajo costo. Su arquitectura se basa en un microcontrolador RISC, lo que le permite ejecutar instrucciones en un solo ciclo de reloj, alcanzando hasta 1 MIPS, optimizando la velocidad de procesamiento y el consumo energético. [2]

Entre sus principales características se destacan:

- Memoria: 32 KB de memoria flash (ISP) con capacidad de lectura mientras se escribe, 1 KB de memoria EEPROM y 2 KB de SRAM.
- Entradas/Salidas: Hasta 23 líneas de E/S de propósito general, 32 registros de propósito general y 6 canales PWM.
- Convertidor A/D: 6 canales de 10 bits, que permiten la adquisición de señales analógicas de los sensores.
- Temporizadores: 3 temporizadores flexibles con modo de comparación y un temporizador “watchdog” programable.
- Interfaz de comunicación: Soporta USART, SPI y comunicación serie de 2 hilos.
- Modos de ahorro de energía: Cinco modos configurables por software, que optimizan la autonomía del dispositivo en aplicaciones portátiles.
- Rango de operación: Funciona entre 1,8 y 5,5 V, lo que lo hace compatible con distintos sistemas de alimentación.

El ATmega328P se utiliza en proyectos que requieren control preciso de periféricos, adquisición de datos y transmisión de información, siendo la base de plataformas como Arduino Uno y Nano. [2]

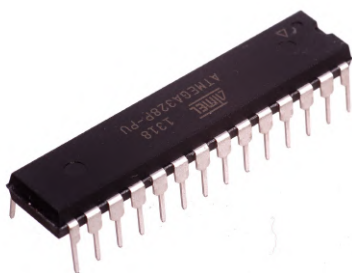


Fig.2 Microcontrolador ATmega328P.

Lenguaje C

C es un lenguaje de programación de propósito general desarrollado por Dennis Ritchie en los Laboratorios Bell entre 1969 y 1972, orientado originalmente a la implementación de sistemas operativos como Unix. Es valorado por la eficiencia del código que genera y se considera un lenguaje de nivel medio, pues combina estructuras de alto nivel con un control de bajo nivel, como el acceso directo a la memoria mediante punteros. El lenguaje posee un núcleo simple que se complementa con bibliotecas y utiliza un preprocesador para tareas como la inclusión de archivos. Carece de recolección de basura nativa, por lo que el programador es responsable de gestionar la memoria explícitamente. [3]

PYTHON

Python es un lenguaje de programación interpretado, de alto nivel y propósito general, diseñado para ser altamente legible y permitir un desarrollo rápido [4]. Si bien es un lenguaje de propósito general, su poder para este proyecto (recibir y graficar datos de un Arduino) no proviene de funciones nativas para el manejo de hardware, sino de su filosofía de extensibilidad y su vasto ecosistema de bibliotecas de terceros.

Conceptualmente, Python funciona como el glue code que une el hardware (el ATmega328p enviando datos por el puerto COM) y la visualización de alto nivel (el gráfico en la PC). Para la adquisición de datos, se recurre a bibliotecas especializadas como pySerial, que abstrae la complejidad de la comunicación serial (configuración de baudios, buffers, timeouts) en comandos simples y legibles. Una vez que los datos son recibidos, la fortaleza de Python en el manejo de datos (parsing de strings, conversión de tipos) entra en juego.

Finalmente, para la visualización en tiempo real, se utiliza el robusto ecosistema científico de Python, específicamente bibliotecas como Matplotlib y NumPy. Matplotlib proporciona la capacidad de crear gráficos dinámicos que se actualizan en bucle, mientras que NumPy ofrece estructuras de datos eficientes (arrays) para almacenar y manipular los flujos de datos numéricos. Por lo tanto, Python es la plataforma ideal que integra la comunicación de bajo nivel con el análisis y la visualización de alto nivel de forma eficiente. [4]

UART

El protocolo UART (Universal Asynchronous Receiver Transmitter) es fundamental en sistemas embebidos para la transmisión de datos entre dispositivos. [5]

El UART es un protocolo de comunicación serial asíncrona que utiliza dos líneas: una para la transmisión (TX) y otra para la recepción (RX). Su principal característica es que no requiere una señal de reloj externa, ya que la sincronización entre el transmisor y el receptor se establece mediante la

configuración previa de parámetros como la velocidad de transmisión (baud rate), bits de datos, paridad y bits de parada. Esta simplicidad lo hace ideal para comunicaciones de corto alcance y baja velocidad, como la conexión entre microcontroladores y módulos periféricos. [5]

I2C

I²C, sigla de Inter-Integrated Circuit, es un bus de datos serie desarrollado por Philips en 1982, diseñado para la comunicación interna entre diferentes circuitos integrados dentro de un mismo equipo, como un microcontrolador y sus periféricos. Funciona bajo un protocolo maestro-esclavo, donde el dispositivo maestro es el que inicia la transferencia de datos. Este bus también es conocido como TWI (Two-Wire Interface), una designación introducida por Atmel que es técnicamente idéntica. [8]

El bus I²C solo requiere dos líneas de señal: una para los datos (SDA) y otra para el reloj (SCL). Ambas líneas son de drenador abierto y requieren resistencias de pull-up. Esta configuración permite que múltiples dispositivos se conecten al mismo bus. La comunicación se inicia con una condición de arranque y los datos se transfieren en paquetes de 8 bits (octetos), donde cada paquete es seguido por un bit de confirmación (ACK) del receptor. El direccionamiento estándar utiliza 7 bits, lo que permite hasta 112 nodos en el bus, aunque se introdujo un modo de 10 bits para ampliar esta capacidad. La principal ventaja de I²C es su capacidad para controlar una red de dispositivos usando solo dos pines de E/S, lo que reduce la complejidad y el costo, siendo comúnmente usado para sensores, memorias EEPROM y relojes en tiempo real. [8]

SPI

El Serial Peripheral Interface (SPI) es un estándar de comunicación fundamental en el diseño de sistemas embebidos, que funciona como un "lenguaje" de alta velocidad para que los circuitos integrados (chips) hablen entre sí a corta distancia. Conceptualmente, SPI opera bajo una arquitectura de maestro-esclavo. En un sistema con Arduino, el microcontrolador (ATmega328p) actúa como el maestro, orquestando toda la comunicación, mientras que los periféricos (como sensores, pantallas LCD, o módulos de tarjeta SD) actúan como esclavos. [9]

Su funcionamiento se basa en un protocolo síncrono, lo que significa que todas las transferencias de datos están sincronizadas por una señal de reloj (SCLK) generada por el maestro. Esto elimina la necesidad de que ambos dispositivos tengan relojes internos perfectamente calibrados, permitiendo velocidades de transferencia muy altas. La comunicación requiere cuatro líneas principales: SCLK (el reloj o "ritmo"), MOSI (Master Out Slave In, por donde el maestro habla al esclavo), MISO (Master In Slave Out, por donde el maestro escucha al esclavo) y SS (Slave Select), que es la línea que el maestro utiliza para "activar" al esclavo específico con el que desea comunicarse. [9]

La característica clave de SPI es que es full-duplex: gracias a que las líneas de envío (MOSI) y recepción (MISO) son independientes, el maestro puede enviar datos (ej. "enviar siguiente comando") y recibir datos (ej. "resultado de la medición anterior") simultáneamente, dentro del mismo ciclo de reloj. Esto lo hace extremadamente eficiente para aplicaciones que requieren un flujo de datos constante y rápido entre el microcontrolador y sus periféricos, mucho más rápido

que otros protocolos como I²C o la comunicación serial (UART) que se usa para hablar con la PC. [9]

EEPROM

EEPROM son las siglas de Electrically Erasable Programmable Read-Only Memory, que se traduce como memoria de solo lectura programable y borrrable eléctricamente. Es un tipo de memoria ROM no volátil, lo que significa que retiene los datos incluso sin alimentación eléctrica. A diferencia de las EPROM, que deben borrarse exponiéndolas a rayos ultravioleta, las EEPROM pueden programarse, borrarse y reprogramarse utilizando únicamente impulsos eléctricos. [10]

Las celdas de memoria de una EEPROM están constituidas por transistores MOS con una compuerta flotante. Aunque estos dispositivos pueden leerse un número ilimitado de veces, el número de ciclos de borrado y escritura es limitado, situándose habitualmente entre cien mil y un millón de ciclos. Para la comunicación, las EEPROM suelen emplear protocolos estándar como I²C o SPI, o bien se integran directamente en chips como microcontroladores para lograr una mayor rapidez de acceso. La memoria flash es considerada una forma avanzada de EEPROM. [10]

PWM

La Modulación por Ancho de Pulsos (PWM) es una técnica fundamental utilizada en microcontroladores, como el Arduino, para "simular" una señal de salida analógica utilizando únicamente una salida digital. Los microcontroladores en su mayoría solo pueden entregar dos niveles de voltaje (por ejemplo, 0V para "apagado" y 5V para "encendido"). El PWM permite generar un voltaje promedio variable entre esos dos extremos, lo cual es esencial para controlar la intensidad de un LED, la velocidad de un motor o la posición de un servomotor. [11]

El concepto se basa en variar el ciclo de trabajo (duty cycle) de una señal digital periódica (un pulso cuadrado) sin cambiar su frecuencia. El ciclo de trabajo es el porcentaje de tiempo que la señal se mantiene en estado "encendido" (nivel alto) dentro de un período fijo. Por ejemplo, si un pin PWM tiene un ciclo de trabajo del 50%, la señal está la mitad del tiempo en 5V y la otra mitad en 0V, resultando en un voltaje promedio de 2.5V. Si el ciclo de trabajo es del 20%, el voltaje promedio será de 1V (20% de 5V). Al variar rápidamente este ancho del pulso (manteniéndolo "encendido" más o menos tiempo) se controla la cantidad total de energía entregada a la carga, logrando un control analógico efectivo desde una fuente digital. [11]

Pen Plotter

Un pen plotter es un dispositivo automatizado diseñado para dibujar imágenes o diagramas sobre una superficie plana utilizando un bolígrafo, marcador u otro instrumento de escritura. A diferencia de las impresoras convencionales, que aplican tinta mediante procesos de impresión, las pen plotters mueven el instrumento de manera controlada sobre los ejes X e Y, permitiendo reproducir con precisión figuras vectoriales.

En su funcionamiento típico, la pen plotter utiliza motores paso a paso para controlar el movimiento horizontal y vertical del bolígrafo, mientras que un mecanismo adicional permite levantar y bajar el instrumento de escritura según sea necesario. La máquina puede ser controlada mediante comandos de control de movimiento, como los que se encuentran en el G-code, lo que permite programar trayectorias complejas y repetir dibujos de manera consistente.

Este tipo de dispositivos se emplea tanto en proyectos educativos como en aplicaciones de prototipado rápido y automatización, ofreciendo una plataforma práctica para comprender conceptos de control de movimiento, sincronización de ejes y coordinación mecánica, sin necesidad de equipos industriales complejos.

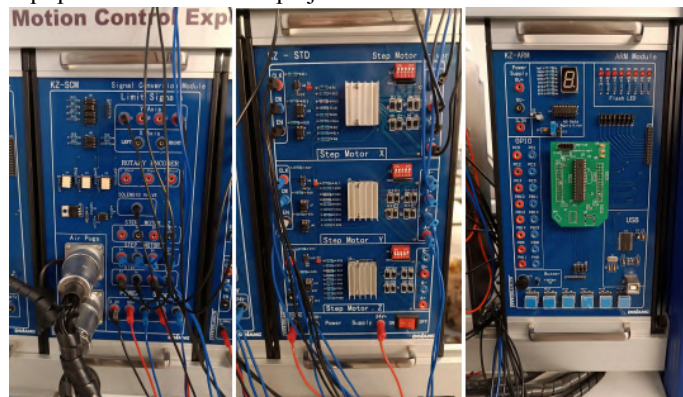


Fig.3 Pen Plotter: Tablero de control

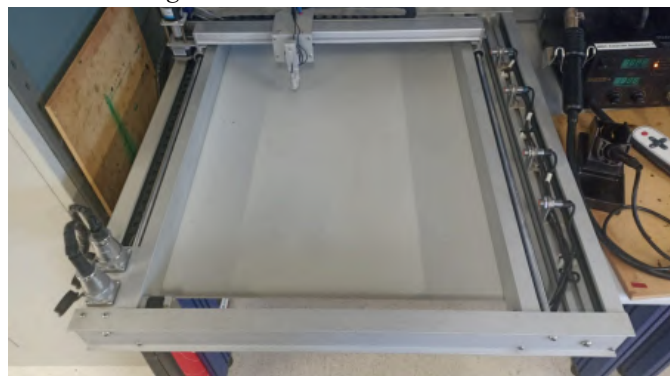


Fig.4 Pen Plotter

TABLE II
CONEXIONES PEN PLOTTER

MÓDULO AVR	ATMEGA382P	ETIQUETA
PC0	PB3	CLK_X
PC2	PB4	DIR_X
PC3	PB5	EN_X
PC4	PC0	SOLENOID
PB10	PC3	CLK_Y
PB12	PC4	DIR_Y
PB14	PC5	EN_Y
PA8	PD2	LIMIT_YA
PA11	PD3	LIMIT_YD
PB6	PD5	LED

TCLAB

Conceptualmente, este hardware es un sistema térmico simple basado en un Arduino. La placa está equipada con componentes específicos para crear un problema de control: dos sensores de temperatura y dos calentadores (actuadores), además de un LED. El diseño físico permite que la energía térmica de los calentadores se transfiera a los sensores de temperatura mediante los fenómenos físicos de conducción, convección y radiación. Al mismo tiempo, el calor se disipa al entorno, creando una dinámica de proceso que debe ser controlada. [12]

La función de esta placa es cerrar la brecha entre la teoría puramente simulada (en Python o MATLAB) y la implementación en el mundo real. En lugar de simular una respuesta, el Arduino en el TCLab mide una temperatura real (la variable de proceso) y aplica una potencia real a los calentadores (la variable manipulada). Esto permite a los estudiantes realizar pruebas de escalón (step tests) para identificar modelos dinámicos (como un FOPDT, Modelo de Primera Orden más Tiempo Muerto) y luego diseñar y sintonizar controladores PID en un hardware tangible, enfrentándose a las complejidades del ruido del sensor y las perturbaciones térmicas del entorno. [12]



Fig.5 TCLAB

VENTILADOR DC 24V 3110KL-05W-B69

El componente 3110KL-05W-B69-B00 es un ventilador axial de corriente continua (DC). En el contexto de un sistema de control, este dispositivo no es un sensor, sino un actuador destinado a la gestión térmica activa. Su función conceptual es implementar la refrigeración por convección forzada. Mientras que la convección natural disipa el calor de forma pasiva al aire circundante, un ventilador manipula activamente el entorno forzando un flujo de aire de mayor volumen sobre una superficie caliente (como un disipador de calor o los calentadores del TCLab). [13]

Este modelo opera a 24 VDC y utiliza rodamientos de bolas (Ball Bearings), una característica clave que lo diferencia de los rodamientos de manguito (sleeve) al ofrecer una mayor fiabilidad, menor fricción y una vida útil significativamente más larga, lo cual es crítico en aplicaciones de control que pueden requerir un funcionamiento continuo. [13]



Fig.6 VENTILADOR DC 24V

L298N

El driver L298 permite controlar dos motores de corriente continua o motores paso a paso con una corriente máxima de 2

amperios. Actualmente existen módulos comerciales que incluyen el driver soldado y conectores para las entradas y salidas, facilitando su uso. En el módulo típico del driver L298 se encuentran los siguientes componentes:

Regulador de voltaje LM7805, que permite alimentar la lógica del módulo.

Conectores de salida Output A y Output B para conectar los motores.

Terminales de control (control inputs), que incluyen pines de habilitación para cada motor.

Un jumper que permite decidir si se usa o no el regulador de voltaje LM7805. [14]

Es importante considerar el voltaje con el que se alimentará el módulo. Aunque el driver soporta tensiones de hasta 36 V, si la alimentación es similar a la del microcontrolador (entre 6 y 12 V), no es necesario activar el regulador, ya que el L298 tolera esos niveles. En cambio, si se alimenta con tensiones superiores a 12 V, es obligatorio activar el jumper para usar el regulador; de lo contrario, el módulo puede dañarse rápidamente. [14]

Para controlar un motor de corriente continua con el driver L298, normalmente se utilizan dos pines de control que determinan el sentido de giro del motor, enviando señales digitales de uno o cero según corresponda. [14]

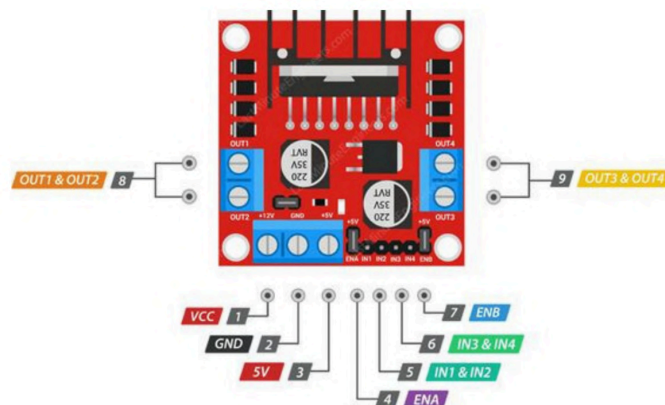


Fig.7 Driver L298 Pinout

Potenciómetro

Un potenciómetro es un componente electrónico pasivo que actúa como una resistencia variable mecánica. Su estructura fundamental consta de tres terminales: dos extremos conectados a una pista resistiva (hecha de carbón o alambre bobinado) y un tercer terminal central, llamado cursor o patín, que se desliza físicamente sobre esa pista. [15]

Conceptualmente, la principal aplicación del potenciómetro en un circuito con microcontrolador no es simplemente limitar la corriente (como un reóstato), sino funcionar como un divisor de tensión ajustable. Al aplicar un voltaje fijo (ej. 5V) a los dos terminales extremos y tomar la salida desde el terminal central (el cursor), la posición de este determina la fracción del voltaje total que se obtiene. Si el cursor está en un extremo, entregará 0V; si está en el otro, 5V; y si está en el centro, entregará 2.5V (en un potenciómetro lineal). Esto permite que un movimiento mecánico del usuario (girar una perilla o deslizar un control) se traduzca directamente en una señal de voltaje analógica, sirviendo como una entrada de control manual ideal para un Arduino. [15]

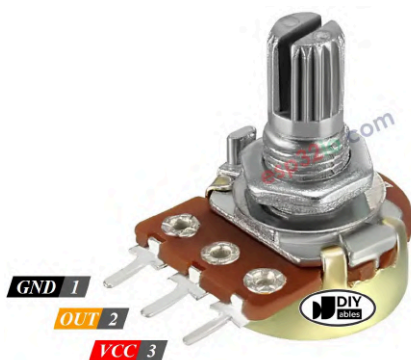


Fig.8 Potenciómetro Pinout

MOTOR DC

Un motor de corriente continua (DC) es un actuador cuya función principal es la conversión de energía eléctrica en energía mecánica, produciendo un movimiento rotatorio. Conceptualmente, su operación se basa en la fuerza de Lorentz, un principio electromagnético que describe la fuerza que experimenta un conductor eléctrico (una bobina) cuando una corriente fluye a través de él mientras se encuentra dentro de un campo magnético. [16]

El motor está compuesto por dos partes esenciales: el estator, que es la parte fija y proporciona el campo magnético (ya sea con imanes permanentes o electroimanes), y el rotor, que es la parte móvil que gira. Cuando se aplica una corriente continua (DC) al bobinado del rotor, este campo magnético interactúa con el campo del estator, generando un par de fuerzas (torque) que provoca el giro. Para que la rotación sea continua y no se detenga, el motor utiliza un conmutador (o colector). Este componente mecánico invierte inteligentemente la dirección de la corriente en el rotor cada media vuelta, asegurando que el par de empuje se mantenga siempre en la misma dirección. [16]

Una característica fundamental del motor DC es su naturaleza bidireccional. El sentido del torque (y, por tanto, el sentido de giro) depende directamente de la polaridad del voltaje aplicado. Al invertir la conexión de la alimentación (positivo por negativo), la dirección de la corriente en el rotor se invierte, lo que revierte la dirección de la fuerza de Lorentz y hace que el motor gire en el sentido opuesto. Para gestionar electrónicamente esta inversión desde un microcontrolador (como un Arduino), no se puede conectar el motor directamente; se utiliza un Puente H, que controla la polaridad y, por ende, la dirección del giro. [16]



Fig.9 Motor DC fischertechnik

JOYSTICK

El módulo de joystick es un dispositivo de entrada analógica que traduce el movimiento físico en dos ejes (X e Y) en señales

eléctricas que un microcontrolador puede interpretar. Conceptualmente, el módulo no es un solo componente, sino un sistema integrado por dos potenciómetros independientes y un mecanismo de cardán (gimbal) de resorte que los auto-centra. [17]

El mecanismo de cardán es la clave de su funcionamiento: al mover la palanca, este sistema mecánico descompone el movimiento (incluso uno diagonal) en rotaciones separadas para el eje horizontal (X) y el eje vertical (Y). Cada uno de estos ejes está acoplado a un potenciómetro que actúa como un divisor de tensión. Cuando el Arduino lee los pines VRx y VRy, en realidad está midiendo un voltaje analógico (entre 0V y 5V) que es proporcional a la posición de la palanca en ese eje. Un valor central (ej. ~512 en el ADC) indica la posición de reposo, mientras que los valores extremos (0 o 1023) indican el movimiento completo hacia un lado. Adicionalmente, el módulo incluye un pulsador digital (pin SW) que se activa al presionar la palanca, proporcionando una tercera vía de entrada. [17]



Fig.10 Joystick Pinout

Matriz/Tira NeoPixel (WS2812)

Los LED RGB direccionables, conocidos popularmente como NeoPixel o por su designación técnica WS2812, son una familia de componentes LED que se presentan en tiras, matrices u otros formatos. Estos dispositivos permiten un control individualizado de cada LED, facilitando la creación de efectos de iluminación complejos. Cada píxel suele consistir en un encapsulado 5050 que integra tres LED (rojo, verde y azul) junto con un chip controlador, como el WS2811 o el WS2812. Esto los diferencia de las tiras analógicas, donde todos los LED están conectados en paralelo y muestran el mismo color. [18]

El control de estas tiras digitales se realiza mediante una única línea de datos. Los píxeles se pueden encadenar conectando el pin de salida de datos de uno al pin de entrada del siguiente, requiriendo solo tres conexiones principales: Vcc (generalmente 5V), GND (tierra) y Datos. Debido a su consumo de energía, que puede alcanzar hasta 60 mA por píxel a máximo brillo, no deben alimentarse directamente desde un microcontrolador como Arduino. Se requiere una fuente de alimentación externa de 5V dimensionada para la corriente total de la tira. [18]



Fig.11 Tiras LED RGB direccionables
RFID-RC522

La Identificación por Radiofrecuencia (RFID) es una tecnología de comunicación inalámbrica diseñada para la identificación de objetos. Conceptualmente, el sistema se basa en dos componentes: un lector y una etiqueta (o tag). El lector genera activamente un campo electromagnético de alta frecuencia. Cuando una etiqueta, que usualmente es pasiva (sin batería), entra en este campo, sus circuitos internos se energizan y responden transmitiendo la información que tienen almacenada (como un número de serie único) de vuelta al lector. [19]

El módulo RC522 es un circuito integrado específico que actúa como el lector/grabador de RFID. En un proyecto con Arduino, el RC522 es el "traductor" que maneja la complejidad de la comunicación por radiofrecuencia. Opera en la frecuencia de 13.56 MHz y se encarga de generar el campo para energizar las etiquetas, así como de decodificar la "retrodispersión" (la señal de respuesta) de estas. Se comunica con el Arduino (el microcontrolador maestro) a través de un protocolo de alta velocidad, comúnmente SPI, permitiendo que el Arduino envíe comandos (como "buscar etiqueta") y reciba los datos de identificación. [19]

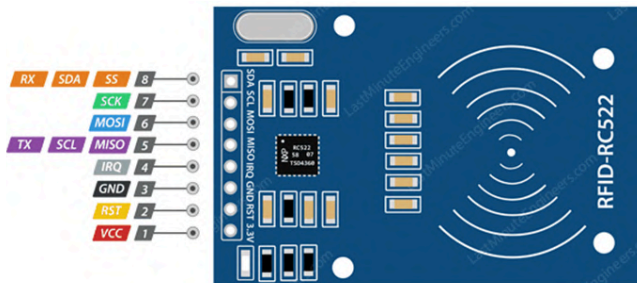


Fig.12 RFID-RC522 Pinout

LCD 16x2

Las pantallas de cristal líquido (LCD) de caracteres, como los modelos 16x2 o 20x4, son componentes comunes para mostrar datos como texto o símbolos en proyectos. Su funcionamiento se basa en el uso de cristales líquidos que, al recibir una corriente eléctrica, cambian su alineación para modular una luz de fondo, permitiendo o bloqueando selectivamente el paso de la luz a través de filtros polarizados para formar los caracteres en la pantalla. La mayoría de estas pantallas están basadas en el controlador Hitachi HD44780. [20]

Estas pantallas suelen tener 16 pines. Estos incluyen los pines de alimentación VCC y GND, los pines A y K para alimentar la luz de fondo, y un pin V0 que se conecta a un potenciómetro para ajustar el contraste de la pantalla. Para la comunicación, se utilizan los pines RS (Register Select), para diferenciar entre comandos y datos de caracteres, y E (Enable),

para indicarle a la pantalla cuándo debe leer los datos. Aunque la pantalla dispone de ocho pines de datos (D0-D7) para operar en modo de 8 bits, comúnmente se utiliza el modo de 4 bits. Este modo ahorra conexiones al microcontrolador, ya que solo requiere el uso de cuatro líneas de datos (D4-D7) para enviar la información en dos partes. [20]

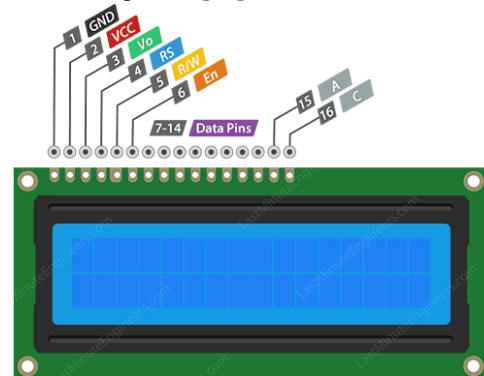


Fig.13 LCD 16X2 Pinout

IO PCF8564T

Este componente no es el LCD en sí, sino un módulo adaptador (o "mochila") que simplifica drásticamente su conexión. Conceptualmente, su propósito es actuar como un traductor de protocolo, convirtiendo la comunicación serial I2C en la comunicación paralela que requiere una pantalla LCD 1602 estándar. [21]

Un LCD 1602, por sí solo, necesita conectarse a un mínimo de 6 pines digitales del Arduino (RS, E, y 4 pines de datos D4-D7), consumiendo una gran cantidad de puertos. Este módulo adaptador resuelve ese problema utilizando un chip PCF8574 (un expansor de E/S). El PCF8574 "escucha" en el bus I2C (usando solo 2 cables: SDA y SCL) y pone sus 8 pines de salida a disposición. En este módulo, esos 8 pines ya vienen pre-cableados a los pines de control y datos del LCD. [21]

Por lo tanto, en lugar de que el Arduino tenga que manejar 6 pines complejos, simplemente envía un comando de alto nivel (como "escribir 'Hola'") a la dirección I2C del módulo. El PCF8574 recibe ese comando serial simple y lo traduce, generando las señales paralelas correctas en los pines del LCD para que muestre el texto. En resumen, abstrae la complejidad del control del LCD, liberando pines del microcontrolador. [21]

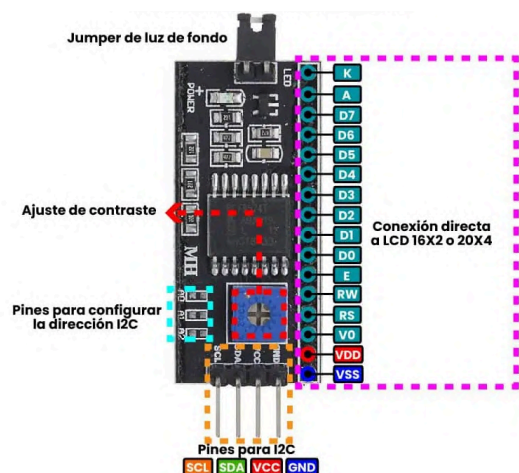


Fig.14 Modulo expensor IO PCF8564T Pinout

V. METODOLOGÍA

En el presente laboratorio se llevaron a cabo actividades basadas en la programación en lenguaje C aplicadas al microcontrolador ATmega328P. La práctica estuvo compuesta por cuatro actividades principales: la utilización de un plotter para llevar a cabo ciertas figuras, un sistema detector de temperatura, el control de un motor dc mediante potenciómetros, una matriz RGB controlada por un joystick y una cerradura RFID. A continuación, se detalla el procedimiento correspondiente a cada una de estas etapas.

Parte A:

En este problema se plantea la utilización de un plotter, mediante el desarrollo de un código en C, para realizar figuras geométricas y pictóricas: triángulo, cruz, círculo, gato y rana.

El desarrollo del código se estructuró en varias capas funcionales. Inicialmente, se configura el hardware mediante la función `setup_plotter()`, que establece los pines de los motores y del actuador del lápiz como salidas, y activa los controladores de los ejes. Seguido de la implementación de funciones generales de movimiento (`move_motor`, `move_axis`), que permiten desplazar los motores paso a paso en ambas direcciones, controlando la velocidad y el número de pasos. Estas funciones básicas se complementan con funciones de movimiento en los ejes (`mover_derecha`, `mover_izquierda`, `mover_arriba`, `mover_abajo`) y movimientos diagonales (como `mover_diag_arriba_der`), que combinan desplazamientos en ambos ejes para permitir trayectorias más específicas.

Para el control del lápiz, se definieron funciones `lower_pen()` y `lift_pen()` que bajan y levantan el lápiz respectivamente, permitiendo separar los trazos entre diferentes partes de la figura. Sobre estas funciones básicas se construyeron subrutinas específicas para dibujar figuras geométricas simples como triángulo, cruz y círculo, definiendo los pasos y movimientos necesarios para trazar cada forma.

Finalmente, se implementaron las funciones de dibujo de las figuras complejas como `dibujar_gato()` y `dibujar_rana()`. Estas rutinas combinan movimientos lineales, diagonales y control del lápiz. Cada figura está compuesta por múltiples segmentos secuenciales, planificados para asegurar continuidad y proporción en el dibujo final.

El flujo general del sistema consiste en inicializar el plotter, esperar un breve retardo de estabilización, y luego ejecutar un bucle infinito en el que se dibujan secuencialmente todas las figuras definidas. Entre cada dibujo se realiza un pequeño retraso para permitir visualizar correctamente cada figura antes de iniciar la siguiente. De esta manera, el código permite completar el trazado de múltiples figuras, utilizando una lógica modular que facilita la adición de nuevas formas o modificaciones en los movimientos de los motores sin afectar la estructura general del programa.

Parte B:

En este apartado se desarrolló un sistema de control de temperatura utilizando un sensor LM35, un calefactor y un ventilador. El sistema gestiona diferentes rangos de temperatura, determinando el encendido o apagado del calefactor y la velocidad del ventilador. Inicialmente se define un punto medio (PM), en el cual el sistema permanece en reposo (ningún componente activo). Este PM puede ser ajustado por el usuario, lo que permite recalibrar automáticamente los rangos de temperatura sin modificar el

código. El sistema fue implementado y probado bajo dos modalidades: física y simulada.

Para la implementación física, el desarrolló del código se basa en la lectura del conversor analógico-digital (ADC), con ese valor y en relación a un punto medio, el sistema determina la acción correspondiente del calefactor y el ventilador. Si la temperatura es inferior al rango deseado, el calefactor se enciende a potencia media o alta para aumentar la temperatura, si la temperatura se encuentra cercana al PM, ambos dispositivos permanecen apagados (reposo), y si la temperatura supera los rangos establecidos, el ventilador se enciende a velocidad baja, media o alta según el nivel de temperatura. El PM puede ajustarse mediante un menú interactivo enviado a través del puerto UART, permitiendo al usuario recalibrar los rangos de control sin necesidad de modificar el código. Durante la operación, todos los datos de temperatura, estados del calefactor y ventilador, y el PM actual, se almacenan en arreglos internos para su posterior análisis. Para la visualización de estos resultados, se utiliza un script en Python que lee los datos guardados en un archivo de texto generado por el microcontrolador, graficando la evolución de la temperatura, el PM y los estados de los actuadores, lo que permite analizar el comportamiento del sistema físico.

Con respecto a la simulación, se utiliza un código muy similar, con la diferencia de que, no se guardan datos en arreglos internos para exportarlos a un archivo, sino que cada medición de temperatura, así como los estados del calefactor y el ventilador, se transmiten inmediatamente por UART al script de Python. Este enfoque permite observar en tiempo real cómo el sistema responde a cambios de temperatura y ajustes del PM. El script Python realiza un filtrado de las líneas recibidas para descartar comandos o información de menú y asegura que únicamente los datos válidos de temperatura, calefactor, ventilador y PM se almacenen y representen en las gráficas. Asimismo, incluye un widget interactivo que permite enviar un nuevo valor de PM directamente al microcontrolador simulado, actualizando instantáneamente los rangos de control y facilitando la validación del algoritmo sin necesidad de hardware.

El flujo de operación del sistema, tanto en su versión física como simulada, sigue un esquema similar. Inicialmente se configuran los módulos de UART, ADC y PWM para calefactor y ventilador. Posteriormente, se ejecuta un bucle de control que realiza la lectura de la temperatura, calcula el rango de acción según el PM, aplica la acción correspondiente sobre los actuadores y envía o almacena los datos para su visualización. En la versión física, los datos se registran y se grafican posteriormente mediante Python; en la versión simulada, los datos se transmiten y grafican en tiempo real, con posibilidad de ajuste dinámico del PM.

Parte C:

El problema C se basa en un sistema de control de un motor mediante un potenciómetro. El programa lee el valor de un potenciómetro que actúa como referencia y ajusta la velocidad y el sentido de giro del motor, cuyo eje está conectado a un segundo potenciómetro que mide la posición actual.

El sistema se divide en dos fases: simulación e implementación física, pero ambas comparten el mismo código base en C para el ATmega y el mismo monitor gráfico en Python.

La lógica implementada en C gestiona el control de posición del motor y la monitorización del sistema. El microcontrolador comienza configurando los pines de salida del motor, el Conversor Analógico-Digital (ADC) para la lectura de señales y un PWM de 8 bits en modo Fast PWM para la modulación de velocidad. Para facilitar los procesos, la transmisión UART se inicializa y se redirige a stdout, lo que permite el uso directo de la función printf.

Dentro del bucle de control, el microcontrolador lee continuamente las señales analógicas de referencia y de posición actual mediante el ADC. Con estos valores, calcula el error y aplica un control proporcional (KP) que incluye una zona muerta (DEADBAND) para evitar vibraciones. El resultado de este control determina tanto la dirección como el valor PWM que se aplicará al motor. Finalmente, estos datos (Referencia, Posición Actual, PWM y Dirección) se transmiten periódicamente por UART en el formato Ref,Actual,PWM,Dir\n, permitiendo su visualización en tiempo real mediante un programa externo.

El monitor en Python se encarga de recibir estos datos por el puerto serial, interpretarlos y representarlos en gráficas en tiempo real. Se utilizan dos subplots: uno para la referencia y la posición actual del motor, y otro para el PWM aplicado, representando la dirección del motor mediante el signo del PWM. Los datos se almacenan en listas tipo deque para mantener un número limitado de puntos visibles y se actualizan mediante matplotlib.animation cada 20 ms.

En la fase de simulación, el código se prueba en un modelo virtual del ATmega dentro de Proteus, donde printf se redirige al componente COMPIM que simula el puerto serial. Para que Python reciba los datos, se utiliza un software de puente virtual (como VSPE) que conecta el COM virtual de Proteus con el COM que Python escucha.

En el ensamblaje físico, se utilizan los mismos códigos. Se carga el código en C a un software como Microchip, se compila y se envía el archivo .hex al ATmega328P real. El sistema operativo detecta este hardware y le asigna un puerto COM real, en nuestro caso COM3. Por lo que se ajustó esa variable SERIAL_PORT en el script de Python para que coincida con el puerto asignado por Windows. Es fundamental que la velocidad de transmisión (BAUD_RATE) y la configuración del protocolo UART coincidan exactamente entre el microcontrolador y los códigos.

Una vez que los datos llegan al script de Python, se grafican en tiempo real. La gráfica de PWM se representa de -255 a +255, usando el signo para indicar la dirección del motor (positivo para horario, negativo para antihorario), permitiendo observar simultáneamente la magnitud y el sentido de la acción de control aplicada.

Parte D :

Este problema consistió en la implementación de una matriz RGB de 8x8 controlada por un joystick analógico de dos ejes y un pulsador. El sistema comienza con la inicialización de un Led en el centro de la matriz, el cual podrá ser controlado mediante el joystick: si se mueve el control analógico hacia arriba, el led se desplazará una posición hacia arriba, comportamiento que será implementado con el resto de los movimientos (abajo, derecha e izquierda). Por otra parte, cuando se presione el botón del joystick, el LED debe cambiar de color de forma aleatoria.

El código desarrollado para este apartado se puede separar en 3 partes principales: inicialización y generación de valores aleatorios, control de la matriz led y control del joystick (movimiento y botón).

Con respecto a la primera parte, se configuran los pines de entrada y salida y se inicializa el ADC para el joystick. Para la generación de colores aleatorios, se obtiene una semilla basada en múltiples lecturas del ADC para garantizar variabilidad. El bucle principal combina la lectura de entradas, la actualización de posición y color, y la transmisión a la matriz, permitiendo un control interactivo en tiempo real del LED seleccionado.

Para el control de la Matriz, se utiliza una estructura llamada Color para almacenar los valores RGB de cada LED y se implementa la función ws2812_send, la cual envía los datos a los LEDs utilizando el protocolo WS2812 mediante bit-banging. Esta función desactiva temporalmente las interrupciones para asegurar la temporización precisa requerida por los LEDs. En cada iteración del bucle principal, todos los LEDs se apagan y solo se enciende el LED activo con su color correspondiente, logrando la actualización visual de la matriz en tiempo real.

En cuanto al control del joystick, se lee continuamente la posición en los ejes X e Y mediante el ADC, comparando cada lectura con el centro calibrado y aplicando una zona muerta para evitar movimientos involuntarios. Dependiendo de la dirección detectada, la posición del LED se actualiza desplazándose en la matriz hacia arriba, abajo, izquierda o derecha, respetando los límites de la matriz.

Por otro lado, la lectura del botón permite cambiar el color del LED activo; para evitar lecturas erróneas se implementa un pequeño retardo que funciona como anti-rebote, y la generación del nuevo color se realiza de manera aleatoria utilizando la semilla obtenida al inicio.

Una vez realizado el código, se continuó con la implementación física del sistema. Se siguieron las conexiones correspondientes. Posteriormente, se realizaron pruebas de funcionamiento iniciales, comprobando la respuesta del LED a los movimientos del joystick y la correcta actualización del color al presionar el botón, ajustando los retardos y calibraciones necesarias para asegurar un control preciso y estable del sistema.

Parte E:

El problema E consiste en el desarrollo de una cerradura electrónica inteligente que utiliza un lector RFID RC522 para la autenticación de usuarios mediante tarjetas o llaveros RFID. El objetivo del sistema es permitir el acceso únicamente a usuarios autorizados, ofreciendo al mismo tiempo opciones de registro, actualización y eliminación de tarjetas.

Además el sistema cuenta con una pantalla LCD con interfaz I2C, utilizada para mostrar mensajes informativos e instrucciones al usuario, junto con dos indicadores LED (rojo y verde) que reflejan el estado del acceso. Además, se implementan dos botones físicos, uno destinado a borrar la tarjeta registrada y otro a actualizarla con una nueva.

El código desarrollado para este programa se puede dividir en varias partes esenciales: inicialización del sistema, lectura y procesamiento de tarjeta RFID, comparación y control de acceso, registro de actualización de tarjeta, borrado de tarjeta registrada y comunicación UART.

En la parte inicial se configuran los puertos de entrada y salida de los Leds, botones y la comunicación UART. Se

inicializan los periféricos necesarios: UART (transmisión de mensajes de estado), pantalla LCD con módulo I2C (para mensajes de estado), RFID RC522 (lectura de las tarjetas mediante comunicación SPI) y EEPROM interna (almacena de forma permanente el UID autorizado). Una vez configurado todo el hardware, el sistema muestra el mensaje de bienvenida y queda a la espera de una tarjeta.

Para la lectura y procesamiento de tarjetas RFID, el microcontrolador monitorea continuamente la presencia de una tarjeta en el campo del lector RC522. Cuando se detecta una tarjeta, el módulo RFID lee su número de identificación (UID) y lo almacena temporalmente en un arreglo. Este valor se transmite también por UART (solo por este medio de comunicación), permitiendo visualizar el UID leído desde el monitor serial.

En la etapa de comparación y control de acceso, el UID leído se compara con el valor almacenado en la EEPROM. Si coinciden, el sistema activa el LED verde y muestra “Acceso permitido”. Si el UID no coincide, se enciende el LED rojo y se muestra “Acceso denegado”, indicando un intento no autorizado.

El sistema también permite la actualización de tarjetas mediante un botón. Al presionarlo, el microcontrolador entra en modo registro, solicita acercar una nueva tarjeta, lee su UID y lo almacena en la EEPROM, reemplazando el anterior. Esto se confirma mediante la pantalla LCD y comunicación UART mostrando “Nueva tarjeta registrada”.

Para el borrado de una tarjeta registrada se realiza el mismo procedimiento mediante otro botón, que elimina el ID almacenado en la EEPROM y muestra “Tarjeta borrada”. Hasta que se registre una nueva tarjeta autorizada, ningún otro intento de acceso será aceptado.

La comunicación UART se mantiene activa durante todo el funcionamiento del sistema, enviando mensajes informativos, resultados de lectura y estados del sistema, lo que facilita la supervisión y validación del código.

Al final, el sistema fue probado y validado en ambas modalidades (físico y simulado). En la simulación, dado que el módulo RFID RC522 no está disponible en PicSimLab ni en Proteus, se utilizó únicamente la comunicación UART, ingresando manualmente los UID de las tarjetas para simular la lectura del lector. Esto permitió comprobar la lógica de registro, actualización, borrado y control de acceso, así como la correcta visualización de los mensajes en la pantalla LCD y el envío de información por UART.

En la implementación física, se realizaron todas las conexiones necesarias con el lector RFID RC522, la pantalla LCD I2C, los LEDs indicadores y los botones, logrando que el sistema funcionara correctamente con tarjetas reales.

VI. RESULTADOS

En este apartado se ejemplifican y desarrollan los resultados obtenidos en cada parte de la práctica.

Parte A:

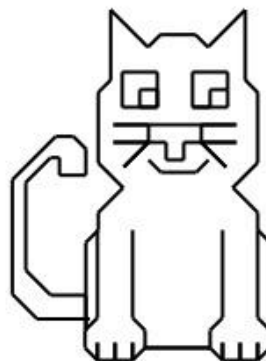


Fig.15 Imagen de referencia para la figura Gato

Esta imagen sirvió como referencia a la hora de realizar la figura en el plotter. Se contabilizaron sus movimientos y se tradujeron al código del plotter. Este mismo procedimiento se realizó para la Rana, la imagen de referencia se puede observar en la siguiente imagen.

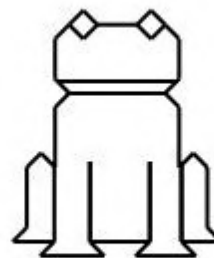


Fig.16 Imagen de referencia para la figura Rana

Para lograr estas imágenes se realizó el siguiente manejo del código:

```
void move_axis(uint8_t axis, uint8_t
direction, uint16_t steps) {
    if (axis == 0) move_motor(&PORTB, DIR_X,
&PORTB, STEP_X, direction, steps);
    else move_motor(&PORTC, DIR_Y, &PORTC,
STEP_Y, direction, steps);
}
```

Este fragmento muestra cómo se abstrae el movimiento de los motores, indicando el eje, la dirección y la cantidad de pasos. Con esta función se puede controlar de forma sencilla cualquier desplazamiento en los ejes X e Y, permitiendo combinar movimientos para realizar las figuras más complejas.

Para dibujar las figuras se siguió este procedimiento:

```
void dibujar_triangulo(float lado){
    lower_pen();
    mover_derecha(lado);
    mover_diag_arriba_izq(lado);
    mover_abajo(lado);
    lift_pen();
}
```

En este fragmento se evidencia cómo se combinan movimientos lineales y diagonales para trazar una figura completa.

Para figuras más complejas, como el gato o la rana, se utilizó la misma lógica básica de movimientos, pero secuenciada en múltiples pasos, alternando la bajada y elevación del lápiz (lower_pen() / lift_pen()) según los trazos. Esto permitió reproducir detalles finos y garantizar que cada figura se dibuje correctamente en la hoja.

Parte B:

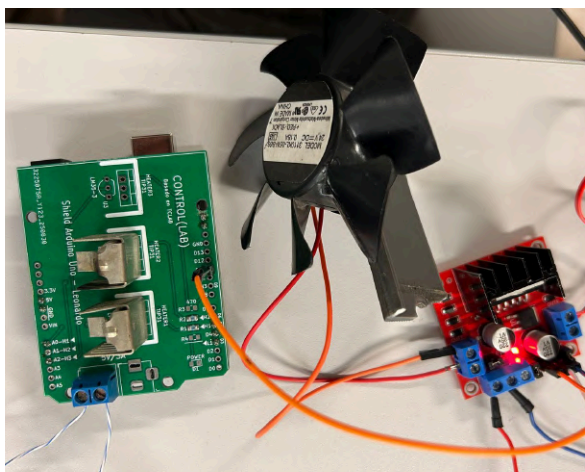


Fig.17 Ensamblaje físico - Sistema de control de temperatura

En la presente figura se evidencia el ensamblaje realizado a la hora de implementar el sistema de control de temperatura.

TABLE III

CONEXIONES SISTEMA DE CONTROL DE TEMPERATURA

Elemento	Conexión
TCLAB	
LM35	PC0
Calentador	PD5
Pwm Ventilador	PB3
-V	GND compartido
+V	5V
Puente H	
Pwm TCLAB (PD5)	IN3
Voltaje de Entrada	12V
Tierra	GND compartido
Salida motor (out3)	GND ventilador
Salida motor (out4)	+V ventilador

En esta tabla se pueden observar las conexiones realizadas en el ensamblaje del sistema de control de temperatura, como se muestra en la misma, se utilizó un puente H para la alimentación del ventilador, además, se utilizó una segunda fuente para la alimentación de la placa TCLAB.

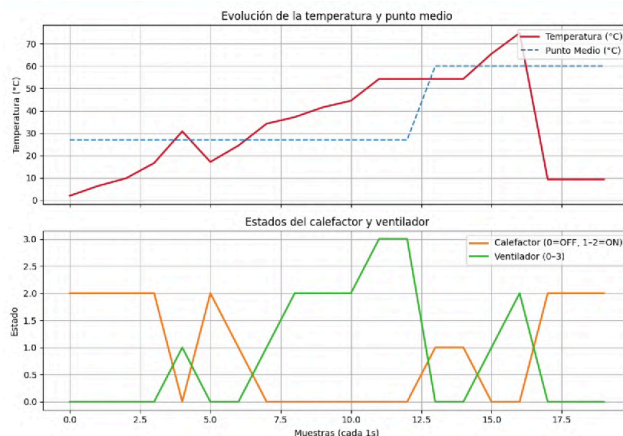


Fig.18 Gráfica - Sistema de control de temperatura

Esta gráfica se realizó respecto a los datos del sistema de control de temperatura. Como se observa hay dos sistemas, uno gráfica refleja la temperatura respecto al tiempo y el punto medio establecido en el sistema. Por su parte, la otra gráfica muestra los estados de los componentes (calefactor y ventilador), teniendo en cuenta que el ventilador cuenta con 3 modos: Bajo (1), Medio (2) y Alto (3), mientras que, el calefactor solo cuenta con dos estados: Alto (2) y Medio (1).

El siguiente fragmento muestra la lógica de actuación según la temperatura medida:

```
if (tempC <= lim1) {
    set_heater_power(254);
    set_fan_speed(0);
    heater_state = 2; fan_state = 0;
    sprintf(buffer, "T:%.2fC | Calefactor encendido (ALTO)\n", tempC);
}
else if (tempC > lim1 && tempC <= lim2) {
    set_heater_power(150);
    set_fan_speed(0);
    heater_state = 1; fan_state = 0;
    sprintf(buffer, "T:%.2fC | Calefactor encendido (MEDIO)\n", tempC);
}
```

El sistema respondió correctamente ante los cambios de temperatura, activando el calefactor cuando el valor medido se encontraba por debajo del punto medio y el ventilador cuando lo superaba. Esto permitió mantener la temperatura dentro del rango de control definido, evidenciando el correcto funcionamiento de la lógica implementada y la coherencia de los datos registrados.

Además, se comprobó la transmisión UART mediante el comando "datos", que permitió imprimir el registro de temperatura y estados de los actuadores (en el ensamblaje físico):


```
sprintf(linea, "%.2f,%d,%d,%d\r\n",
datos_temp[i], datos_heat[i], datos_fan[i],
datos_pm[i]);
UART_print(linea);
```

En la simulación, se mantuvieron los mismos parámetros y rangos de control, pero utilizó una rutina que envía periódicamente los valores al monitor serial en formato CSV

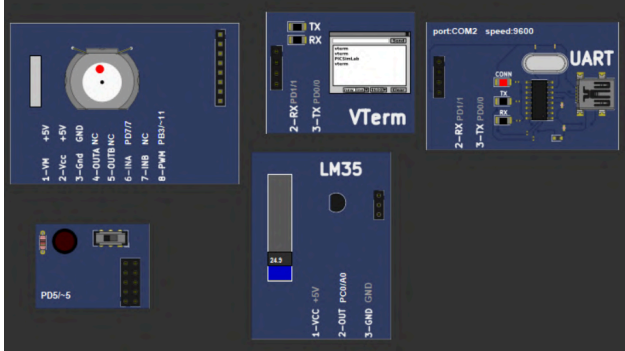


Fig.19 Simulación - Sistema de control de temperatura

Esta imagen evidencia el sistema implementado de forma simulada, el mismo cuenta con: una led representado el calefactor, un sensor LM35, un motor DC simulando el ventilador y dos módulos UART, uno corresponde a la impresión de datos (VTerm) y otro a la conexión entre PicSimLab y PyCharm (simulación).

Parte C:

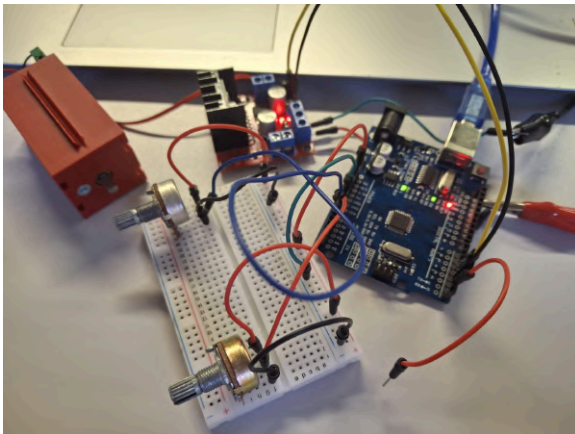


Fig.20 Ensamblaje físico - Control Motor

Esta imagen refleja el ensamblaje realizado para el apartado C del laboratorio, control de un motor.

TABLE IV
CONEXIONES CONTROL MOTOR

Elemento	Pin Microcontrolador
Potenciómetros	
PR VOUT	PC1
PR VCC	5V
PR GND	GND
PA VOUT	PC0
PA VCC	5V
PA GND	GND
Puentes H	
Voltaje de Entrada	9V
Tierra	GND
PD2	IN3
PD3	IN4
Salida motor (out3)	Negativo motor
Salida motor (out4)	Positivo motor
Motor	
ROJO	PB0
VERDE	PB1

Tabla de conexiones para sistema de control de un motor dc. En este caso se volvió a utilizar un puente H, el cuál permitió alimentar el motor (9 V).

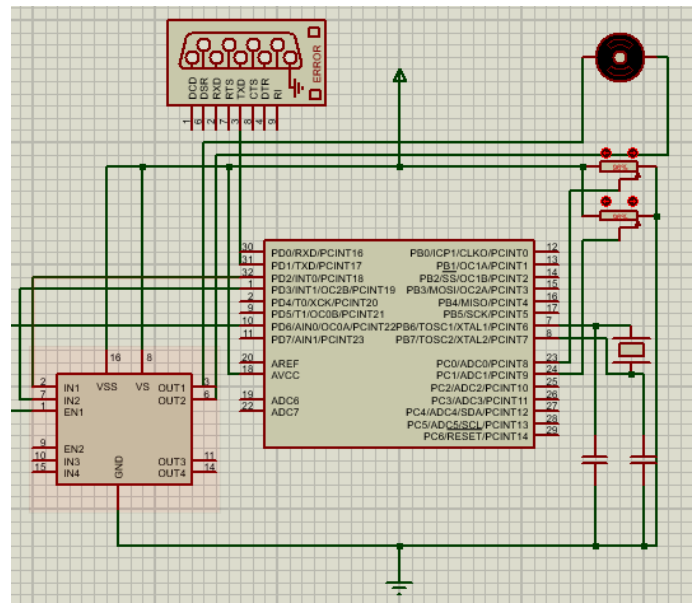


Fig.21 Simulación - Control Motor

Esta imagen evidencia la simulación utilizada para representar el control de motor DC. Se implementó de la misma forma que se ensambló en físico, a diferencia del componente COMPIN que sirve para establecer una relación entre Proteus y PyCharm (simulación), capaz de enviar datos para realizar la gráfica.

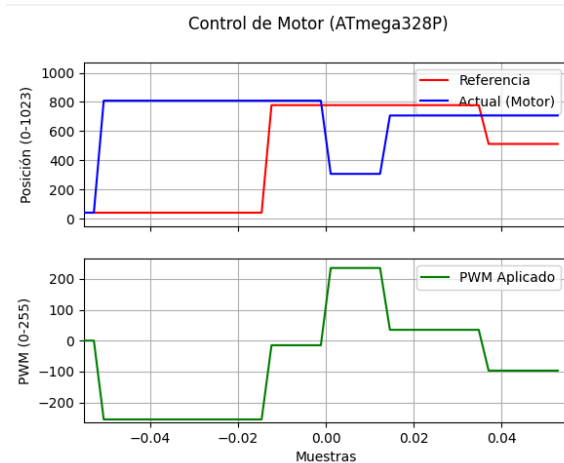


Fig.22 Gráfica simulado - Control Motor

Esta figura permite observar los resultados obtenidos respecto al funcionamiento simulado. Nuevamente podemos visualizar dos gráficas, la primera refleja las posiciones de los dos potenciómetros, la segunda el pwm aplicado.

El siguiente fragmento muestra el envío de datos para su monitoreo:

```
printf("%d,%d,%d,%d\n", referencia, actual,
pwm, direccion);
```

Nuevamente se utiliza el formato CSV mediante UART para el envío de estos datos.

El motor siguió la referencia establecida con un pequeño retardo proporcional al error.

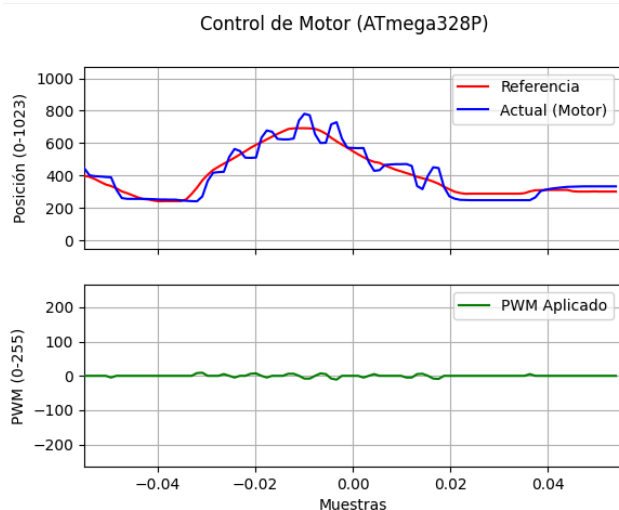


Fig.23 Gráfica implementación física- Control Motor

En esta gráfica visualizamos los mismos datos pero implementados en forma física. El motor siguió la referencia establecida con un pequeño retardo proporcional al error.

Parte D:

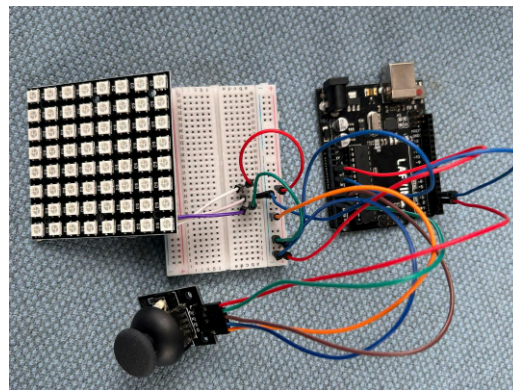


Fig.24 Ensamblaje - Matriz RGB con Joystick

Esta imagen refleja el ensamblaje que se realizó para la matriz RGB controlada por un Joystick. Las conexiones realizadas se observan en la siguiente Tabla.

TABLE V

CONEXIONES MATRIZ RGB CON JOYSTICK

Elemento	Pin Microcontrolador
Matriz	
+V	5V
Out	PD3
-V	GND
Joystick	
+V	5V
-V	GND
VRx	A0
VRy	A1
SM	PD2

El desplazamiento del LED se realizó mediante lectura de los canales ADC correspondientes a los ejes X y Y, con calibración inicial y zona muerta para evitar ruido:

```
int leerMovimiento(uint8_t canal) {
    int valor = ADC_read(canal);
    if (valor > (centro - DEADZONE) && valor
    < (centro + DEADZONE)) return 0;
    return (valor > centro) ? 1 : -1;
}
```

La posición activa del LED se actualizó en la matriz según el movimiento del joystick:

```
if (movX == 1 && posX < ANCHO - 1) posX++;
else if (movX == -1 && posX > 0) posX--;
```

Para garantizar la variabilidad en cada cambio de color, se inicializó la semilla del generador rand() con lecturas analógicas del joystick:

```
uint32_t semilla = 0;
for (int i = 0; i < 50; i++) semilla +=
ADC_read(0);
srand((unsigned int)semilla);
```

Y al presionar el botón:

```
if (botonPresionado()) {
    generarColorAleatorio(&colorActual);
}
```

Todo esto permitió un correcto funcionamiento del sistema.
Parte E:

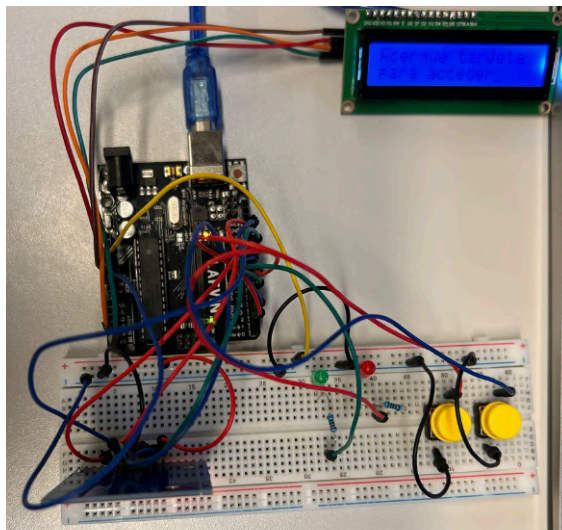


Fig.25 Ensamblaje físico - Cerradura RFID

Esta imagen evidencia el ensamblaje físico del sistema de cerradura RFID.

TABLE VI
CONEXIONES CERRADURA RFID

Elemento	Pin Microcontrolador
RC522	
VCC	3.3V
GNC	GND
RST	PB1
SDA/SS	PB2
MOSI	PB3
MISO	PB4
SCK	PB5
LCD	
SDA	PC4
SCL	PC5
VCC	5V
GND	GND
Leds / Pulsadores	
Led verde	PD6
Led rojo	PD7
Pulsador Borrar	PD2
Pulsador Actualizar	PD3

Tabla de conexiones para la cerradura RFID.

La detección y comparación del UID se realiza mediante:

```
mfr522_standard(card_uid);
if (memcmp(card_uid, stored_uid, UID_LEN) ==
0) {

display_and_log("Acceso", "permitido!", "Acceso
o permitido!!");
PORTD |= (1<<LED_VERDE);
} else {

display_and_log("Acceso", "denegado!", "Acceso
denegado");
PORTD |= (1<<LED_ROJO);
}
```

Durante el registro, el sistema espera una nueva tarjeta y guarda su UID en la memoria EEPROM:

```
void handle_registration(void) {
    display_and_log("Modo registro",
"Acerque tarjeta", "Modo registro");
mfr522_standard(card_uid);
write_stored_uid(card_uid);
uart_print("Nueva tarjeta guardada: ");
uart_print_hex_array(card_uid, UID_LEN);
}
```

Los mensajes de estado se envían por UART y se muestran en la pantalla LCD:

```
uart_print("Sistema Cerradura RFID
iniciado\r\n");
display_and_log("Bienvenido al", "sistema
RFID", "Sistema iniciado");
```

A continuación, se muestran los mensajes UART, los cuáles se reproducen en la pantalla LCD.

```
=== Enviando REQA ===
Soft Reset...
Configurando temporizadores y modulacion...
Sistema Cerradura RFID iniciado
Bienvenido al sistema RFID

=== Enviando REQA ===

=== Enviando REQA ===
Tarjeta detectada! Intentando leer UID...
UID leído!
Tarjeta detectada: 0x93 0x11 0xFF 0x28 0x55
Acceso permitido!!
```

Fig.26 Mensaje Uart - Cerradura RFID

```

=== Enviando REQA ===
Timeout REQA, tarjeta no detectada
Tarjeta borrada

=== Enviando REQA ===

=== Enviando REQA ===
Tarjeta detectada! Intentando leer UID...
UID leído!
Tarjeta detectada: 0x93 0x11 0xFF 0x28 0x55
No hay tarjeta registrada
Modo registro

```

Fig.27 Mensaje Uart - Cerradura RFID

```

=== Enviando REQA ===

=== Enviando REQA ===

=== Enviando REQA ===

=== Enviando REQA ===
Tarjeta detectada! Intentando leer UID...
UID leído!
Nueva tarjeta guardada: 0x93 0x11 0xFF 0x28 0x55 0x00 0x00 0x00 0x00 0x00
Tarjeta registrada

=== Enviando REQA ===
Timeout REQA, tarjeta no detectada

=== Enviando REQA ===

```

Fig.28 Mensaje Uart - Cerradura RFID

Las interrupciones externas (pulsadores) permiten actualizar o borrar la tarjeta registrada sin reiniciar el sistema:

```

ISR(INT1_vect) { // Actualizar tarjeta
    flag_actualizar = 1;
}
ISR(INT0_vect) { // Borrar tarjeta
    flag_borrar = 1;
}

```

Al detectar una tarjeta registrada, el sistema permitió el acceso mediante mensajes en el LCD y UART, activando el LED verde. En caso contrario, mostró "Acceso denegado" y encendió el LED rojo. Los botones de actualización y borrado respondieron correctamente.

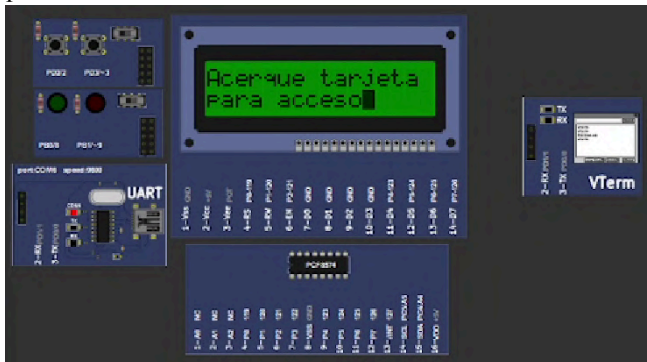


Fig.29 Simulación - Cerradura RFID

Esta figura refleja la simulación realizada en este apartado. Para poder realizarlo de forma simulada se utilizó únicamente la comunicación UART.

VII. CONCLUSIONES

El presente laboratorio permitió integrar conocimientos teóricos y prácticos en el ámbito del control electrónico mediante microcontroladores, consolidando habilidades de programación en lenguaje C, sensado, actuación y comunicación serial. Cada uno de los sistemas desarrollados evidenció la importancia del diseño estructurado, la validación experimental y la documentación técnica como pilares fundamentales en la implementación de sistemas de control.

Parte A

La implementación del sistema de control del plotter permitió comprender de forma práctica y aplicada el principio de funcionamiento de los motores paso a paso, su secuenciación de pasos y la sincronización entre los ejes X e Y para la generación de trayectorias bidimensionales. A través del uso del microcontrolador, se logró establecer un control preciso del movimiento lineal, garantizando la correcta ejecución de figuras geométricas predefinidas dentro de los límites físicos del sistema. La correcta gestión de las señales de dirección (DIR) y reloj (CLK) para cada motor fue esencial para obtener desplazamientos uniformes y coherentes, evitando vibraciones o pérdidas de pasos durante el trazado.

Asimismo, se implementó el control de una válvula solenoide encargada de activar o desactivar el trazo del lápiz o herramienta de corte, coordinando su accionar con el movimiento de los ejes. Esta integración demostró la importancia de la sincronización temporal entre eventos mecánicos y señales de control digital, destacando el papel del temporizado por software y hardware para mantener la precisión de ejecución.

Debido a limitaciones en el entorno de laboratorio, no fue posible implementar la comunicación serial mediante UART para la interacción con el usuario. En su lugar, el control del sistema se realizó directamente a través del programa principal, ejecutando las secuencias de movimiento y activación del solenoide de forma predefinida. Esta configuración permitió validar correctamente la lógica de control y la sincronización entre los ejes, aun sin la interfaz serial. A pesar de esta restricción, la experiencia resultó valiosa para comprender la estructura modular del código y la forma en que podrían integrarse futuras mejoras, como menús interactivos o comandos externos, mediante comunicación UART o interfaces gráficas en PC.

Parte B

La implementación del sistema de control de temperatura basado en el microcontrolador se desarrolló con éxito, cumpliendo los objetivos de medición, regulación y visualización del comportamiento térmico. El sistema logró leer correctamente las señales del sensor LM35, procesarlas mediante el convertidor analógico-digital (ADC) y accionar los actuadores correspondientes (calefactor y ventilador) según los rangos definidos. En simulación, el control respondió de manera adecuada ante las variaciones de temperatura, mostrando una regulación progresiva y estable.

Durante la implementación del proyecto se evidenció que la doble utilización de un mismo puerto COM para enviar datos vía UART y, al mismo tiempo, graficarlos en tiempo real no es posible de manera directa. Los puertos COM funcionan como

canales de comunicación secuenciales, y cualquier intento de lectura y escritura simultánea puede generar pérdida de datos o errores de sincronización. Por ello, la visualización de gráficos y la transmisión de datos requieren estrategias que gestionen adecuadamente el flujo de información, como buffers intermedios o la separación de tareas en tiempo, para garantizar que la comunicación sea confiable y los datos representados sean precisos.

A pesar de estas dificultades, el sistema se logró implementar con éxito, respondiendo correctamente ante los cambios de temperatura y accionando los actuadores según lo previsto. En las pruebas, la simulación de la evolución térmica y la generación de la gráfica temperatura-tiempo en Python permitieron visualizar el comportamiento dinámico y la estabilidad alcanzada por el control automático.

Parte C

El desarrollo del sistema de control de motor con referencia analógica resultó satisfactoria tanto en la simulación como en la ejecución física. El microcontrolador logró interpretar correctamente las señales analógicas provenientes de ambos potenciómetros, estableciendo una correspondencia dinámica entre el valor de referencia y el valor real del eje del motor. Durante las pruebas, se observó que el potenciómetro acoplado al eje del motor modificaba su posición en sentido horario o antihorario, dependiendo de la diferencia respecto al potenciómetro de referencia, hasta alcanzar el equilibrio entre ambos valores.

En la primera gráfica obtenida se puede apreciar el seguimiento del valor del potenciómetro de referencia por parte del potenciómetro acoplado al motor, evidenciando la estabilidad del sistema y la capacidad de corrección del control implementado. En la segunda gráfica, se muestra la señal PWM utilizada para ajustar la velocidad y el sentido de giro del motor; los valores negativos representados responden a una convención adoptada para indicar el cambio de dirección, no a una inversión real de la polaridad.

Durante la implementación física, se detectó que el ensamble mecánico presentaba baja robustez estructural, lo que dificultó la estabilidad del sistema durante las pruebas. Tanto la protoboard como el motor tendían a desplazarse o girar involuntariamente al aplicar el par de fuerza, por lo que fue necesario sostener manualmente el conjunto para evitar movimientos no deseados. Esta limitación, si bien no afectó el funcionamiento lógico del control, sí evidenció la importancia del diseño mecánico y la fijación adecuada de los componentes en sistemas mecatrónicos.

En conjunto, el experimento permitió validar el principio de control de realimentación utilizando señales analógicas y modulación por ancho de pulso, demostrando la efectividad del algoritmo implementado y la capacidad del microcontrolador para regular actuadores en función de variables continuas. Además, se destacaron aspectos de mejora futuros, como el uso de una estructura más sólida, el empleo de sensores de posición más precisos y la incorporación de controladores de tipo proporcional-integral para optimizar la respuesta del sistema.

Parte D

La implementación del sistema de control de la matriz RGB mediante un joystick analógico se desarrolló exitosamente, cumpliendo con todos los requerimientos planteados en la consigna. El microcontrolador logró interpretar de forma precisa las señales analógicas provenientes de los ejes X e Y del joystick, permitiendo el desplazamiento controlado del LED activo dentro de la matriz en las cuatro direcciones. Este comportamiento demostró un correcto funcionamiento del convertidor analógico-digital (ADC) interno, así como una adecuada gestión de las señales de entrada para obtener una respuesta estable y coherente con el movimiento del usuario.

Para evitar movimientos erráticos o activaciones múltiples, se implementó una zona muerta (umbral de sensibilidad) en la lectura analógica, lo que permitió filtrar variaciones menores de voltaje y mejorar la estabilidad del sistema. De esta forma, los desplazamientos se produjeron únicamente cuando el joystick superó los límites establecidos, garantizando un control preciso y sin ruido en las lecturas.

La función de cambio de color al presionar el botón del joystick también se ejecutó correctamente, generando combinaciones aleatorias de color RGB mediante el uso de la función de número pseudoaleatorio del estándar C. El color seleccionado permaneció activo al desplazarse el LED a nuevas posiciones, cumpliendo con lo solicitado y evidenciando un control eficiente de los canales de color y del manejo interno de la matriz. La comunicación con los LEDs WS2812 se implementó por software, controlando los tiempos de transmisión a nivel de bit con precisión, lo que permitió una actualización correcta de los 64 elementos sin fallas de sincronización.

En términos generales, el sistema mostró un comportamiento estable, fluido y confiable, validando la integración efectiva entre hardware y software. La experiencia permitió reforzar el manejo de entradas analógicas, el procesamiento de señales y la generación de salidas PWM.

Parte E

La implementación del sistema de cerradura RFID fue exitosa, cumpliendo con todos los requerimientos funcionales establecidos en la consigna. El dispositivo logró identificar de manera confiable las tarjetas o llaveros RFID mediante el módulo RC522, gestionando correctamente las operaciones de registro, verificación, actualización y borrado de credenciales. La integración con la memoria EEPROM interna permitió almacenar de forma persistente el identificador único (UID) de la tarjeta autorizada, manteniendo los datos aún tras interrupciones de energía, lo que demuestra una adecuada gestión de la no volatilidad del sistema.

La interfaz de usuario, compuesta por la pantalla LCD con protocolo I²C y los LEDs indicadores, proporcionó una comunicación visual clara e intuitiva durante todas las etapas de funcionamiento. Los mensajes de estado ("Acceso permitido", "Acceso denegado", "Nueva tarjeta registrada", "Tarjeta borrada") se mostraron correctamente tanto en la LCD como a través del puerto UART, lo que permitió realizar un monitoreo remoto y una trazabilidad completa del proceso. La comunicación serial también facilitó las pruebas de validación y depuración del código durante el desarrollo.

El sistema demostró un comportamiento estable y tiempos de respuesta adecuados al momento de aproximar una tarjeta,

con una correcta sincronización entre la lectura del módulo RFID y la actualización de los indicadores visuales. Además, la incorporación de botones físicos para las funciones de borrado y actualización de tarjetas otorgó mayor versatilidad y seguridad al dispositivo, al permitir una gestión directa de las credenciales sin requerir modificaciones de software.

Desde el punto de vista técnico, este problema integró exitosamente los conceptos de lectura digital, almacenamiento no volátil, comunicación serial (UART) e interfaz hombre-máquina, evidenciando la capacidad del microcontrolador para coordinar múltiples periféricos mediante distintos protocolos de comunicación. En particular, se implementó el protocolo SPI para la interacción con el módulo RC522, permitiendo una lectura eficiente y confiable de las etiquetas RFID, y el protocolo I2C para la gestión de la pantalla LCD, optimizando el uso de pines y simplificando el cableado del sistema. Asimismo, la correcta estructuración del código, junto con el uso modular de librerías dedicadas a cada periférico, contribuyó significativamente a la mantenibilidad, escalabilidad y robustez del programa.

VIII. APÉNDICES

IX. EVIDENCIAS

S. Modernell, V. Etcheverry, y G. Sosa, [https://github.com/Victoriaetch/Microcontroladores/tree/main/Laboratorios/Laboratorio%203]. [Accedido: Noviembre, 2025]

REFERENCIAS

[1] Arduino, "Arduino Uno Rev3 – Reliable ATmega328P Board with Digital & Analog I/O," *Arduino Store*. [Online]. Available: <https://store-usa.arduino.cc/products/arduino-uno-rev3>. [Accessed: Sep. 26, 2025].

[2] Wikipedia, "Atmega328," *Wikipedia*, 20-Mar-2024. [Online]. Available: <https://es.wikipedia.org/w/index.php?title=Atmega328&oldid=158934266>. [Accessed: Sep. 20, 2025].

[3] "C (lenguaje de programación)," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: [https://es.wikipedia.org/wiki/C_\(lenguaje_de_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/C_(lenguaje_de_programaci%C3%B3n)).

[4] Python Software Foundation, "Welcome to Python.org." [En línea]. Disponible: <https://www.python.org/>. [Accedido: 5 de noviembre de 2025].

[5] Cadence PCB Solutions, "USART vs. UART: What's the Difference?," *Cadence PCB Design & Analysis*, 8 de febrero de 2023. [Enlace](#). [Accedido: 26-sep-2025].

[6] R. J. H., "AVR UART Tutorial," RJH Coding. Available: <http://www.rjhcoding.com/avrc-uart.php>. [Accessed: 10-oct-2025].

[7] Python and Turtle, "Python Turtle - Online." Available: <https://pythonandturtle.com/turtle/>. [Accessed: 16-oct-2025].

[8] "I²C," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/I%C2%B2C>.

[9] "Serial Peripheral Interface," *Wikipedia, la enciclopedia libre*. 8 de junio de 2025. [En línea]. Disponible: https://es.wikipedia.org/wiki/Serial_Peripheral_Interface. [Accedido: 5 de noviembre de 2025].

[10] "EEPROM," *Wikipedia, la enciclopedia libre*. [En línea]. Disponible: <https://es.wikipedia.org/wiki/EEPROM>.

[11] "¿Qué es PWM y cómo usarlo?," *Solectroshop Blog*, 26 de agosto de 2020. [En línea]. Disponible: <https://solectroshop.com/es/blog/que-es-pwm-y-como-usarlo--n38>. [Accedido: 5 de noviembre de 2025].

[12] "Temperature Control Lab," *Dynamics and Control, APMonitor.com*. 9 de octubre de 2025. [En línea]. Disponible: <https://apmonitor.com/pdc/index.php/Main/ArduinoTemperatureControl>. [Accedido: 5 de noviembre de 2025].

[13] "3110KL-05W-B69-B00 NMB | Mouser México," *Mouser.mx*. [En línea]. Disponible: <https://www.mouser.mx/ProductDetail/NMB/3110KL-05W-B69-B00?qs=XN%252Belh2ZYYBw7Rd1BOiBg%3D%3D>. [Accedido: 5 de noviembre de 2025].

[14] Talos Electronics. (s.f.). Puente H Doble L298N. Recuperado el 21 de junio de 2025, de <https://www.taloselectronics.com/blogs/tutoriales/puente-h-doble-l298n>

[15] "Potenciómetro," *Wikipedia, la enciclopedia libre*. 5 de septiembre de 2025. [En línea]. Disponible: <https://es.wikipedia.org/wiki/Potenci%C3%B3metro>. [Accedido: 5 de noviembre de 2025].

[16] "Motor de corriente continua," *Wikipedia, la enciclopedia libre*. 13 de mayo de 2025. [En línea]. Disponible: https://es.wikipedia.org/wiki/Motor_de_corriente_continua. [Accedido: 5 de noviembre de 2025].

[17] "How 2-Axis Joystick Works & Interface with Arduino + Processing," *LastMinuteEngineers.com*. [En línea]. Disponible: <https://lastminuteengineers.com/joystick-interfacing-arduino-processing/>. [Accedido: 6 de noviembre de 2025].

[18] "Getting Started With NeoPixel WS2812 RGB LED," *Instructables*. [En línea]. Disponible: <https://www.instructables.com/Getting-Started-With-NeoPixel-WS2812-RGB-LED/>.

[19] Paguayo, "¿Qué es RFID? ¿Cómo funciona? Interfaz del módulo RC522 RFID con Arduino," *MCI Educación*, 5 de enero de 2023. [En línea]. Disponible: <https://cursos.mcielectronics.cl/2023/01/05/que-es-rfid-como-functiona-interfaz-del-modulo-rc522-rfid-con-arduino/>. [Accedido: 6 de noviembre de 2025].

[20] "Arduino 1602 Character LCD Tutorial," *Last Minute Engineers*. [En línea]. Disponible: <https://lastminuteengineers.com/arduino-1602-character-lcd-tutorial/>.

[21] "I2C Interfaz LCD 1602 (PCF8574)," *Uelectronics*. [En línea]. Disponible: <https://uelectronics.com/producto/i2c-interfaz-lcd-1602-pcf8574/>. [Accedido: 6 de noviembre de 2025].

[22] Admin, "RFID Based Door Lock System using AVR," *MaxPhi*, 2 de septiembre de 2017. [En línea]. Disponible: <https://www.maxphi.com/rfid-based-door-lock-system-using-avr>. [Accedido: 6 de noviembre de 2025].

[23] I. Ayyub, "How to interface RFID with AVR ATmega32 microcontroller," *atmega32-avr.com*, 1 de mayo de 2017. [En línea]. Disponible: <https://atmega32-avr.com/interface-rfid-avr-atmega32-microcontroller/>. [Accedido: 6 de noviembre de 2025].